

Multitasking with Raspberry Pi



Multitasking with Raspberry Pi



Dogan Ibrahim



elektor

LEARN > DESIGN > SHARE

● This is an Elektor Publication. Elektor is the media brand of
Elektor International Media B.V.
78 York Street, London W1H 1DP, UK
Phone: (+44) (0)20 7692 8344

● All rights reserved. No part of this book may be reproduced in any material form, including photocopying, or storing in any medium by electronic means and whether or not transiently or incidentally to some other use of this publication, without the written permission of the copyright holder except in accordance with the provisions of the Copyright Designs and Patents Act 1988 or under the terms of a licence issued by the Copyright Licensing Agency Ltd., 90 Tottenham Court Road, London, England W1P 9HE. Applications for the copyright holder's permission to reproduce any part of the publication should be addressed to the publishers.

● Declaration

The author and publisher have used their best efforts in ensuring the correctness of the information contained in this book. They do not assume, or hereby disclaim, any liability to any party for any loss or damage caused by errors or omissions in this book, whether such errors or omissions result from negligence, accident or any other cause..

● British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

● **ISBN 978-1-907920-96-7**

© Copyright 2020: Elektor International Media b.v.

Prepress Production: D-Vision, Julian van den Berg

First published in the United Kingdom 2020



Elektor is part of EIM, the world's leading source of essential technical information and electronics products for pro engineers, electronics designers, and the companies seeking to engage them. Each day, our international team develops and delivers high-quality content - via a variety of media channels (including magazines, video, digital media, and social media) in several languages - relating to electronics design and DIY electronics. www.elektormagazine.com

LEARN > DESIGN > SHARE

Preface	10
Chapter 1 • Installing the Raspberry Pi operating system	12
1.1 Overview	12
1.2 Raspbian Buster installation steps on Raspberry Pi 4	12
1.3 Remote access	15
1.4 Using Putty	16
1.4.1 Configuring the Putty	18
1.5 Remote access of the desktop	18
Chapter 2 • Using the Raspberry Pi command line	21
2.1 Overview	21
2.2 Command examples	21
2.3 Installing and removing software.	34
2.4 Shutting down.	34
2.5 Summary	35
Chapter 3 • Process management and resource monitoring on Raspberry Pi	36
3.1 Overview	36
3.2 Foreground and background processing	36
3.3 Task scheduling.	37
3.3.1 Task scheduling management	41
3.3.2 The crontab generator	41
3.4 Running a program or script automatically at system startup	43
3.4.1 Using /etc/rc.local	43
3.4.2 Using crontab	43
3.4.3 Using /etc/init.d.	44
3.5 Resource monitoring	45
3.5.1 Uptime and load average	45
3.5.2 Tasks	45
3.5.3 CPU utilization	46
3.5.4 Memory usage.	46
3.5.5 Process table.	46
3.5.6 Disk usage	49
3.5.7 CPU information	49

3.5.8 Memory use	50
3.5.9 Operating system information	50
3.5.10 USB devices	50
3.5.11 SD card information	51
3.5.12 CPU architecture information	51
3.6 Summary	51
Chapter 4 • Multiprocessing and multithreading	52
4.1 Overview	52
4.2 What is multithreading?	52
4.3 What is multiprocessing?	53
4.4 Differences between multithreading and multiprocessing	53
4.5 Task Scheduling algorithms.	54
4.5.1 Co-operative scheduling	54
4.5.2 Round-robin scheduling	56
4.5.3 Pre-emptive scheduling	57
4.5.4 Scheduling algorithm goals	58
4.5.5 Difference between preemptive and non-preemptive scheduling	59
4.5.6 Some other scheduling algorithms	59
4.5.7 Choosing a scheduling algorithm	60
4.6 Summary	60
Chapter 5 • Raspberry Pi multitasking projects - using the fork()	61
5.1 Overview	61
5.2 Running shell commands from Python	61
5.3 Process forks	62
5.3.1 Project 1 – Two LEDs flashing at different rates	65
5.3.2 Project 2 – Four LEDs flashing at different rates	68
5.3.3 Project 3 – Setting the LED flashing rate from the keyboard	72
5.3.4 Project 4 – Multitasking event counter	74
5.3.5 Project 5 – LED flashing and LED control with a button.	76
5.3.6 Project 6 – Synchronizing the parent and child processes - multitasking event counter	78
5.3.7 Project 7 – Up/down counter.	80

5.4 Summary	83
Chapter 6 • Raspberry Pi multitasking projects - using threads	84
6.1 Overview	84
6.2 Threads	84
6.3 Forking or Threads?	85
6.4 Using threads	85
6.4.1 Project 1 – Two LEDs flashing at different rates	90
6.4.2 Project 2 – Up/down counter	93
6.4.3 Project 3 – Setting the LED flashing rate from the keyboard	97
6.4.4 Project 4 – Setting the LED flashing rate using a button	98
6.4.5 Project 5 – Two-digit 7-segment display seconds counter	101
6.4.6 Project 6 – Two digit 7-segment temperature display	112
6.4.7 Project 7 – Square waveform generator with 7-segment LED display and keyboard	116
6.4.8 Project 8 – Square waveform generator with 7-segment LED display and buttons	120
6.4.9 Project 9 – Four-digit 7-segment display seconds counter	126
6.4.10 Project 10 – Four-digit 7-segment display conveyor belt object counter	131
6.4.11 Project 11 – ON/OFF Temperature controller with LCD	136
6.5 Summary	145
Chapter 7 • Raspberry Pi multitasking projects - using threading	146
7.1 Overview	146
7.2 Threading	146
7.2.1 Thread lock objects	149
7.2.2 Semaphores	150
7.2.3 Events	151
7.2.4 Timer threading	153
7.3 Threading based projects	153
7.3.1 Project 1 – LED flashing counter	154
7.4 Summary	155
Chapter 8 • Using subprocesses	156
8.1 Overview	156

8.2 Subprocesses call	156
8.3 Subprocess run	156
8.4 Subprocess check_call	156
8.5 Subprocess check_output	157
8.6 Subprocess Popen and communicate	157
8.7 Running a Python program	157
8.8 Project 1 – Two LEDs flashing at different rates.	158
8.9 Summary	160

Chapter 9 • Raspberry Pi multitasking projects - using multiprocessing161

9.1 Overview	161
9.2 Multiprocessing or threading?	161
9.3 How many CPU cores?	162
9.4 Multiprocessing process calls.	162
9.5 Using Events in multiprocessing	163
9.6 Conditions in multiprocessing	163
9.7 Multiprocessing Queues	164
9.8 Sharing data in multiprocessing using Value and Array.	164
9.9 Anonymous Pipes in multiprocessing	164
9.10 Named Pipes	164
9.11 Signals in multiprocessing.	164
9.12 Multiprocessing based projects	164
9.12.1 Project 1 - Two LEDs flashing at different rates	164
9.12.2 Project 2 – Setting the LED flashing rate from the keyboard	166
9.12.3 Project 3 - ON/OFF Temperature controller	167
9.12.4 Project 4 - Metronome	171
9.12.5 Project 5 – Traffic lights controller	175
9.12.6 Project 6 – Ultrasonic car parking aid with buzzer	181
9.12.7 Project 7 – Reaction timer.	188
9.12.8 Project 8 – Stepper motor controller with keyboard	191
9.12.9 Project 9 – Setting the flashing rate of an LED with keypad.	202
9.12.10 Project 10 – Secure door lock with keypad	210
9.12.11 Project 11 – Car park control.	214

Appendix A • List of components used in the book	228
Appendix B • Raspberry Pi 4 pin configuration	229
Index	230

Preface

A microcontroller is a single-chip microprocessor system which contains data and program memory, serial and parallel I/O, timers, external and internal interrupts, all integrated into a single chip that can be purchased for as little as \$2.00. About 40% of microcontroller applications are in office automation, such as PCs, laser printers, fax machines, intelligent telephones, and so forth. About one-third of microcontrollers are found in consumer electronic goods. Products like CD and DVD players, hi-fi equipment, video games, washing machines, cookers and so on fall into this category. The communications, automotive, and military markets share the rest of the application areas.

Microcontrollers have traditionally been programmed using the assembly language of the target processor. Although assembly language is fast, it has a major disadvantage in that it is difficult to develop and maintain large projects. Additionally, microcontrollers from different manufacturers use different assembly language instruction sets making it very time consuming for the programmers to learn new languages every time a different microcontroller is to be used. Assembly code developed for one type of microcontroller cannot be ported to another type of microcontroller. Nowadays microcontrollers are programmed using high-level languages, such as C, C++, Pascal, Python, or Basic. Perhaps the biggest advantage of using a high-level language is that developed code can easily be ported to other types of microcontrollers. Additionally, it is easier to maintain a program developed using a high-level programming language.

The topic of this book is multitasking. Multitasking has become one of the most important topics in microcontroller-based systems, namely in automation applications. As the complexity of the projects grows, more functionality is demanded from the projects and such projects require the use of several inter-related tasks running on the same system and sharing the CPU (or multiple CPUs) to implement the required operations. As a result of this, the importance of multitasking operation in microcontroller-based applications has been steadily growing over the last few years. Many complex automation projects nowadays make use of some form of a multitasking kernel. In this book, the Python 3 programming language is used with the Raspberry Pi 4. Other models of Raspberry Pi can also be used without any change to the code.

This book is project-based. Its main aim is to teach the basic features of multitasking using Python on Raspberry Pi. Many fully tested projects are given in the book using the multitasking modules of Python. Each project is fully described and in detail. Complete program listings are provided for each project. Readers should be able to use the projects as they are, or modify them to suit their own needs.

The following Python multitasking modules have been described and used in the projects:

- Fork
- Thread
- Threading
- Subprocess
- Multiprocessing

Knowledge of the Python programming language will be useful to the readers. Also, familiarity with at least one model of the Raspberry Pi computer will be an advantage. The knowledge of assembly language programming is not required because all the projects in the book are based on using the Python language.

This book is written for students, for practising engineers, and for hobbyists interested in developing multitasking projects using the Python 3 programming language on the Raspberry Pi computer.

I hope you like reading the book and base your next Raspberry Pi project on multitasking.

Dogan Ibrahim
London, 2020

Chapter 1 • Installing the Raspberry Pi operating system

1.1 Overview

In this chapter, we will learn how to install the latest Raspberry Pi operating system, Raspbian Buster, on an SD card for the Raspberry Pi 4. Advanced users can skip this chapter if they already have installed the latest operating system.

1.2 Raspbian Buster installation steps on Raspberry Pi 4

Raspbian Buster is the latest operating system of the Raspberry Pi 4. This section gives the steps for installing this operating system on a new blank SD card, ready to use with your Raspberry Pi 4. You will need a micro SD card with a capacity of at least 8GB (16GB is even better and is the recommended size) before installing the new operating system.

The steps to install the Raspbian Buster are as follows:

- Download the Buster image to a folder on your PC (e.g. C:\RPIBuster) from the following link by clicking the Download ZIP under section **Raspbian Buster with desktop and recommended software** (see Figure 1.1). At the time of writing this book, the file was called: **2019-07-10-raspbian-buster-full.img**. You may have to use 7Zip to unzip the download because some of the features are not supported by older unzip software.

<https://www.raspberrypi.org/downloads/raspbian/>

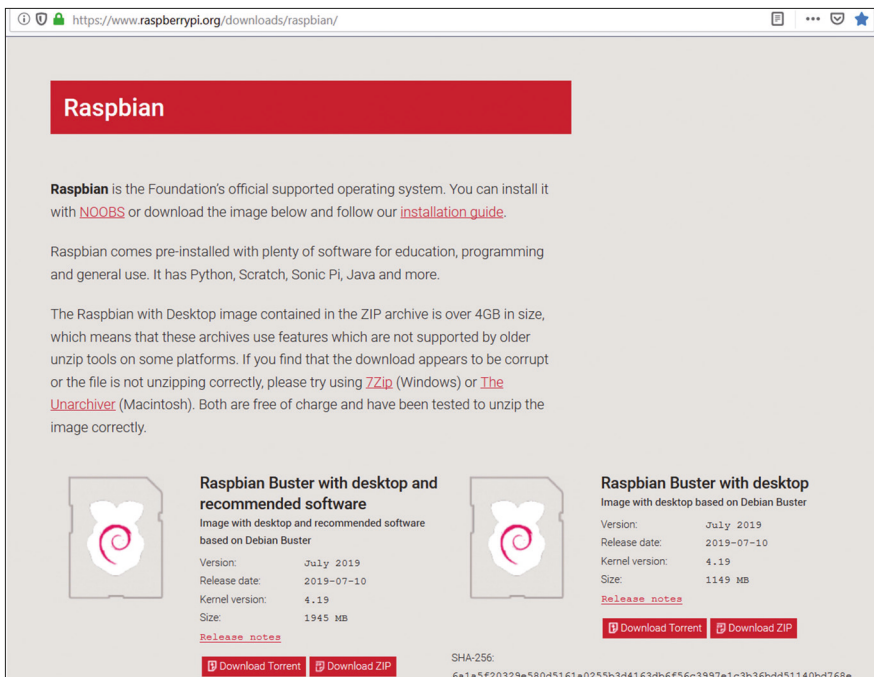


Figure 1.1 Raspbian Buster download page

- Put your blank micro SD card into the card slot on your computer. You may need to use an adapter.
- Download the Etcher program on your PC to flash the disk image. The link is (see Figure 1.2):

<https://www.balena.io/etcher/>

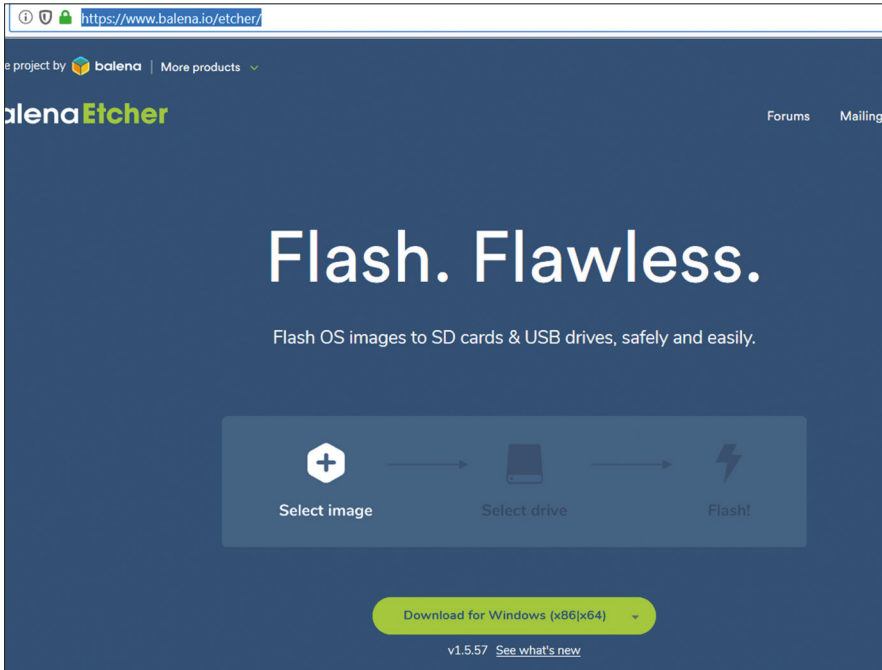


Figure 1.2 Download Etcher

- Double click to open Etcher, and click **Select image**. Select the Raspbian Buster file you just downloaded.
- Click **Select target** and select the micro SD card.
- Click **Flash** (see Figure 1.3). This may take several minutes. Wait until it is finished. The program will then validate and unmount the micro SD card. You can remove your micro SD card after it is unmounted.

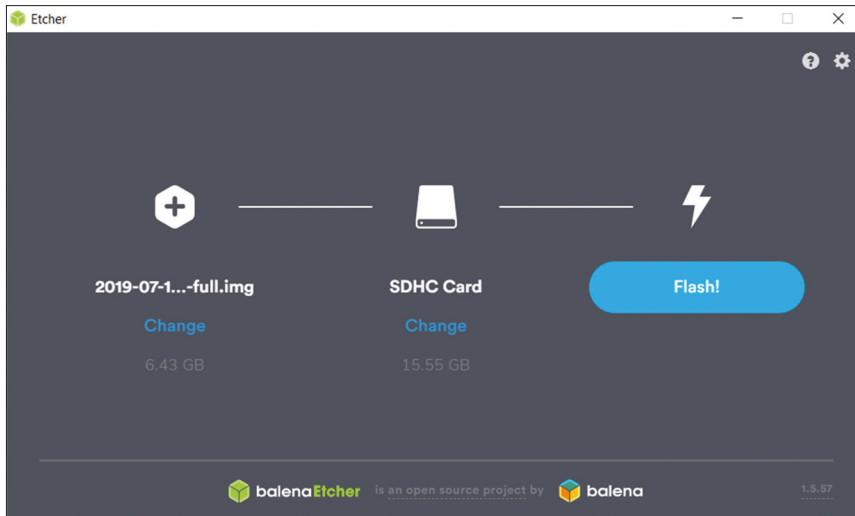


Figure 1.3 Click Flash to flash the disk image

- You are now ready to use your micro SD card on your Raspberry Pi 4.
- Connect your Raspberry Pi 4 to a HDMI monitor (you may need to use an adapter cable for mini HDMI to standard HDMI conversion), connect a USB keyboard, and power up the Raspberry Pi.
- You will see the startup menu displayed on the monitor. Click Next to get started.
- Select the Wi-Fi network and enter the password of your Wi-Fi router
- Click on the Wi-Fi icon on the top right-hand side of the screen and note the Wireless IP address of your Raspberry Pi (notice the IP address is not static and it can change next time you power-up your Raspberry Pi).
- You should now be ready to use your Raspberry Pi 4 (see Desktop in Figure 1.4)

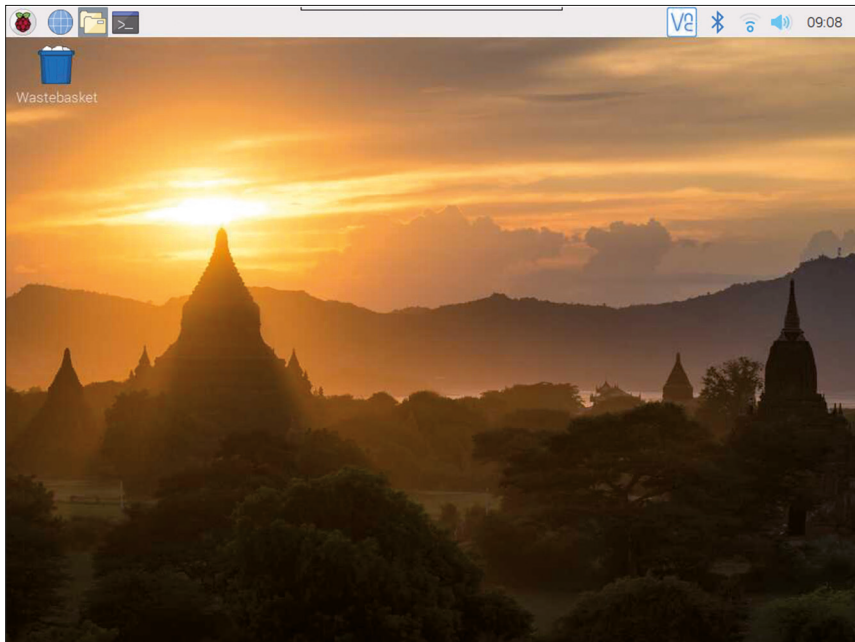


Figure 1.4 Raspberry Pi 4 desktop

Notice that the IP address of your Raspberry Pi can also be seen in your router. You can also get the IP address of your Raspberry Pi using your mobile phone. Several programs can be installed on your mobile phone to show you the IP addresses of all the devices connected to your router. In this section, the use of the Android **app** called **Who's On My Wi-Fi – Network Scanner** by *Magdalm* is used to show how the IP address of your Raspberry Pi can be displayed. Running this program will display the Raspberry Pi Wireless IP address under the heading **Raspberry Pi Trading Ltd**. In addition to the IP address, other parameters such as the MAC address, gateway address, IP mask, etc are all displayed by this program.

1.3 Remote access

It is much easier to access the Raspberry Pi remotely over the internet, for example using a PC rather than connecting a keyboard, mouse, and display to it. Before being able to access the Raspberry Pi remotely, we have to enable SSH and VNC by entering the following command on a terminal session:

```
pi$raspberrypi:~ $ sudo raspi-config
```

Go to the configuration menu and select **Interface Options**. Go down to **P2 SSH** (see Figure 1.5) and enable SSH. Click **<Finish>** to exit the menu.

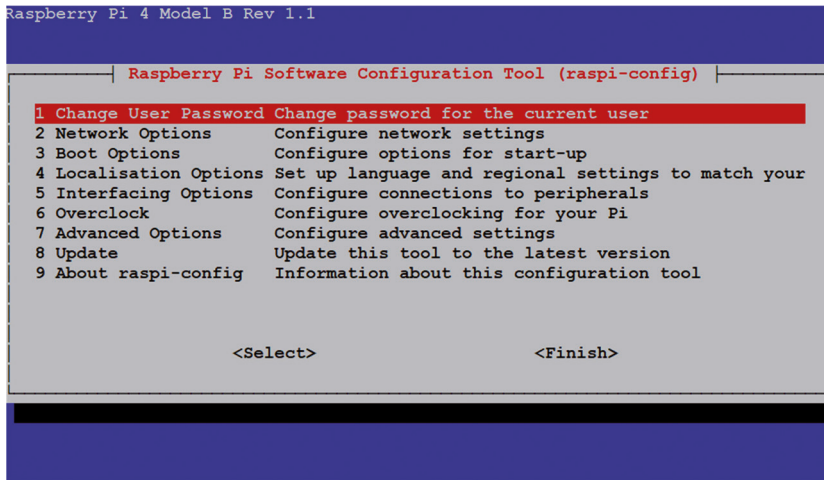


Figure 1.5 Enable SSH

You should also enable VNC so that the Raspberry Pi can be accessed graphically over the internet. This can be done by entering the following command on a terminal session:

```
pi$raspberrypi:~ $ sudo raspi-config
```

Go to the configuration menu and select **Interface Options**. Go down to **P3 VNC** and enable VNC. Click **<Finish>** to exit the menu. At this stage, you may want to shut down your Raspberry Pi by clicking the **Applications Menu** on the desktop and selecting **Shutdown**

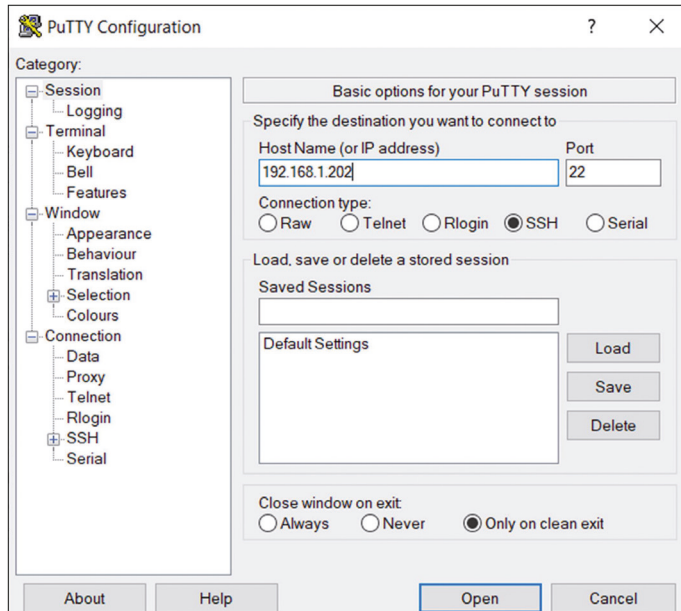
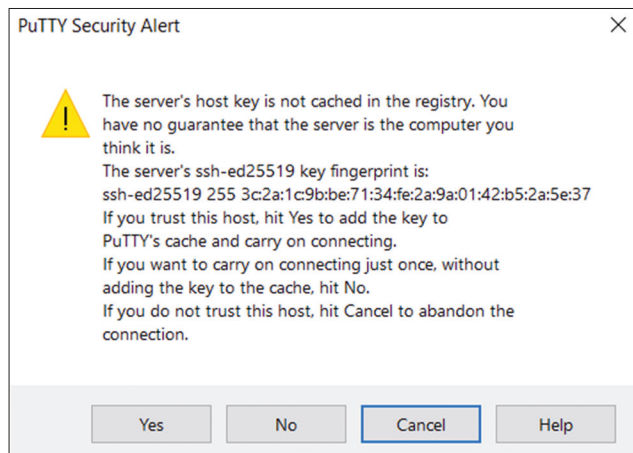
1.4 Using Putty

Putty is a communications program used to create a connection between your PC and the Raspberry Pi. This connection uses a secure protocol called SSH (Secure Shell). Putty doesn't need to be installed as it can be stored in any folder of your choice and run from there.

Putty can be downloaded from the following web site:

<https://www.putty.org/>

Simply double click to run it and the Putty startup screen will be displayed. Click **SSH** and enter the Raspberry Pi IP address, then click **Open** (see Figure 1.6). The message shown in Figure 1.7 will be displayed the first time you access the Raspberry Pi. Click **Yes** to accept this security alert.

*Figure 1.6 Putty startup screen**Figure 1.7 Click Yes to accept*

You will be prompted to enter the username and password. Notice that the default username and password are:

username: **pi**
password: **raspberrypi**

You now have a terminal connection with the Raspberry Pi and you can type in commands, including super user **sudo** commands. You can use the cursor keys to scroll up and down through the commands you've previously entered in the same session. You can also run programs (not graphical).

1.4.1 Configuring the Putty

By default, the Putty screen background is black with white foreground characters. In this book, we use a white background with black foreground characters, with the character size set to 12 points in bold. The steps to configure Putty with these settings are given below. Notice that in this example, the settings are saved with the name **RPI4** so that they can be recalled whenever Putty is restarted:

- Restart Putty
- Select **SSH** and enter your Raspberry Pi IP address
- Click **Colours** under **Window**
- Set the **Default Foreground** and **Default Bold Foreground** colours to black (Red:0, Green:0, Blue:0)
- Set the **Default Background** and **Default Bold Background** to white (Red:255, Green:255, Blue:255)
- Set the **Cursor Text** and **Cursor Colour** to black (Red:0, Green:0, Blue:0)
- Select **Appearance** under **Window** and click **Change** in **Font settings**. Set the font to **Bold 12**.
- Select **Session** and give a name to the session (e.g. RPI4) and click **Save**.
- Click **Open** to open Putty with the saved configuration
- Next time you restart Putty, select the saved session and click **Load** followed by **Open** to start a session with the saved configuration

1.5 Remote access of the desktop

You can control your Raspberry Pi via Putty, and run programs on it from your Windows PC. However, this will not work with graphical programs because Windows doesn't know how to represent the display. As a result of this, for example, we cannot run any graphical programs in Desktop mode. We can get around this problem using some additional software. Two popular programs are VNC (Virtual Network Connection), and Xming. Here, we will learn how to use VNC.

Installing and using VNC

VNC consists of two parts: VNC Server and VNC Viewer. VNC Server runs on the Raspberry Pi, and VNC Viewer runs on a PC. VNC server is already installed on your Raspberry Pi and should be enabled using the **raspi-config** tool. After enabling it, VNC server can start on the Raspberry Pi by entering the following in command mode:

```
pi$raspberrypi:~ $ vncserver :1
```

The steps to install and use VNC Viewer on your PC are provided below:

- There are many VNC Viewers, but the one I recommend is TightVNC which can be downloaded from the following web site:

<https://www.tightvnc.com/download.php>

- Download and install **TightVNC** for your PC. You will have to choose a password during the installation.
- Start the **TightVNC Viewer** on your PC and enter the Raspberry Pi IP address (see Figure 1.8) followed by :1. Click **Connect** to connect to your Raspberry Pi.

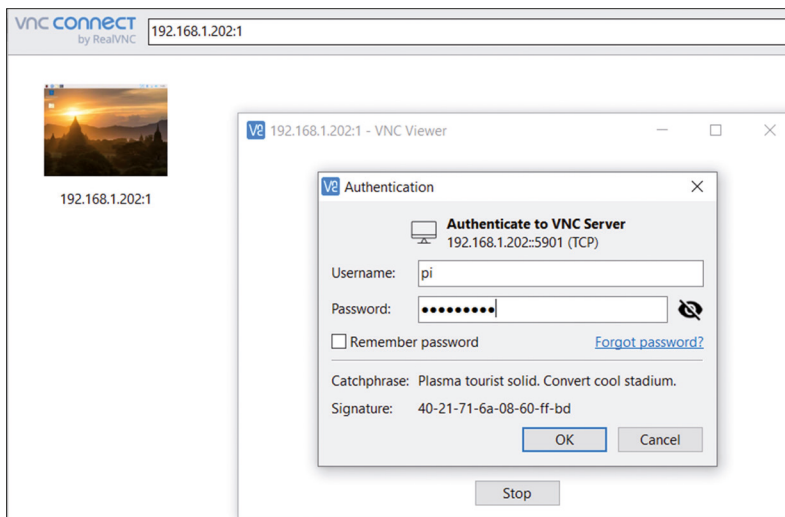


Figure 1.8 Start the TightVNC and enter the IP address

Figure 1.9 shows the Raspberry Pi Desktop displayed on the PC screen.

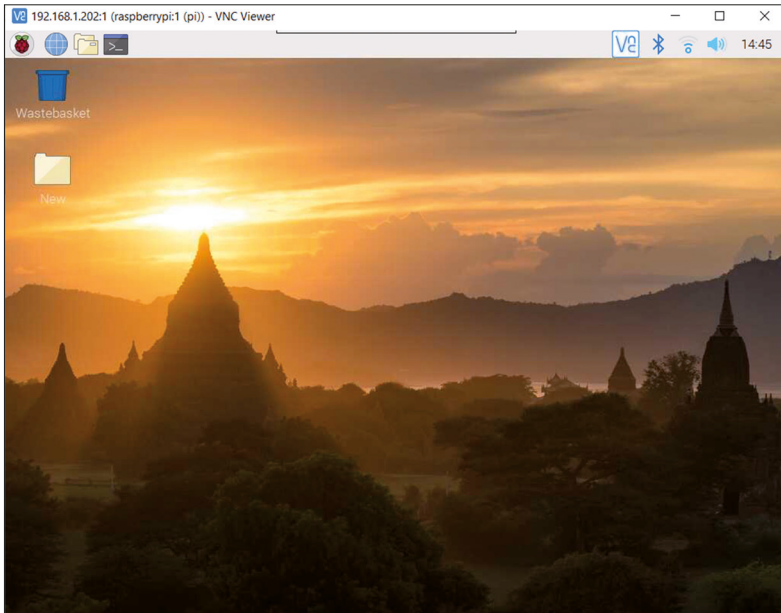


Figure 1.9 Raspberry Pi Desktop on the PC screen

1.6 Summary

In this chapter, we learned how to install the latest Raspberry Pi operating system on an SD card. Now that the software has been installed and our Raspberry Pi is working, in the next chapter, we will look at some important commands of the Raspberry Pi operating system.

Chapter 2 • Using the Raspberry Pi command line

2.1 Overview

Linux is one of the most popular operating systems in use today. It is very similar to other operating systems, such as Windows and UNIX. Linux is an open operating system based on UNIX and has been developed collaboratively by many companies since 1991. In general, Linux is harder to manage than some other operating systems like Windows but offers more flexibility and configuration options. There are several popular versions of the Linux operating system such as Debian, Ubuntu, Red Hat, Fedora, and so on.

Linux instructions are text-based. In this chapter, we will look at some useful Linux commands and discover how to manage your Raspberry Pi. Commands entered by the user are in bold for clarity.

2.2 Command examples

Current directory

To show the current working directory, enter the command:

```
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $
```

Directory structure

To show the directory structure, enter the command:

```
pi@raspberrypi:~ $ ls /
bin dev home lost+found mnt proc run srv tmp var
boot etc lib media opt root sbin sys usr
pi@raspberrypi:~ $
```

To show the subdirectories and files in our working directory, enter:

```
pi@raspberrypi:~ $ ls
book Documents MagPi Music Pictures Scratch Videos
Desktop Downloads _mu_code mytest.txt Public Templates
pi@raspberrypi:~ $
```

Notice above that the subdirectories are displayed in blue. There is a text file in the working directory called **mytest.txt** which is displayed in black.

Creating a subdirectory

Lets create a new subdirectory called **myfiles** in our working directory:

```
pi@raspberrypi:~ $ mkdir myfiles
pi@raspberrypi:~ $ ls
bok Documents MagPi Music mytest.txt Public Templates
Desktop Downloads _mu_code myfiles Pictures Scratch Videos
pi@raspberrypi:~ $
```

As shown in Table 2.1, the **ls** command can take several arguments. Some examples are given below.

To display the subdirectories and files in a single row:

```
pi@raspberrypi:~ $ ls -1
book
Desktop
Documents
Downloads
MagPi
mu_code
Music
myfiles
mytest.txt
Pictures
Public
Scratch
Templates
Videos
pi@raspberrypi:~ $
```

To display the file type, enter the following command. Note that directories have a "/" after their names, and executable files have a "*" character after their names:

```
pi@raspberrypi:~ $ ls -F
book/   Documents/ MagPi/   Music/   mytest.txt Public/  Templates/
Desktop/ Downloads mu_code/ myfiles/ Pictures/ Scratch/ Videos/
pi@raspberrypi:~ $
```

To list the results, separated by commas:

```
pi@raspberrypi:~ $ ls -m
book, Desktop, Documents, Downloads, MagPi, mu_code, Music, myfiles, mytest.txt,
Pictures, Public Scratch, Templates, Videos
pi@raspberrypi:~ $
```

We can mix the arguments as in the following example:

```
pi@raspberrypi:~ $ ls -m -F
book/, Desktop/, Documents/, Downloads/, MagPi/, mu_code/, Music/, myfiles/,
mytest.txt, Pictures/, Public/, Scratch/, Templates/, Videos/
pi@raspberrypi:~ $
```

Displaying file permissions

One of the important arguments used with the **ls** command is **-l** (lower case letter l) which displays the file permissions, file sizes, and when they were last modified. In the example below, each line relates to one directory or file. Reading from right to left, the name of the directory or the file is on the right-hand side. The date the directory or file was created is on the left-hand side of its name. Next comes the size, given in bytes. For example, file **mytest.txt** consists of 13 bytes. The characters at the beginning of each line are about the permissions. i.e. who is allowed to use or modify the file or the directory

The permissions are divided into 3 categories:

- What the user (or owner, or creator) can do – called USER
- What the group owner (people in the same group) can do - GROUP
- What everyone else can do – called WORLD

The first word **pi** in the example in Figure 2.1 shows who the user of the file (or directory) is, and the second word, **pi**, shows the group name that owns the file. In this example, both the user and the group names are **pi**.

```
pi@raspberrypi:~ $ ls -l
total 56
drwxr-xr-x 13 pi pi 4096 Sep 21  2019 book
drwxr-xr-x  2 pi pi 4096 Sep 18  2019 Desktop
drwxr-xr-x  3 pi pi 4096 Sep 18  2019 Documents
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Downloads
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 MagPi
drwxr-xr-x  6 pi pi 4096 Dec 19 10:49 mu_code
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Music
drwxr-xr-x  2 pi pi 4096 May 13  09:06 myfiles
-rw-r--r--  1 pi pi   13 May 13  09:03 mytest.txt
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Pictures
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Public
drwxr-xr-x  2 pi pi 4096 Sep 18  2019 Scratch
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Templates
drwxr-xr-x  2 pi pi 4096 Jul 10  2019 Videos
pi@raspberrypi:~ $
```

Figure 2.1 Directory listing with file permissions

The permissions can be analysed by breaking down the characters into four chunks: **File**, **User**, **Group**, and **World**. The first character for a file is **-** and for a directory, it is **d**. Next comes the permissions for the User, Group, and World. The permissions are as follows:

- Read permission (r): the permission to open and read a file or to list a directory
- Write permission (w): the permission to modify a file, or to delete or create a file in a directory

- Execute permission (x): the permission to execute the file (applies to executable files), or to enter a directory

Three letters, **rwX**, are used as a group and if there is no permission assigned then a "-" character is used.

As an example, considering the **Desktop** directory, we have the following permission codes:

drwxr-xr-x which translates to:

d: it is a directory
rwx: user (owner) can read, write, and execute
r-x: group can read and execute, but cannot write (e.g. create or delete)
r-x: world (everyone else) can read and execute, but cannot write

As another example, let's look at the permissions for file **mytest.txt**:

-rw-r--r-- which translates to:

-: it is a file
rw-: user (owner) can read and write, but cannot execute (this is not an executable file)
r--: group can only read it, they cannot modify, delete, or execute the file
r--: everyone else (world) can only read it, they cannot modify, delete, or execute the file

Argument	Description
-1	Output results in a single column
-F	Put a symbol after the name to indicate directories and executable files
-l	Output in long format, including file permissions
-m	Output results separated by commas
-R	Output contents of all subdirectories as well
-S	Sort the output by size
-t	Sort the output according to date and time of last modification
-a	Output all files, including hidden files

Table 2.1 ls command arguments

Changing file permissions

The **chmod** command is used to change the file permissions. Before going into details of how to change the permissions, let us look and see what arguments are available in **chmod** for changing the file permissions.

The available arguments for changing file permissions are given below. We can use these arguments to add/remove or explicitly set permissions. It is important to realize that if we explicitly set permissions, any unspecified permissions in the command will be revoked:

```

u:      user (or owner)
g:      group
o:      other (world)
a:      all

+:      add
-:      remove
=:      set

r:      read
w:      write
x:      execute

```

To change the permissions of a file, we type the **chmod** command, followed by one of the letters **u**, **g**, **o**, or **a**, followed by the **+** - or **=** to select the type of change, and finally followed by the filename. An example is given in Figure 2.2. In this example, file **mytest.txt** has the user read and write permissions. We will be changing the permissions so that the user does not have read permission on this file as shown in Figure 2.3.

```

pi@raspberrypi:~ $ ls -lh
total 56K
drwxr-xr-x 13 pi pi 4.0K Sep 21 2019 book
drwxr-xr-x  2 pi pi 4.0K Sep 18 2019 Desktop
drwxr-xr-x  3 pi pi 4.0K Sep 18 2019 Documents
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Downloads
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 MagPi
drwxr-xr-x  6 pi pi 4.0K Dec 19 10:49 mu_code
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Music
drwxr-xr-x  2 pi pi 4.0K May 13 09:06 myfiles
-rw-r--r--  1 pi pi 13 May 13 09:03 mytest.txt
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Pictures
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Public
drwxr-xr-x  2 pi pi 4.0K Sep 18 2019 Scratch
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Templates
drwxr-xr-x  2 pi pi 4.0K Jul 10 2019 Videos
pi@raspberrypi:~ $ █

```

Figure 2.2 Displaying file permissions

```

pi@raspberrypi:~ $ chmod u-r mytest.txt
pi@raspberrypi:~ $ ls -lh
total 56K
drwxr-xr-x 13 pi pi 4.0K Sep 21 2019 book
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Desktop
drwxr-xr-x 3 pi pi 4.0K Sep 18 2019 Documents
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Downloads
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 MagPi
drwxr-xr-x 6 pi pi 4.0K Dec 19 10:49 mu_code
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Music
drwxr-xr-x 2 pi pi 4.0K May 13 09:06 myfiles
--w-r--r-- 1 pi pi 13 May 13 09:03 mytest.txt
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Pictures
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Public
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Scratch
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Templates
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Videos
pi@raspberrypi:~ $ █

```

Figure 2.3 Changing the file permission of file `mytest.txt`

Notice that if we now try to display the contents of file **mytest.txt** using the **cat** command we will get an error message:

```

pi@raspberrypi:~ $ cat mytest.txt
cat: mytest.txt: Permission denied
pi@raspberrypi:~ $

```

All the permissions can be removed from a file by the command **chmod a=<filename>** as shown in Figure 2.4.

```

pi@raspberrypi:~ $ chmod a= mytest.txt
pi@raspberrypi:~ $ ls -lh
total 56K
drwxr-xr-x 13 pi pi 4.0K Sep 21 2019 book
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Desktop
drwxr-xr-x 3 pi pi 4.0K Sep 18 2019 Documents
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Downloads
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 MagPi
drwxr-xr-x 6 pi pi 4.0K Dec 19 10:49 mu_code
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Music
drwxr-xr-x 2 pi pi 4.0K May 13 09:06 myfiles
----- 1 pi pi 13 May 13 09:03 mytest.txt
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Pictures
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Public
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Scratch
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Templates
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Videos
pi@raspberrypi:~ $ █

```

Figure 2.4 Removing permissions from file `mytest.txt`

In the example in Figure 2.5, **rwX** user permissions are given to file **mytest.txt**.

```

pi@raspberrypi:~ $ chmod u+rx mytest.txt
pi@raspberrypi:~ $ ls -lh
total 56K
drwxr-xr-x 13 pi pi 4.0K Sep 21 2019 book
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Desktop
drwxr-xr-x 3 pi pi 4.0K Sep 18 2019 Documents
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Downloads
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 MagPi
drwxr-xr-x 6 pi pi 4.0K Dec 19 10:49 mu_code
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Music
drwxr-xr-x 2 pi pi 4.0K May 13 09:06 myfiles
-rwx----- 1 pi pi 13 May 13 09:03 mytest.txt
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Pictures
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Public
drwxr-xr-x 2 pi pi 4.0K Sep 18 2019 Scratch
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Templates
drwxr-xr-x 2 pi pi 4.0K Jul 10 2019 Videos
pi@raspberrypi:~ $ █

```

Figure 2.5 Giving user permissions to file *mytest.txt*

To change the working directory

To change our working directory to **myfiles**, enter the following command:

```

pi@raspberrypi:~ $ cd /home/pi/myfiles
pi@raspberrypi:~/myfiles $

```

to go up one directory, i.e. to our default working directory:

```

pi@raspberrypi:~/myfiles $ cd..
pi@raspberrypi:~ $

```

to change our working directory to **myfiles**, we can also enter the command:

```

pi@raspberrypi:~ $ cd ~/myfiles
pi@raspberrypi:~/myfiles $

```

to go back to the default working directory, we can enter:

```

pi@raspberrypi:~/myfiles $ cd ~
pi@raspberrypi:~ $

```

to find out more information about a file we can use the `file` command. For example:

```

pi@raspberrypi:~ $ file myfile.txt
myfile.txt: ASCII text
pi@raspberrypi:~ $

```

The **-R** argument of command **ls** lists all the files in all the subdirectories of the current working directory. An example is given below. Notice here that there are no files in subdirectory **myfiles**:

```
pi@raspberrypi:~ $ ls -R
.:
book    Documents MagPi    Music  mytest.txt Public  Templates
Desktop Downloads mu_code  myfiles Pictures  Scratch Videos

./book:
04
.....
.....
.....
pi@raspberrypi:~ $
```

To display information on how to use a command, we can use the **man** command. As an example, to get assist on using the **mkdir** command:

```
pi@raspberrypi ~$ man mkdir
MKDIR(1)

NAME
    Mkdir – make directories

SYNOPSIS
    Mkir [OPTION]...DIRECTORY...

DESCRIPTION
    Create the DIRECTORY(ies), if they do not already exist.

    Mandatory arguments to long options are mandatory for short options

    -m, --mode=MODE          Set file mode (as in chmod), not a=rwx – umask
    .....
    .....
```

Help

The **man** command usually gives several pages of information on how to use a command. We can type **q** to exit the **man** command and return to the operating system prompt.

The **less** command can be used to display a long listing one page at a time. Using the up and down arrow keys, we can move between pages. An example is given below. Type **q** to exit:

```
pi@raspberrypi:~ $ man ls | less
<display of help on using the ls command>
pi@raspberrypi:~ $
```

Date, time, and calendar

To display the current date and time:

```
pi@raspberrypi:~ $ date
wed 13 May 25 20:50:46 BST 2020
pi@raspberrypi:~ $
```

The current calendar (month and year) can be displayed by the command:

```
pi@raspberrypi:~ $ cal
May 2020
Su   Mo   Tu   We   Th   Fr   Sa
 1    2    3    4    5    6    7
 8    9   10   11   12   13   14
15   16   17   18   19   20   21
22   23   24   25  26   27   28
29   30
pi@raspberrypi:~ $
```

Copying a file

To make a copy of a file, use the command **cp**. In the following example, a copy of file **mytest.txt** is made and the new file is given the name **another.txt**:

```
pi@raspberrypi:~ $ cp mytest.txt another.txt
pi@raspberrypi:~ $
```

If we list all the files in our working directory we should see the new file listed:

```
pi@raspberrypi:~ $ ls
another.txt  Documents  mu_code   mytest.txt  Scratch
book         Downloads  Music     Pictures    Templates
Desktop      MagPi      myfiles   Public      Videos
pi@raspberrypi:~ $
```

Copying a directory with its contents

The **-r** option of the **cp** command copies a directory with all of its contents. An example is shown below where directory **Documents** and its contents are copied to a new directory called **NewDocuments**:

```
pi@raspberrypi:~ $ ls Documents
hello.sh  Hello.sh.odt
pi@raspberrypi:~ $ cp -r Documents NewDocuments
pi@raspberrypi:~ $ ls NewDocuments
hello.sh  Hello.sh.odt
pi@raspberrypi:~ $
```

Wildcards

We can use wildcard characters to select multiple files with similar characteristics. e.g. files having the same file-extension names. The "*" character is used to match any number of characters. Similarly, the "?" character is used to match any single character. In the example below all the files with extensions **.txt** are listed:

```
pi@raspberrypi:~ $ ls *.txt
another.txt  mytest.txt
pi@raspberrypi:~ $
```

The wildcard characters [a-z] can be used to match any single character in the specified character range. An example is given below which matches any files that start with letters o, p, q, r, s, and t, and with the **.png** extension:

```
pi@raspberrypi:~ $ ls [a-d]*.txt
another.txt
pi@raspberrypi:~ $
```

Renaming a file

You can rename a file using the **mv** command. In the example below, the name of file **another.txt** is changed to **new.txt**:

```
pi@raspberrypi:~ $ mv another.txt new.txt
pi@raspberrypi:~ $
```

Deleting a file

The command **rm** can be used to remove (delete) a file:

```
pi@raspberrypi:~ $ rm new.txt
pi@raspberrypi:~ $
```

The argument **-v** can be used to display a message when a file is removed. Also, the **-i** argument asks for confirmation before a file is removed. In general, the two arguments are used together as **-vi**. An example is shown in Figure 2.6.

```
pi@raspberrypi:~ $ cp mytest.txt newfile.txt
pi@raspberrypi:~ $ rm -vi newfile.txt
rm: remove regular file 'newfile.txt'? y
removed 'newfile.txt'
pi@raspberrypi:~ $ █
```

Figure 2.6 Deleting a file with confirmation

Removing a directory

A directory can be removed using the **rmdir** command:

```
pi@raspberrypi:~ $ rmdir myfiles
pi@raspberrypi:~ $
```

Removing a directory with all of its contents

The **-r** option of command **rm** is used to delete a folder with all of its contents. An example is given below:

```
pi@raspberrypi:~ $ ls NewDocuments
hello.sh  Hello.sh.odt
pi@raspberrypi:~ $ rm -r NewDocuments
pi@raspberrypi:~ $ ls NewDocuments
ls: cannot access 'NewDocuments': No such file or directory
pi@raspberrypi:~ $
```

Re-directing the output

The greater sign **>** can be used to re-direct the output of a command to a file. For example, we can re-direct the output of the **date** command to a file called **curdate.txt**:

```
pi@raspberrypi:~ $ date > curdate.txt
pi@raspberrypi:~ $ cat curdate.txt
Wed 13 May 09:56:24 BST 2020
pi@raspberrypi:~ $
```

Using two greater signs **>>** adds to the end (appends) of a file. An example is shown in Figure 2.7.

```
pi@raspberrypi:~ $ cal >> curdate.txt
pi@raspberrypi:~ $ cat curdate.txt
Wed 13 May 09:56:24 BST 2020
      May 2020
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
pi@raspberrypi:~ $ █
```

Figure 2.7 Using command >>

Writing to the screen or file

The **echo** command can be used to write to the screen. It can be used to perform simple mathematical operations if the numbers and operation are enclosed in two brackets, preceded by a **\$** character:

```
pi@raspberrypi:~ $ echo $((5*6))
30
pi@raspberrypi:~ $
```

The **echo** command can also be used to write a line of text to a file. An example is shown below:

```
pi@raspberrypi:~ $ echo a line of text > lin.dat
pi@raspberrypi:~ $ cat lin.dat
a line of text
pi@raspberrypi:~ $
```

Matching a string

The **grep** command can be used to match a string in a file. An example is given below assuming that the file `lin.dat` contains a line of text. The matched word is shown in bold:

```
pi@raspberrypi:~ $ grep line lin.dat
a line of text
pi@raspberrypi:~ $
```

Head and tail commands

The **head** command can be used to display the first 10 lines of a file. The format of this command is as follows:

```
pi@raspberrypi:~ $ head mytest.txt
.....
.....
pi@raspberrypi:~ $
```

Similarly, the **tail** command is used to display the last 10 lines of a file. The format of this command is as follows:

```
pi@raspberrypi:~ $ tail mytest.txt
.....
.....
pi@raspberrypi:~ $
```

Super user commands

Some commands are privileged and only authorised persons can use them. Inserting the word **sudo** at the beginning of a command gives the authority to use the command without having to log in as an authorised user.

What software is installed on my Raspberry Pi

To find out what software is installed on your Raspberry Pi, enter the following command. You should get several pages of display:

```
pi@raspberrypi:~ $ dpkg --get-architecture
.....
.....
pi@raspberrypi:~ $
```

We can also find out if a certain software package is already installed on your computer. An example is given below which checks whether or not software called **xpdf** (PDF reader) is installed. In this example, **xpdf** is installed and the details of this software is displayed:

```
pi@raspberrypi:~ $ dpkg --status xpdf
Package: xpdf
Status: install ok installed
Priority: optional
Section: text
Installed-Size: 395
.....
.....
pi@raspberrypi:~ $
```

If the software is not installed we get a message similar to the following (assuming we are checking to see if a software package called **bbgd** is installed):

```
pi@raspberrypi:~ $ dpkg --status bbgd
```

dpkg-query: package 'bbgd' is not installed and no information is available

```
.....
.....
pi@raspberrypi:~ $
```

Command history

The command **history** displays a list of the commands entered on the terminal. As shown below, each history item has a number associated with it:

```
pi@raspberrypi:~ $ history
1s
2man scrot
3man find
4scrot -d
5ls -lh
6date
pi@raspberrypi:~ $
```

A command in the command history can be run using **!** followed by the command history number. For example, command 6 is run as follows:

```
pi@raspberrypi:~ $ !6
Wed 13 May 10:05:00 BST 2020
Pi@raspberrypi:~ $
```

2.3 Installing and removing software

The `apt-get` command is used for installing software from the command line. The basic format of this command is:

```
pi@raspberrypi:~ $ sudo apt-get install <name of software to be installed>
```

The list of software that can be installed is normally included in the Raspberry Pi operating system distribution. This soon becomes out of date. You should use the following update command to update the list before installing software:

```
pi@raspberrypi:~ $ sudo apt-get update
```

Sometimes we may want to remove an installed software. The remove option of the **apt-get** command can be used to remove software as in the following example:

```
pi@raspberrypi:~ $ sudo apt-get remove<name of software to be removed>
```

A list of the installed software packages can be displayed by entering the following command:

```
pi@raspberrypi:~ $ dpkg -l
```

2.4 Shutting down

Although you can disconnect the power supply from your Raspberry Pi when you finish working with it, it is not recommended since many processes are running on the system and it is possible to corrupt the file system. It is much better to shut down the system in an orderly manner.

The following command will stop all the processes and make the file system safe and then turn off the system safely:

```
pi@raspberrypi:~ $ sudo halt
```

The following command stops and then restarts the system:

```
pi@raspberrypi:~ $ sudo reboot
```

The system can also be shut down and then restarted after a time by entering the following command. Optionally, a shutdown message can be displayed if desired:

```
pi@raspberrypi:~ $ sudo shutdown -r <time> <message>
```

For quick shutdown of the system, enter the following command:

```
pi@raspberrypi:~ $ sudo shutdown now
```

2.5 Summary

This chapter has described the use of some important Linux commands. You should be able to get further information on each command and other Linux commands from the internet and other books on Raspberry Pi and Linux.

In the next chapter, we will look at system resource monitoring commands which are important for multitasking applications.

Chapter 3 • Process management and resource monitoring on Raspberry Pi

3.1 Overview

Process management and resource monitoring are important topics in multiprocessing and multitasking applications. Readers must be familiar with the various tools and commands available for efficient process management and resource monitoring. In this chapter, we will look at the commonly used tools and commands available for this purpose.

3.2 Foreground and background processing

All commands entered at the terminal run in the foreground where any output is displayed on the screen and at the same time the terminal is locked to the running program. There are some applications where we may want to run commands in the background so that the terminal is available for entering other commands. The outputs of the background processes cannot be seen on screen.

A command is forced to run in the background by appending the **&** character at the end of the command. A command that runs in the background is called a **job**.

In the following example, the **date** command runs in the background, creates a file called **curdate.txt**, and stores the current date in this file. Notice here that [1] is the background job number id, and 6309 is the background process id. **Done** indicate that the background process has completed and terminated:

```
pi@raspberrypi:~ $ date > curdate.txt &
[1] 6309
pi@raspberrypi:~ $ cat curdate.txt
Wed 13 May 13:52:09 BST 2020
[1]+  Done      date > curdate.txt
pi@raspberrypi:~ $
```

A list of the processes running in the background can be listed using command **jobs**. If no background processes are running, then nothing is displayed by the command:

```
pi@raspberrypi:~ $ jobs
```

We may sometimes want to bring a background process to the foreground. This can be done using the command **fg** followed by its job number id, provided that the background job is not terminated. For example, if it is required to run a background process with job number id [1] in the foreground, you should enter the following command:

```
pi@raspberrypi:~ $ fg 1
```

Similarly, in some applications, we may want to send a running foreground process to the background. This can be done by pressing **Cntrl+Z** keys (pressing the Cntrl and Z keys together).

A foreground process requires user input stops and waits until the user enters an input and the process then continues. A background process that requires user input also sits and waits until the user enters the required input. If we run a process in the background, we must make sure it is not waiting for input, otherwise, the background process will be suspended forever.

Foreground and background processes terminate when their parent process ends, for example when the user logs out from the system. To have the background process run even after we log out, we must start the process in a specific way, e.g. using the **nohup** command. An example is given below where the background process runs even after we log out:

```
pi@raspberrypi:~ $ nohup ls -l > files &  
[1] 6735 nohup: ignoring input and redirecting stderr to stdout
```

3.3 Task scheduling

There are many applications where we may want to run tasks at regular intervals, such as backup operations, time synchronization, running a process in the future, etc.

Running tasks at regular intervals is managed by the **crontab** command. This consists of a set of tables (crontab tables) and the **crond** daemon. The daemon is started by the **init** process at system startup and it wakes up every minute and checks the crontab tables to determine if there are any tasks scheduled to run.

To create a crontab table we use the **crontab** command with the **-e** option. This opens the vi editor (unless another editor is specified in the environment variable). Each crontab contains 6 fields. The values in the fields can have fixed values or a range of values, or a list of values separated by commas:

- Minute (0 - 59)
- Hour (0 - 23)
- Day of the month (1 - 31)
- The month of the year (1 - 12. It can also be specified as jan, feb, mar and so on)
- Day of the week (0 - 6, where 0=Sunday, it can also be mon, tue, wed, etc)
- String (command) to be executed

A ***** character in the digit position means *every*. Ranges of numbers are specified by separating them with a hyphen and the specified range is inclusive. For example, 9-12 for an Hours entry specifies execution at hours 9, 10, 11, and 12.

Skips of numbers in ranges can be specified by adding character **/** after the range. For example, 0-12/2 in the Hours field specifies execution every other hour i.e. at hours 0, 2, 4, 6, 8, 10.

By using a combination of ***** and **/**, we can specify steps. For example, ***/2** in the Hours field specifies every two hours.

Instead of the first five fields, we can specify special strings as follows:

String	meaning
@reboot	run once at startup
@yearly	run once every year ("0 0 1 1 *")
@monthly	run once every month ("0 0 1 * *")
@weekly	run once a week ("0 0 * * 0")
@daily	run once a day ("0 0 * * *")
@hourly	run once an hour ("0 * * * *")

Multiple commands can be entered on a line. Such commands must be separated with && characters.

Some examples of crontab lines are given below:

1. 30,50 22-23 * 6 fri-sat/home/pi/mycron.sh

Run at the 30th and 50th minutes, for the hours between 10 p.m. and midnight on Fridays and Saturdays during June.

2. @daily <command1>&&<command2>

Run command 1 and command 2 daily

3. 30 0 * * */home/pi/mycron.sh

Run at 12:30 daily

4. 0 4 12 * * /home/pi/mycrob.sh

Run at 4 a.m. on the 12th of every month

5. *** /home/pi/mycron.sh**

Run every minute

6. 0 4 15-21 * 1 /home/pi/mycron.sh

Run every month at 4 a.m. on Mondays, and on the days between 15-21. The day of the month and the day of the week are used with no restrictions (no *). This is an "or" condition. Both will be executed

7. 0 11,16 * * /home/pi/mycron.sh

Run every day at 11:00 and 16:00 hours

8. 0 11-14 * * * /home/pi/mycron.sh

Run every day during the hours 11 a.m. -2 p.m. (i.e. 11 a.m., 12 a.m., 1 p.m., 2 p.m.)

9. */10 ** /home/pi/mycron.sh**

Run every 10 minutes

10. @yearly /home/pi/mycron.sh

Run on the first minute of every year

By default, crontab sends the job to the user who scheduled the job. If you don't want mail to be sent to anyone, you should specify the following line in the crontab:

```
MAIL=""
```

Any outputs in a scheduled process are usually logged in files. For example, to run **mycron.sh** daily and send the output to file **daily.txt**, enter the following command:

```
@daily /home/pi/mycron.sh > daily.txt
```

Instead of directly editing the crontab file, you can also add the entries to a cron-file first, and then install them to cron using the crontab command and specify the filename. An example is given below:

```
pi@raspberrypi:~ $ crontab /home/pi/mycron.sh
```

This will install the mycron.sh to the crontab, which will also remove any old cron entries. The created crontab is stored in directory **/etc/spool/cron/<user>**

In all the examples above, we have specified the absolute path of the script file that should be executed. We can specify this path in the PATH environment variable in the crontab and then just enter the filename. An example is given below where the absolute path to the file is **/home/pi/mycron.sh**

```
PATH=/home/pi
@daily mycron.sh
```

If the script or the command we wish to run requires privilege, then it should be prefixed with the **sudo** command.

Example

It is required to run **myscript.sh** every minute. Assume that this script file has the following line of command:

```
date >> /home/pi/myfile
```

Schedule this event using the crontab command. Send the output from the command to a file called **myfile**.

Solution

First of all, create the myscript.sh file and type in the above command. The default editor is **nano**. Save the file (Ctrl+X, followed by Y and Enter) and exit the editor. Next, give the file execute permission by entering the following command:

```
pi@raspberrypi:~ $ sudo chmod 755 myscript.sh
```

The steps for using crontab command to schedule this event are given below:

- Run crontab command with the **-e** flag. You should see the crontab editor screen as in Figure 3.1

```
pi@raspberrypi:~ $ crontab -e
```



```
GNU nano 3.2 /tmp/crontab.2U2rAM/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -czf /var/backups/home.tgz /home/
[ Read 23 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File ^\ Replace  ^U Uncut Text ^T To Spell  ^_ Go To Line
```

Figure 3.1 Empty crontab text editor screen

- Enter the following line to the end of the file:

```
* * * * * /home/pi/myscript.sh
```

- Exit the **nano** editor by entering:

```
Cntrl+X
Y
<Enter>
```

- The commands in the myscript.sh will now be executed every minute and the output will be sent to file **myfile**. We can verify that the commands are executed by looking at the contents of **myfile** after a few minutes:

```
pi@raspberrypi:~ $ cat myfile
Wed 13 May 16:27:01 BST 2020
Wed 13 May 16:28:01 BST 2020
Wed 13 May 16:29:01 BST 2020
pi@raspberrypi:~ $
```

- You can stop the scheduling by deleting the line entered by command **crontab -e**

3.3.1 Task scheduling management

Several commands are available for managing scheduled tasks.

Displaying scheduled tasks

Enter the command **crontab -l** to display the scheduled tasks:

```
pi@raspberrypi:~ $ crontab -l
```

Deleting scheduled tasks

We can delete all scheduled tasks using the **crontab -r** command:

```
pi@raspberrypi:~ $ crontab -r
pi@raspberrypi:~ $ crontab -l
nocrontab for pi
pi@raspberrypi:~ $
```

The **-i** option can be added to the delete command to confirm the delete action.

3.3.2 The crontab generator

An online tool called the **Crontab Generator** is available free of charge on the Internet that can be used to easily generate crontab entries. This tool is available on the following web site:

<https://crontab-generator.org/>

Figure 3.2 shows part of the Crontab Generator. An example is given to show how to use this tool.

Complete the following form to generate a crontab line

Ctrl-click (or command-click on the Mac) to select multiple entries

Minutes ☒ Every Minute ☐ Even Minutes ☐ Odd Minutes ☐ Every 5 Minutes ☐ Every 15 Minutes ☐ Every 30 Minutes 0 1 2 3 4 5 6 7 8 9	Hours ☒ Every Hour ☐ Even Hours ☐ Odd Hours ☐ Every 6 Hours ☐ Every 12 Hours Midnight 1am 2am 3am 4am 5am 6am 7am 8am 9am	Days ☒ Every Day ☐ Even Days ☐ Odd Days ☐ Every 5 Days ☐ Every 10 Days ☐ Every Half Month 1 2 3 4 5 6 7 8 9 10
Months ☒ Every Month ☐ Even Months ☐ Odd Months ☐ Every 4 Months ☐ Every Half Year Jan Feb Mar Apr May Jun	Weekday ☒ Every Weekday ☐ Monday-Friday ☐ Weekend Days Sun Mon Tue Wed Thu Fri	

Figure 3.2 The Crontab Generator

Example

It is required to run the script file **myscript.sh** every day at 11:00 a.m. and 4:00 p.m. hours.

Solution

The steps are given below:

- Start the Crontab Generator.
- Select the Minutes as 0, Hours as 11 am and 4 pm (click **Ctrl** to select more than one entry)
- Enter **myscript.sh** to the field **Command to Execute** as shown in Figure 3.3)

Complete the following form to generate a crontab line

Ctrl-click (or command-click on the Mac) to select multiple entries

Minutes

☐ Every Minute
☐ Even Minutes
☐ Odd Minutes
☐ Every 5 Minutes
☐ Every 15 Minutes
☐ Every 30 Minutes

0

Hours

☐ Every Hour
☐ Even Hours
☐ Odd Hours
☐ Every 6 Hours
☐ Every 12 Hours

11am 4pm

Days

☒ Every Day
☐ Even Days
☐ Odd Days
☐ Every 5 Days
☐ Every 10 Days
☐ Every Half Month

Months

☒ Every Month
☐ Even Months
☐ Odd Months
☐ Every 4 Months
☐ Every Half Year

Jan Feb Mar Apr May Jun Jul Aug Sep Oct

Weekday

☒ Every Weekday
☐ Monday-Friday
☐ Weekend Days

Sun Mon Tue Wed Thu Fri Sat

Command To Execute

myscript.sh

Figure 3.3 Configure to schedule at 11:00 a.m. and 4:00 p.m

- Click **Save output to file** and enter **myfile** so that the output will be stored in file **myfile**.
- Click button **Generate Crontab Line**
- You should see the generated line as shown in Figure 3.4, which is:

```
0 11,16 * * * myscript.sh > myfile
```

```
0 11,16 * * * myscript.sh > myfile

Your cron job will be run at: (5 times displayed)

• 2020-05-14 11:00:00 UTC
• 2020-05-14 16:00:00 UTC
• 2020-05-15 11:00:00 UTC
• 2020-05-15 16:00:00 UTC
• 2020-05-16 11:00:00 UTC
• .....
```

Figure 3.4 The generated line

- The tool also displays sample dates and times that the script file will be scheduled to run. It is clear from Figure 3.4 that the script file **myscript.sh** will run at 11:00 a.m. and 4:00 p.m. every day.
- You should copy the above line to the end of the crontab table by entering the command **crontab -e** as discussed earlier.

3.4 Running a program or script automatically at system startup

There are many applications, especially control and automation based applications, where we may want to start a program automatically as soon as our Raspberry Pi boots. As explained below, there are several ways that this can be done.

3.4.1 Using **/etc/rc.local**

This is perhaps the easiest way to automatically start a program at system startup time. All we have to do is to insert the program name together with its full path in file **/etc/rc.local**. This is a special system file. Its contents are executed while the Raspberry Pi is booting and before any users log in.

The following points must be observed when entering a program name in file **/etc/rc.local**, otherwise the system may not be able to complete the boot process:

- The program must not prompt for user input
- The full path environment of the program must be specified
- The program must be terminated with the **&** character so that it runs in the background and does not wait for the completion of the program

As an example, insert the following statement to the end of file **/etc/rc.local** so that program **myprog** runs automatically after the system boots:

```
/home/pi/myprog &
```

3.4.2 Using **crontab**

As discussed earlier, the crontab tool can be used to schedule the required program to run. The **@reboot** option should be used so that the program runs just after the system boots.

An example statement is given below which should be included at the end of the crontab table so that program **myprog** runs just after the system boot:

```
@reboot /home/pi/myprog
```

3.4.3 Using **/etc/init.d**

When Raspberry Pi boots or shuts down it looks into the **/etc/init.d** directory to see if any scripts should run. We can create a script in this directory so that it is started automatically just after the system startup.

Assuming our script file is called **simple**, the **chmod** command should be used to make the script executable. e.g.:

```
pi@raspberrypi:~ $ sudo chmod +x /home/pi/simple
```

To make the program execute on startup, it is necessary to run the following command:

```
pi@raspberrypi:~ $ sudo update-rc.d /etc/init.d/simple defaults
```

This command creates a link to **/etc/init.d/simple** and when Raspberry Pi starts or shuts down it looks to see if any scripts or programs should run. An example is given below.

Example

It is required to start a program called **led-control** every time the system is started. Assume the program resides in the default user directory of **/home/pi**. Use the **/etc/init.d** method to start the program automatically.

Solution

The steps are as follows:

- Use the **nano** editor to create a script file called **simple** in directory **/etc/init.d**

```
pi@raspberrypi:~ $ sudo nano /etc/init.d/simple
```

- Enter the following lines into this script file, save the file, and exit the editor:

```
#!/bin/bash
/home/pi/led-control
```

- Make the script file executable:

```
pi@raspberrypi:~ $ sudo chmod +x /etc/init.d/simple
```

- Link to the file so that it starts at boot time:

```
pi@raspberrypi:~ $ sudo update-rc.d /etc/init.d/simple defaults
```

3.5 Resource monitoring

System resource monitoring is an important topic, especially in real-time multiprocessing applications where it may be required to monitor and control the processes running in the system.

Perhaps one of the most important system monitoring commands is **top**, which displays current system processes, how much memory they use, CPU time consumed, process priorities, and much more. Figure 3.5 shows a typical system resource display obtained by entering the following command:

```
pi@raspberrypi:~ $ top
```

```

top - 20:30:20 up 11:43, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 117 total, 1 running, 116 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.0 sy, 0.0 ni, 99.8 id, 0.2 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1939.5 total, 1638.7 free, 70.2 used, 230.6 buff/cache
MiB Swap: 100.0 total, 100.0 free, 0.0 used, 1772.8 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
13389	pi	20	0	10188	2836	2476	R	0.3	0.1	0:02.75	top
1	root	20	0	33672	7940	6352	S	0.0	0.4	0:06.47	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.04	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par_gp
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_percpu+
9	root	20	0	0	0	0	S	0.0	0.0	0:00.07	ksoftirqd+
10	root	20	0	0	0	0	I	0.0	0.0	0:02.97	rcu_sched
11	root	20	0	0	0	0	I	0.0	0.0	0:00.00	rcu_bh
12	root	rt	0	0	0	0	S	0.0	0.0	0:00.02	migration+
13	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
14	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
15	root	rt	0	0	0	0	S	0.0	0.0	0:00.02	migration+

Figure 3.5 Typical system resource display

The display is dynamic and is updated continuously. It consists of the following sections.

3.5.1 Uptime and load average

The following information is given in the first line:

```

Current time (20:30:20)
System uptime (11:43 hours)
Number of users (2)
Load average (0.00 0.00 0.00)

```

The load average is the average number of processes in the run queue, i.e. processes that are using the CPU or waiting for their turn.

3.5.2 Tasks

The second line lists the tasks the system is currently performing. The following information is given:

- The total number of processes in the system (117)
- The number of actively running processes (1)
- The number of processes that are sleeping in the background (116)
- The number of stopped processes (0)
- The number of zombie processes (0)

Zombie processes are the ones that have finished executing but are still resident in system memory. Such processes can cause problems.

3.5.3 CPU utilization

The third line is the CPU utilization percentage values. The following information is given:

- us, for user applications (0.1%)
- sy, for system applications (0.0%)
- ni, for processes that have been niced (0.0%)
- id, for idle processes (99.8%)
- wa, for processes waiting for I/O completion (0.2%)
- hi, for processes waiting for hardware interrupts (0.0%)
- si, for processes waiting for software interrupts (0.0%)
- st, for hypervisor processes (0.0%)

It is important to notice high values of wa, hi, si, or st. They are not a good sign as high values indicate the system is waiting on hardware or software to finish processing.

3.5.4 Memory usage

Lines 4 and 5 represent memory usage of the system, where line 4 displays actual memory usage. Line 5 displays the swap space usage.

The following information is given in line 4:

- Total memory (1939.5MB)
- Free memory (1638.7MB)
- Used memory (70.2MB)
- Memory allocated to buffers/cache (230.6MB)

The following information is given in line 5:

- Total swap space (100MB)
- Free swap space (100MB)
- Used swap space (0MB)
- Available swap space (1772.8MB)

3.5.5 Process table

The process table gives the following information for all the processes running in the system. The following information is given for each process:

- PID: the process ID number
- USER: the owner of the process
- PR: priority of the process
- NI: the nice value of the process
- VIRT: the amount of virtual memory used by the process
- RES: the size of the resident memory

- SHR: shared memory the process is using
- S: process status (sleeping, running, zombie)
- %CPU: the percentage of CPU consumed
- %MEM: percentage of RAM used
- TIME+: total CPU time the task used
- COMMAND: The actual name of the command

Notice that the **htop** command gives a display similar to the **top** command but it is more user friendly with colours and dynamic load bars.

The process ID number is a unique number identifying a process. The PR and NI fields represent the priority of a process. NI displays the **nice** level which is a static priority of a process when it is initialized. By default, when a process is initialised it has a nice value of 0, but can be changed using the nice command. NI has 40 values from -20 to +19. -20 is the highest priority, and +19 the lowest priority. PR shows the process dynamic priority which is calculated based on the nice value and when a process starts, the PR value equals NI value plus 20. Usually, processes start with NI values of 0 and PR values of 20. During the lifetime of a process, PR can change depending upon the system.

The priority of a running process can be changed using the **renice** command. As an example, assume that process **rcu_bh** has ID = 11 and NI priority of 0. The priority of this process can be changed to 1 as follows:

```
pi@raspberrypi:~ $ sudo renice 1 -p 11
11 (process ID) old priority 0, new priority 1
pi@raspberrypi:~ $
```

The ps command

The nice value of a process can be displayed using the **top** or **ps** command. For example, the PID and **nice** numbers of process 11 (**rcu_bh**) can be displayed as follows:

```
pi@raspberrypi:~ $ ps -o pid,nice -p 11
PID    NI
11      1
pi@raspberrypi:~ $
```

The **ps** command can also be used to list all processes used by the current user. An example is given below:

```
pi@raspberrypi:~ $ ps
PID    TTY    TIME      CMD
982    pts/0  00:00:00    bash
1172   pts/0  00:00:00     ps
pi@raspberrypi:~ $
```

The command **ps -ef** gives more information about the processes running on the system, as shown in Figure 3.6.

```
pi@raspberrypi:~ $ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root          1      0  0  10:20 ?        00:00:03 /sbin/init splash
root          2      0  0  10:20 ?        00:00:00 [kthreadd]
root          3      2  0  10:20 ?        00:00:00 [rcu_gp]
root          4      2  0  10:20 ?        00:00:00 [rcu_par_gp]
root          8      2  0  10:20 ?        00:00:00 [mm_percpu_wq]
root          9      2  0  10:20 ?        00:00:00 [ksoftirqd/0]
root         10      2  0  10:20 ?        00:00:00 [rcu_sched]
root         11      2  0  10:20 ?        00:00:00 [rcu_bh]
root         12      2  0  10:20 ?        00:00:00 [migration/0]
root         13      2  0  10:20 ?        00:00:00 [cpuhp/0]
root         14      2  0  10:20 ?        00:00:00 [cpuhp/1]
root         15      2  0  10:20 ?        00:00:00 [migration/1]
```

Figure 3.6 The **ps -ef** command (only part of the output is shown)

The PID of a process can be displayed using command **pidof**, followed by the name of the process. An example is shown below where the PID of process **rcu_bh** is displayed:

```
pi@raspberrypi:~ $ pidof rcu_bh
11
pi@raspberrypi:~ $
```

To display the name of a process given its PID, use the **ps** command with option **-q** as shown in the following example:

```
pi@raspberrypi:~ $ ps -q 11
PID    TTY          TIME         CMD
 11     ?           00:00:00      rcu_bh
pi@raspberrypi:~ $
```

Sometimes we may want to know if a process is running in the system. This can be checked using the **ps** command with option **-C**, followed by the process name. For example, to check if process **rcu_bh** is running, enter the following command:

```
pi@raspberrypi:~ $ ps -C rcu_bh
PID    TTY          TIME         CMD
 11     ?           00:00:00      rcu_bh
pi@raspberrypi:~ $
```

To display the processes started by a specific user (e.g. **pi**), enter the following command:

```
pi@raspberrypi:~ $ ps -u pi
```

Killing a process

There are many options for killing (or stopping) a process. A process can easily be killed by specifying its PID and using the following command:

```
pi@raspberrypi ~$ kill <PID>
```

Sometime you may need to use the `-9` option to force all related commands to stop. i.e.

```
pi@raspberrypi ~$ kill -9 <PID>
```

You could also use the `killall` command to stop all occurrences of a process. This command is not a clean stop as it will immediately stop the specified process:

```
pi@raspberrypi ~$ killall <process name>
```

You may need to specify `sudo` before using kill commands.

3.5.6 Disk usage

The disk free command `df` with option `-h` can be used to display disk usage statistics. An example is given in Figure 3.7.

```
pi@raspberrypi:~ $ sudo df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        14G   6.0G   7.4G  45% /
devtmpfs         841M    0   841M   0% /dev
tmpfs            970M    0   970M   0% /dev/shm
tmpfs            970M  8.5M   962M   1% /run
tmpfs            5.0M  4.0K   5.0M   1% /run/lock
tmpfs            970M    0   970M   0% /sys/fs/cgroup
/dev/mmcblk0p1   253M   40M   214M  16% /boot
tmpfs            194M    0   194M   0% /run/user/1000
pi@raspberrypi:~ $
```

Figure 3.7 Displaying disk usage

3.5.7 CPU information

The command `cat /proc/cpuinfo` can be used to display information about the CPU on your Raspberry Pi as shown in Figure 3.8. As you can see from the display, the processor is the **ARMv7 Processor rev 3 (v7l)**.

```
pi@raspberrypi:~ $ cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 3 (v7l)
BogoMIPS      : 108.00
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4
vfpd32 lpae evtstrm crc32
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part      : 0xd08
CPU revision   : 3

processor       : 1
model name     : ARMv7 Processor rev 3 (v7l)
BogoMIPS      : 108.00
Features       : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4
vfpd32 lpae evtstrm crc32
CPU implementer : 0x41
CPU architecture: 7
CPU variant    : 0x0
CPU part      : 0xd08
CPU revision   : 3
```

Figure 3.8 Displaying the CPU information (only part of the display is shown)

3.5.8 Memory use

The command **cat /proc/meminfo** can be used to display information about memory usage as shown in Figure 3.9.

```
pi@raspberrypi:~ $ cat /proc/meminfo
MemTotal:      1986028 kB
MemFree:       1707872 kB
MemAvailable:  1809584 kB
Buffers:       26840 kB
Cached:        143404 kB
SwapCached:    0 kB
Active:        156240 kB
Inactive:      45572 kB
Active(anon):  31844 kB
Inactive(anon): 8360 kB
Active(file):  124396 kB
Inactive(file): 37212 kB
Unevictable:   16 kB
Mlocked:       16 kB
HighTotal:     1232896 kB
HighFree:      1045720 kB
LowTotal:      753132 kB
```

Figure 3.9 Onboard memory information (only part of the display is shown)

3.5.9 Operating system information

Some useful operating system related information can be obtained by entering the following commands:

```
pi@raspberrypi:~ $ cat /proc/sys/kernel/hostname
raspberrypi
```

```
pi@raspberrypi:~ $ cat /proc/sys/kernel/ostype
Linux
```

```
pi@raspberrypi:~ $ cat /proc/sys/kernel/osrelease
4.19.57-v71+
```

```
pi@raspberrypi:~ $ cat /etc/os-release
PRETTY_NAME="Raspbian GNU/Linux 10 (buster)"
NAME="raspbian GNU/Linux"
VERSION_ID="10"
VERSION="10 (buster)"
VERSION_CODENAME=buster
ID=raspbian
ID_LIKE=debian
HOME_URL="http://www.raspbian.org/"
SUPPORT_URL="http://www.raspbian.org/RaspbianForums"
BUG_REPORT_URL="http://www.raspbian.org/RaspbianBugs"
```

3.5.10 USB devices

A list of the connected USB devices can be displayed by using the command **lsusb -t** as shown in Figure 3.10.

```

pi@raspberrypi:~$ lsusb -t
/: Bus 03.Port 1: Dev 1, Class=root_hub, Driver=dwc_otg/lp, 480M
/: Bus 02.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/4p, 5000M
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/1p, 480M
   |__ Port 1: Dev 2, If 0, Class=Hub, Driver=hub/4p, 480M

```

Figure 3.10 List of USB devices

3.5.11 SD card information

The SD card where the operating system is installed on has the device name `/dev/mmcblk0`. We can display information about this SD card using the command **fdisk -l /dev//mmcblk0** as shown in Figure 3.11.

```

pi@raspberrypi:~$ sudo fdisk -l /dev/mmcblk0
Disk /dev/mmcblk0: 14.5 GiB, 15552479232 bytes, 30375936 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x7a119b46

Device            Boot    Start        End    Sectors    Size Id Type
/dev/mmcblk0p1          8192     532480     524289    256M  c W95 FAT32 (LBA)
/dev/mmcblk0p2    540672  30375935  29835264   14.2G  83 Linux

```

Figure 3.11 SD card information

3.5.12 CPU architecture information

Information about the CPU architecture can be displayed by entering the command **lscpu** as shown in Figure 3.12.

```

pi@raspberrypi:~$ lscpu
Architecture:        armv7l
Byte Order:          Little Endian
CPU(s):              4
On-line CPU(s) list: 0-3
Thread(s) per core:  1
Core(s) per socket:  4
Socket(s):           1
Vendor ID:           ARM
Model:               3
Model name:          Cortex-A72
Stepping:            r0p3
CPU max MHz:         1500.0000
CPU min MHz:         600.0000
BogoMIPS:            108.00
Flags:               half thumb fastmult vfp edsp neon vfpv3
                    vt vfpd32 lpae evtstrm crc32
pi@raspberrypi:~$

```

Figure 3.12 Displaying the CPU architecture information

3.6 Summary

In this chapter, we looked at the commands for process management and resource monitoring. These commands are important especially in multiprocessing and multitasking applications where we may want to manipulate several processes.

In the next chapter, we will look at the concepts of multiprocessing and multitasking.

Chapter 4 • Multiprocessing and multithreading

4.1 Overview

In the last chapter, we looked at important commands for process management and resource monitoring which are useful in multiprocessing and multitasking applications. In this chapter, we will look briefly at the concepts of multiprocessing and multitasking and learn about their differences. Additionally, we will be looking at the principles of some of the commonly used scheduling algorithms.

The concepts of multiprogramming, multiprocessing, multithreading, and multitasking can be very confusing as they are defined in different ways by different experts in this field. In this book, we will use the general term multitasking to refer to the different ways of executing parallel programs (or tasks) on a computer system.

For simplicity, we will use the term multitasking to refer to the use of the following methods for executing parallel programs on the Raspberry Pi:

- Forks
- Threads (or multithreads)
- Threading (or multithreading)
- Subprocesses
- Multiprocessing

4.2 What is multithreading?

In a multithreading system, multiple tasks share a common processing resource such as the CPU. In such a system we can run multiple applications and the operating system can quickly switch between each task to give the impression that all the tasks are executing simultaneously. The CPU clock speed is so fast that the tasks run very fast and are switched quickly, resulting in high performance overall.

In single-core CPU based systems, only one task can run at any point in time. Multithreading is implemented in such systems by scheduling which task may run at any given time, and when other waiting tasks can be given CPU time.

In a multicore CPU based system, there are more than one CPU and the operating system can execute multiple tasks concurrently, where the tasks can work independently of each other. For example, in a quad-core CPU based system, four applications such as e-mail, word processing, spreadsheet, and web browsing can each access a separate processor core at the same time. The user in such a system can perform tasks simultaneously, and hence an improved overall performance is obtained.

In a multithreading system, there could be several threads (or tasks) in a program, and the same memory space is used by all the tasks and as a result, precautions should be taken while reading or writing to the common memory. The Python language supports multithreaded programming and provides libraries (APIs) to make it easier to create such applications.

4.3 What is multiprocessing?

In general, multiprocessing is similar to multithreading where more than one process is required to run on the system. Perhaps the main difference between the two is that in multiprocessing the processes are completely independent of each other with each one having its own memory space. Software libraries (APIs) are then used to establish synchronization, pass data between processes, and establish communication between the processes.

4.4 Differences between multithreading and multiprocessing

It is important to know the difference between multithreading and multiprocessing based applications. A comparison of both methods is summarised below.

- Threads share the same variables and run in the same memory space. Processes, on the other hand, have separate memories
- Sharing objects between threads is easier, but synchronisation to make sure that two threads will not write to the same object at the same time is important. As a result of this, thread-based programming is more prone to errors. On the other hand, process-based programming is less error-prone.
- Threads have a lower overhead compared to processes. Starting processes takes a longer time.
- Threads cannot achieve true parallelism. Processes on the other hand do not have such restrictions.
- Thread scheduling is done using the Python interpreter, while process scheduling is handled by the operating system.
- Child threads can't be easily terminated. On the other hand, they can be easily killed.
- It is recommended to use threads involving I/O based programming or with programs having user interactions. Processes, on the other hand, should be used in CPU bound computation-intensive programs
- Python offers two libraries in the form of APIs: multiprocessing and threading.

In this book, we will learn how to develop applications using both multithreading and multiprocessing with the Python programming language.

4.5 Task Scheduling algorithms

Although many variations of scheduling algorithms are in use today, the three most commonly used algorithms are:

- Co-operative scheduling
- Round-robin scheduling
- Pre-emptive scheduling

The type of scheduling algorithm used depends on the nature of the application. In general, most applications use either one of the above algorithms, a combination of them, or a modified version of them.

4.5.1 Co-operative scheduling

Co-operative scheduling, also known as non-preemptive scheduling, shown in Figure 4.1, is perhaps the simplest algorithm where tasks voluntarily give up CPU usage when they have nothing useful to do or when they are waiting for resources to become available. This algorithm has the main disadvantage that certain tasks can use excessive CPU times, thus not allowing other important tasks to run when needed. Co-operative scheduling is only used in simple multitasking systems where there are no time-critical tasks.

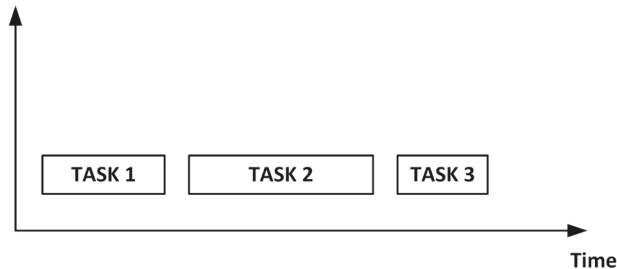


Figure 4.1 Co-operative scheduling

State machines, also called finite-state machines (FSM) are probably the simplest ways of implementing co-operative scheduling. A **while** loop can be used to execute the tasks, one after one, as shown below for a 3 task application. In the code below, a task is represented with a function:

```
Task1()  
{  
    Task 1 code  
}  
  
Task2()  
{  
    Task 2 code  
}  
  
Task3()
```

```
{  
    Task 3 code  
}  
  
while(1)  
{  
    Task1();  
    Task2();  
    Task3();  
}
```

The tasks are executed one after the other one inside the main infinite loop formed using a while statement. In this simple approach, the tasks can communicate with each other using global variables declared at the beginning of the program. The tasks in a co-operative scheduler must satisfy the following requirements for the successful running of the overall system:

- Tasks must not block the overall execution, e.g. by using delays or waiting for some resources and not releasing the CPU.
- The execution time of each task should be acceptable to other tasks
- Tasks should exit as soon as they complete their processings
- Tasks do not have to run to completion and they can exit for example before waiting for a resource to be available
- Tasks should resume their operations from the point after they release the CPU.

The last requirement listed above is very important and is not satisfied in the simple scheduler example given above. Resuming a task requires the address of the program counter and important variables when the task releases the CPU to be saved, and then restored when the task resumes (also called context switching) so that the interrupted task can continue normally as if there has not been any interruption.

Another way of implementing a very simple co-operative scheduling is by using a switch statement inside an infinite loop as shown below. Notice here that as before, the task states are not saved in this simple example:

```
Task1()  
{  
    Task 1 code  
}  
  
Task2()  
{
```

```
    Task 2 code
}

Task3()
{
    Task 3 code
}

nxt = 1;

while(1)
{
switch(nxt)
{
    case 1:
        Task1();
        nxt = 2;
        break;
    case 2:
        Task2();
        nxt = 3;
        break;
    case 3:
        Task3();
        nxt = 1;
        break;
}
}
```

4.5.2 Round-robin scheduling

Round-robin scheduling (see Figure 4.2) allocates each task an equal share of the CPU time. Tasks are in a circular queue and when a task's allocated CPU time expires, it is removed and placed at the end of the queue. This type of scheduling cannot be satisfactory in any real-time application where each task can have a varying amount of CPU requirements depending on the complexity of the processing involved. Round-robin scheduling requires the context of the running task to be saved on the stack when a task is removed from the queue so that the task can resume from the point it was interrupted when it becomes active again. One variation of the pure Round-robin based scheduling is to provide priority-based scheduling, where tasks with the same priority levels receive equal amounts of CPU time.

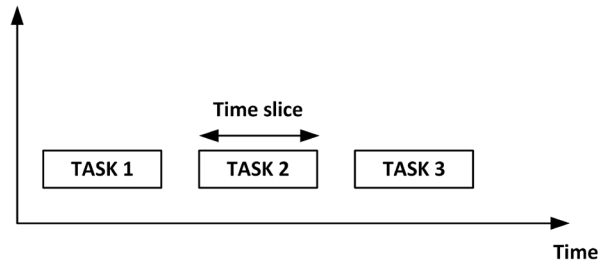


Figure 4.2 Round-robin scheduling

Round-robin scheduling has the following advantages:

- It is easy to implement
- Every task gets an equal share of the CPU
- Easy to compute the average response time

The disadvantages of Round-robin scheduling are:

- It is not generally good to give the same CPU time to each task
- Some important tasks may not run to completion
- Not suitable for real-time systems where tasks have different processing requirements

4.5.3 Pre-emptive scheduling

Pre-emptive scheduling is the most commonly used scheduling algorithm in real-time systems. Here, the tasks are prioritized and the task with the highest priority among all other tasks gets the CPU time (see Figure 4.3). If a task with a priority higher than the currently executing task becomes ready to run, the kernel saves the context of the current task and switches to the higher priority task by loading its context. Usually, the highest priority task runs to completion or until it becomes non-computable, for example waiting for a resource to become available, or calling a function to delay. At this point, the scheduler determines the task with the highest priority that can run and loads the context of this task and starts executing it. Although pre-emptive scheduling is very powerful, care is needed as a programming error can place a high priority task in an endless loop and thus not release the CPU to other tasks. Some multitasking systems employ a combination of Round-robin and pre-emptive scheduling. In such systems, time-critical tasks are usually prioritized and run under pre-emptive scheduling, whereas the non-time-critical tasks run under Round-robin scheduling, sharing the left CPU time among themselves.

It is important to realize that in a preemptive scheduler, tasks at the same priorities run under Round-robin. In such a system, when a task uses its allocated time, a timer interrupt is generated by the scheduler which saves the context of the current task and gives the CPU to another task with an equal priority that is ready to run, provided that there are no other tasks with higher priorities which are ready to run.

The priority in a preemptive scheduler can be static or dynamic. In a static priority system, tasks use the same priority all the time. In a dynamic priority-based scheduler, the priority of tasks can change during their courses of execution.

So far, we have said nothing about how various tasks work together in an orderly manner. In most applications, data and commands must flow between various tasks so that the tasks can co-operate and work together. One very simple way of doing this is through shared data held in RAM where every task can access. Modern RTOS systems, however, provide local task memories and inter-task communication tools such as mailboxes, pipes, queues, etc to pass data securely and privately between various tasks. Also, tools such as event flags, semaphores, and mutexes are usually provided for inter-task communication and synchronization purposes and passing data between tasks.

The main advantage of a preemptive scheduler is that it provides an excellent mechanism where the importance of every task may be precisely defined. On the other hand, it has the disadvantage that a high priority task may starve the CPU such that lower priority tasks can never have the chance to run. This can usually happen if there are programming errors such that the high priority task runs continuously without having to wait for any system resources and never stops.

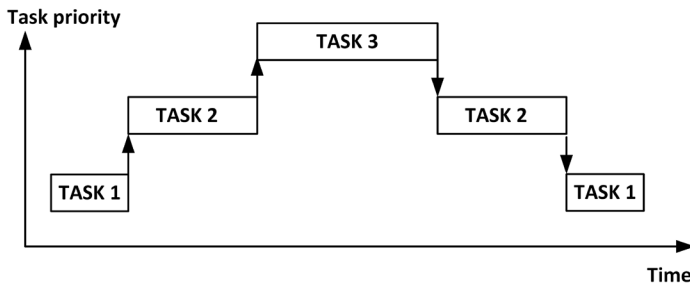


Figure 4.3 Pre-emptive scheduling

4.5.4 Scheduling algorithm goals

It can be said that a good scheduling algorithm should possess the following features:

- Be fair such that each process gets a fair share of the CPU
- Be efficient by keeping the CPU busy. The algorithm should not spend too much time to decide what to do
- Maximize throughput by minimizing the time users have to wait
- Be predictable so that same tasks take the same time when running multiple times
- Minimize response time
- Maximize resource use
- Enforce priorities
- Avoid starvation

4.5.5 Difference between preemptive and non-preemptive scheduling

Some differences between a preemptive and non-preemptive scheduling algorithm are summarized in Table 4.1.

Non-preemptive Scheduling	Preemptive Scheduling
Tasks have no priorities	Tasks have priorities
Tasks cannot be interrupted	A higher priority task interrupts a lower priority one
Waiting and response times are longer	Waiting and response times are shorter
Scheduling is rigid	Scheduling is flexible
Tasks do not have priorities	High priority tasks run to completion
Not suitable for real-time systems	Suitable for real-time systems

Table 4.1 Differences between preemptive and non-preemptive scheduling

4.5.6 Some other scheduling algorithms

There are many other types of scheduling algorithms used in practice. Most of these algorithms are a combination or a derivation of the basic three algorithms described in this Chapter. Brief details of some other scheduling algorithms are outlined in this section.

First come, first served

This is one of the simplest scheduling algorithms, is known as FIFO scheduling. In this algorithm, tasks are run in the order they become ready. Some features of this algorithm are:

- Throughput is low since long processes can hold the CPU, causing short processes to wait for a long time
- There is no prioritization and thus real-time tasks cannot be executed quickly
- It is non-pre-emptive
- The context switching occurs only on task termination and therefore the overhead is minimal
- Each process gets the chance to be executed even if they have to wait for a long time

Shortest remaining time first

In this algorithm, the scheduler arranges the tasks with the least estimated processing time remaining to be next in the queue. Some features of this algorithm are:

- If a shorter task arrives, the currently running task is interrupted, thus causing overhead.
- Waiting time of tasks requiring long processing times can be very long
- If there are too many small tasks in the system, longer tasks may never get the chance to run

Longest remaining time first

In this algorithm, the scheduler arranges the tasks with the longest processing times to be next in the queue. Some features of this algorithm are:

- If a longer task arrives, the currently running task is interrupted, thus causing overhead.
- Waiting time of tasks requiring short processing times can be very long
- If there are too many long tasks in the system, shorter tasks may never get the chance to run

Multilevel queue scheduling

In this type of scheduling, tasks are classified into different groups, such as interactive (foreground) and batch (background). Each group has its scheduling algorithm, Foreground tasks are given higher priorities since the background tasks can always wait.

Dynamic priority scheduling

In dynamic priority scheduling, although the tasks have priorities, their priorities can change, i.e. the priority can be lower or higher than it was earlier. Dynamic priority algorithms achieve high processor utilization, and they can adapt to dynamic environments, where task parameters are unknown. On the contrary, it is not advisable to use dynamic priority in real-time systems because of the uncertainty that an important task may not run in time.

4.5.7 Choosing a scheduling algorithm

When designing a multitasking system with several tasks, a programmer must consider which scheduling algorithm will perform the best for the application to be developed. In simple terms, most real-time systems should be based on preemptive scheduling with fixed priorities where time-critical tasks grab and use the CPU until they complete their processings or wait for a resource to be available. If there are several time-critical tasks, all such tasks should run at the same higher priorities. In general, tasks at the same priorities run as Round-robin and share the available CPU time. Then, all the other tasks which are not time-critical should run at lower priorities. If the time taken for a task to complete is not critical then simple co-operative scheduler algorithms can be employed.

4.6 Summary

In this chapter, we learned the differences between multiprocessing and multithreading. Additionally, the principles of commonly used scheduling algorithms have been briefly described.

In the remaining chapters of this book, we will develop various projects using the Raspberry Pi together with the Python programming language, based on the various multitasking techniques, such as forks, threads, threading, subprocesses, and multiprocessing.

Chapter 5 • Raspberry Pi multitasking projects - using the fork()

5.1 Overview

In the last chapter, we looked at the concepts of multitasking and multiprocessing and also learned the basic principles of some commonly used scheduling algorithms. In this chapter, we will develop various multitasking projects using the Python programming language on our Raspberry Pi. All projects in this chapter are based on using the process forks.

5.2 Running shell commands from Python

There are applications where we may want to run a command from inside Python. There are three operating system (os) functions that allow shell commands to run from Python programs:

`os.system()`, `os.popen()`, and `subprocess.call()`

Using `os.system()`

`os.system()` runs a shell command from a Python program. Some examples are given below. Notice the **`os`** module must be imported to the program and commands must be enclosed within single or double quotes:

```
>>> import os
>>> os.system("pwd")
/home/pi

>>> os.system("whoami")
pi

>>> os.system("date")
Thu 14 May 19:45:00 BST 2020
```

It is important to note that the **`os.system()`** command is blocking. i.e. it pauses until the spawned process exits. One way to get around this problem is to add the **`&`** character at the end of the command to make it a background task as shown below:

```
>>> os.system("python3 myfile.py &")
```

Using `os.popen()`

`Os.popen()` runs a shell command from a Python program and connects to its input and output streams. Module **`os`** must be Imported to the program. An example is given below:

```
>>> import os
>>> d = os.popen("date")
>>> c = d.readline()
>>> print(c)
Thu 14 May 19:48:00 BST 2020
```

Using subprocess.call()

Using this command requires the **subprocess** module to be imported. Its use is similar to **os.system()**, but it gives more control over how streams are connected and used. An example is given below:

```
>>> import subprocess
>>> subprocess.call("date")
Thu 14 May 19:52:00 BST 2020
```

5.3 Process forks

Forking is the way of making copies of a running program. When a program calls the fork routine, the operating system makes a copy of the program and starts running it in parallel with the original program. The original program is called the **parent process**, and the program created by the fork routine is called the **child process**. A parent process can make as many child processes as required. Also, child processes can make other child processes. It is important to realise that all parent and child processes run independently and at the same time. Child processes continue to run even after their parent process exits.

The new child process is an exact duplicate of the parent process, except for the following points:

- The child process has a unique process ID
- The child's parent process ID is the same as the parent's process ID
- The child does not inherit its parent's memory locks
- Process resource utilisations and CPU time counters are reset to zero in the child process
- The child process does not inherit any outstanding I/O operations from its parent
- The child process does not inherit any timers from its parent
- The child does not inherit directory change notifications from its parent
- The child process is created with a single thread and the virtual address space of the parent is replicated in the child (including mutexes, condition variables, etc)
- The child inherits copies of the parent's set of open file descriptors and set of open directory streams

On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and an error number is set appropriately.

An example is given below where a child process is created using the fork routine. In this example, when the **os.fork()** function is called, the operating system makes an exact copy of the calling program. But for the child process, a 0 is returned, while for the parent process the ID is returned (i.e. a non zero value). In this program the child process and the parent process both display their process IDs.

```

import os
def fork_test():
    pid = os.fork()
    if pid == 0:
        print("Child process pid=%d" % os.getpid())
    else:
        print("Parent process pid=%d" % os.getpid())
fork_test()

```

When the program runs, the parent and child processes display their IDs:

```

Parent process pid=1171
Child process pid=1172

```

Waiting for a child

In some applications, we may want to start a child process and wait until it finishes. An example program is given below where the child process waits for 5 seconds and then exits. The parent process waits until the child exits. Notice the **os.fork()** function returns the child PID to the parent (when PID is non zero) and this is used in the **os.waitpid()** function to wait until the child terminates. The child exits by calling to operating system function **os._exit()**.

```

import os
import time
def TimeDelay():
    time.sleep(5)

pid = os.fork()
if pid > 0:
    ChildPID = pid
else:
    TimeDelay()
    os._exit(0)

print("Waiting for child..")
os.waitpid(ChildPID, 0)
print("Child finished..")

```

When the program runs, the following will be displayed:

```

Waiting for child..
Child finished..

```

Using fork/exec

The **os.fork()** function creates an identical copy of a process. This is not very useful as we may want to carry out different operations in the newly created child process. The **fork/exec** combination of functions are used to create a child process that runs a different program. The steps are as follows:

Use the **os.fork()** to create a child process

Call **os.exec()** function to replace the child process with a process that we wish to run. An example is given below. Here, program **parent.py** calls **os.fork()** to create a child process. The child process is then replaced with program **child.py**. Program **parent.py** displays message **This is the Parent...** and the child process displays the message **Hello from the Child...**

Program: parent.py

```
import os
pid = os.fork()
if pid == 0:
    os.execlp("python3", "python3", "child.py")
else:
    print("This is the Parent..")
```

Program: child.py

```
print("Hello from the Child..")
```

The program is run as follows:

```
pi@raspberrypi:~ $ python3 parent.py
This is the Parent..
Hello from the Child..
pi@raspberrypi:~ $
```

The **execlp** command has the following parameters:

```
os.execlp(program, command1, command2, ....)
```

where, program is the executable program's name, and command1, comamnd2 are the command line arguments (the words we would normally type to start a program). The **os.execlp()** function has some variations such as **os.execv()**, **os.execlp()**, **os.execve()**, **os.execvp()**, and **os.execlpe()**.

Some simple projects are given in the following sections to show how multitasking applications can be developed using process forks.

5.3.1 Project 1 – Two LEDs flashing at different rates

Description: In this project, two LEDs are connected to the Raspberry Pi. Two tasks are created using forks such that one of the LEDs flash every second, while the other one flashes every 250 milliseconds.

Block Diagram: Figure 5.1 shows the block diagram of the project.

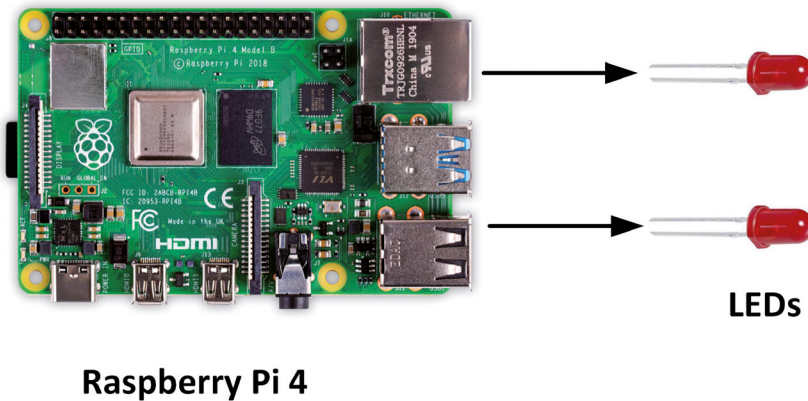


Figure 5.1 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 5.2. The LEDs are connected to Raspberry Pi GPIO 2 and GPIO 3 through 470 Ohm current limiting resistors.

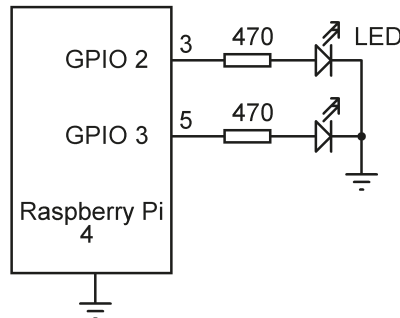


Figure 5.2 Circuit diagram of the project

Program Listing: Figure 5.3 shows the program listing (program: **two_leds.py**). At the beginning of the program modules RPi, os, and time are imported to the program. Variables LED2 and LED3 are assigned to 2 and 3 which corresponds to the GPIO pin numbers. The GPIO mode is then set to BCM and the two LED ports are configured as outputs. The main program consists of the single statement **fork_test()** which calls function **fork_test()**. This function calls to os function **fork()** to create a child process identical to the parent process. The child process is identified with the pid set to 0 and it flashes the LED connected to GPIO 2 every seconds. The parent process flashes the LED connected to GPIO 3 every 250

milliseconds. As a result, you should see the two LEDs flashing at different rates.

The program is started with the command:

```
pi@raspberrypi:~ $ python3 two_leds.py
```

Notice the program runs in the foreground and therefore the keyboard is not available while the program is running. We can run the program in the background by inserting an **&** character at the end of the command:

```
pi@raspberrypi:~ $ python3 two_leds.py &  
[1] 8756
```

You can terminate the background program by entering the command:

```
pi@raspberrypi:~ $ killall python3
```

```
#-----  
#           TWO LEDS FLASHING AT DIFFERENT RATES  
#  
# In this project two LEDs are connected to the  
# Raspberry Pi. The LEDs flash at different rates  
#  
# Author: Dogan Ibrahim  
# File  : two_leds.py  
# Date  : MAY 2020  
#-----  
import RPi.GPIO as GPIO  
import os  
import time  
GPIO.setwarnings(False)  
  
LED2 = 2                # LED at GPIO 2  
LED3 = 3                # LED at GPIO 3  
GPIO.setmode(GPIO.BCM)  
GPIO.setup(LED2, GPIO.OUT)    # LED is output  
GPIO.setup(LED3, GPIO.OUT)    # LED is output  
  
def fork_test():          # Function to fork  
    pid = os.fork()  
    if pid == 0:          # If child process  
        while True:  
            GPIO.output(LED2, 1)    # LED ON  
            time.sleep(1)           # Wait 1 second  
            GPIO.output(LED2, 0)    # LED OFF  
            time.sleep(1)           # Wait 1 second
```

```

else:
    while True:
        GPIO.output(LED3, 1)      # If parent process
        time.sleep(0.250)         # LED ON
        GPIO.output(LED3, 0)      # Wait 250 ms
        time.sleep(0.250)         # LED OFF
        time.sleep(0.250)         # Wait 250 ms

fork_test()                       # Call function

```

Figure 5.3 Program: two_leds.py

Modified program

In Figure 5.3, one of the LEDs is flashed by the child process while the other one is flashed by the parent process. In this modified version of the program, we will create two child processes to flash the two LEDs. The modified program listing is shown in Figure 5.4 (program: **two_leds2.py**). Here, two functions named **Flash_LED1** and **Flash_LED2** are used to create two child processes. One child process flashes the LED at GPIO 2 every second, while the other child process flashes the LED at GPIO 3 every 250 milliseconds.

```

#-----
#           TWO LEDS FLASHING AT DIFFERENT RATES
#
# In this project two LEDs are connected to the
# Raspberry Pi. The LEDs flash at different rates.
#
# In this version of the program two child processes
# are created to flash the two LEDs
#
# Author: Dogan Ibrahim
# File  : two_leds2.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import os
import time
GPIO.setwarnings(False)

LED2 = 2                      # LED at GPIO 2
LED3 = 3                      # LED at GPIO 3
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED2, GPIO.OUT)     # LED is output
GPIO.setup(LED3, GPIO.OUT)     # LED is output

def Flash_LED1():              # Function to fork
    pid1 = os.fork()           # Create child
    if pid1 == 0:              # If child process

```

```
while True:                                # Do forever
    GPIO.output(LED2, 1)                    # LED ON
    time.sleep(1)                           # Wait 1 second
    GPIO.output(LED2, 0)                    # LED OFF
    time.sleep(1)                           # Wait 1 second

def Flash_LED2():                           # Function to fork
    pid2 = os.fork()                        # Create child
    if pid2 == 0:                           # If child process
        while True:                        # Do forever
            GPIO.output(LED3, 1)           # LED ON
            time.sleep(0.250)               # Wait 250 ms
            GPIO.output(LED3, 0)           # LED OFF
            time.sleep(0.250)               # Wait 250 ms

Flash_LED1()                                # Call Flash_LED1
Flash_LED2()                                # Call Flash_LED2
```

Figure 5.4 Program: two_leds2.py

You can terminate the background program by entering the following command as before:

```
pi@raspberrypi:~ $ killall python3
```

5.3.2 Project 2 – Four LEDs flashing at different rates

Description: This project is similar to Project 1, but here 4 LEDs are used. The project creates 4 child processes to flash the LEDs at the following rates:

LED1	every 2 seconds
LED2	every second
LED3	every 500 milliseconds
LED4	every 250 milliseconds

This project aims to show how multiple child processes can be created in a multitasking environment.

Block Diagram: Figure 5.5 shows the block diagram of the project.

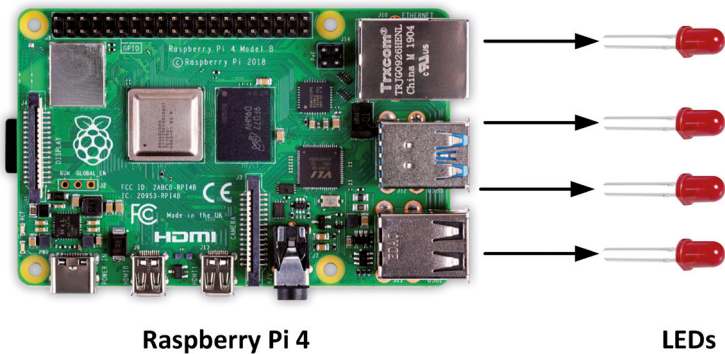


Figure 5.5 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 5.6. The LEDs are connected to the following GPIO pins of Raspberry Pi 4:

LED	GPIO	Pin no
LED1	GPIO 2	3
LED2	GPIO 3	5
LED3	GPIO 4	7
LED4	GPIO 17	11

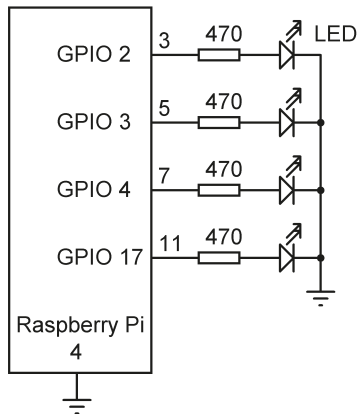


Figure 5.6 Circuit diagram of the project

Program Listing: Figure 5.7 shows the program listing (program: **four_leds.py**). At the beginning of the program, modules RPi, os, and time are imported to the program as in Project 1. Variables LED1, LED2, LED3, and LED4 are assigned to 2, 3, 4, and 17 which correspond to the GPIO pin numbers. The GPIO ports are then configured as outputs. Four functions are used in the program with the names **Flash_LED1**, **Flash_LED2**, **Flash_LED3**, and **Flash_LED4**. Each function creates a child process where each process flashes one of the LEDs as in Project 1.

The program is started with the command:

```
pi@raspberrypi:~ $ python3 four_leds.py
```

You can terminate the background program by entering the following command as before:

```
pi@raspberrypi:~ $ killall python3
```

```
#-----  
#           FOUR LEDS FLASHING AT DIFFERENT RATES  
#  
# In this project four LEDs are connected to the  
# Raspberry Pi. The LEDs flash at different rates  
#  
# Author: Dogan Ibrahim  
# File  : four_leds.py  
# Date  : MAr 2020  
#-----  
import RPi.GPIO as GPIO  
import os  
import time  
GPIO.setwarnings(False)  
  
LED1 = 2           # LED at GPIO 2  
LED2 = 3           # LED at GPIO 3  
LED3 = 4           # LED at GPIO 4  
LED4 = 17          # LED at GPIO 17  
GPIO.setmode(GPIO.BCM)  
GPIO.setup(LED1, GPIO.OUT)   # LED is output  
GPIO.setup(LED2, GPIO.OUT)   # LED is output  
GPIO.setup(LED3, GPIO.OUT)   # LED is output  
GPIO.setup(LED4, GPIO.OUT)   # LED is output  
  
def Flash_LED1():           # Function to fork  
    pid1 = os.fork()        # Create child 1  
    if pid1 == 0:           # If child process  
        while True:        # Do forever  
            GPIO.output(LED1, 1)   # LED1 ON  
            time.sleep(2)         # Wait 2 seconds  
            GPIO.output(LED1, 0)   # LED1 OFF  
            time.sleep(2)         # Wait 2 seconds  
  
def Flash_LED2():           # Function to fork  
    pid2 = os.fork()        # Create child 2  
    if pid2 == 0:           # If child process  
        while True:        # Do forever
```

```

GPIO.output(LED2, 1)           # LED2 ON
time.sleep(1)                  # Wait 1 second
GPIO.output(LED2, 0)           # LED2 OFF
time.sleep(1)                  # Wait 1 second

def Flash_LED3():               # Function to fork
    pid3 = os.fork()            # Create child 3
    if pid3 == 0:               # If child process
        while True:            # Do forever
            GPIO.output(LED3, 1) # LED3 ON
            time.sleep(0.5)      # Wait 500ms
            GPIO.output(LED3, 0) # LED3 OFF
            time.sleep(0.5)      # Wait 500ms

def Flash_LED4():               # Function to fork
    pid4 = os.fork()            # Create child 4
    if pid4 == 0:               # If child process
        while True:            # Do forever
            GPIO.output(LED4, 1) # LED4 ON
            time.sleep(0.250)    # Wait 250ms
            GPIO.output(LED4, 0) # LED4 OFF
            time.sleep(0.250)    # Wait 250ms

Flash_LED1()                   # Flash LED1
Flash_LED2()                   # Flash LED2
Flash_LED3()                   # Flash LED3
Flash_LED4()                   # Flash LED4

```

Figure 5.7 Program: *four_leds.py*

The processes we have created can be displayed by entering the command **ps -u pi** as shown in Figure 5.8.

```

pi@raspberrypi:~ $ ps -u pi
  PID TTY          TIME CMD
  532 ?            00:00:00 systemd
  539 ?            00:00:00 (sd-pam)
  555 tty1         00:00:00 bash
 8485 ?            00:00:01 sshd
 8488 pts/0        00:00:00 bash
10152 pts/0        00:00:00 python3
10153 pts/0        00:00:00 python3
10154 pts/0        00:00:00 python3
10155 pts/0        00:00:00 python3
10163 pts/0        00:00:00 ps

```

Figure 5.8 Processes created by user *pi*

5.3.3 Project 3 – Setting the LED flashing rate from the keyboard

Description: In this project, an LED is connected to the Raspberry Pi. The flashing rate of the LED is set dynamically from the keyboard while the LED is flashing.

Circuit Diagram: The circuit diagram of the project is similar to Figure 5.2, but here only one LED is used and is connected to Raspberry Pi port GPIO 2.

Program Listing: Two processes are used in this process. The child process flashes the LED, while the parent process reads the required flashing rate in seconds from the keyboard and passes this to the child process so that the flashing rate is changed as required.

It is important to realize that the variables are not shared between the parent and the child processes. Therefore, if we change the flashing rate in the parent process, the rate will not change in the child process. There are several ways that variables can be shared, such as using shared memory, using queues, pipes, and so on. In this project, a queue is used to send the flashing rate (a floating-point variable) from the parent process to the child process as shown in Figure 5.9.

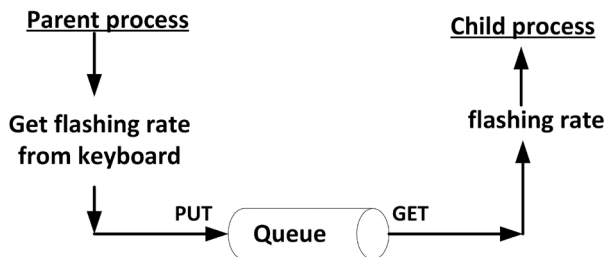


Figure 5.9 Communicating between the parent and the child

Figure 5.10 shows the program listing (program: **led_control.py**). At the beginning of the program, modules `RPi`, `os`, and `time` are imported to the program. Additionally, module `multiprocessing` (we will see the use of this module in detail in a later Chapter) is imported to the program. A queue is created with the name `q` and number 1 is sent to the queue. This is the default flashing rate as 1 second. Variable `LED2` is set to 2 which corresponds to GPIO port 2 and this port is configured as an output.

The main program calls function **fork_child()**. Inside this function, the child process (`pid = 0`) gets the flashing rate from the queue by calling function **q.get()**, and then flashes the LED at the received rate. Notice that a new flashing rate is only obtained if the queue is not empty, i.e. if the parent process has inserted a new value to the queue. The parent process displays the message **Enter flashing rate in seconds:** and waits until the user enters the required flashing rate as a floating-point number. This number is stored in a variable called **dly** and is sent to the queue using function **q.put()**. The flashing rate of the LED is then changed by the child process.

```

#-----
#       CHANGE LED FLASHING RATE FROM THE KEYBOARD
#
# In this project an LEDs is connected to port GPIO 2 of
# the Raspberry Pi. The LED flashing rate is changed by
# entering it from the keyboard
#
# Author: Dogan Ibrahim
# File  : led_control.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO          # Import RPi
import os                        # Import os
import time                      # Import time
import multiprocessing           # Import multiprocessing
q = multiprocessing.Queue()      # Create a queue
GPIO.setwarnings(False)         # Disable warnings

q.put(1)                         # Put 1 into the queue
LED2 = 2                         # LED at GPIO 2
GPIO.setmode(GPIO.BCM)          # GPIO mode
GPIO.setup(LED2, GPIO.OUT)       # LED is output

def fork_child():                # Function to fork
    pid = os.fork()              # Create a child
    if pid == 0:                 # If child process
        while True:             # Do forever
            if not q.empty():    # If queue not empty
                dly = q.get()    # Get flash rate
                GPIO.output(LED2, 1) # LED ON
                time.sleep(dly)   # Wait dly second
                GPIO.output(LED2, 0) # LED OFF
                time.sleep(dly)   # Wait dly second
            else:
                while True:      # If parent process
                    dly = float(input("Enter flashing rate in seconds: "))
                    q.put(dly)   # Send flash rate to queue

fork_child()                     # Call function

```

Figure 5.10 Program: led_control.py

The program is started with the command:

```
pi@raspberrypi:~ $ python3 led_control.py
```

Figure 5.11 shows the message displayed on the screen when the program is run. Here, the flashing rate is changed to 0.5 seconds. You can terminate the program by entering **Cntrl+C** at the keyboard.

```
pi@raspberrypi:~ $ python3 led_control.py
Enter flashing rate in seconds: 0.5
Enter flashing rate in seconds: █
```

Figure 5.11 Message displayed on the screen

5.3.4 Project 4 – Multitasking event counter

Description: This is an event counter project. In this project, an external button is used to simulate the occurrence of external events. An external event is assumed to occur when the button is pressed. The events are counted by a child process and the count is sent to the parent process so that the total count at any time can be displayed on the screen.

Block Diagram: Figure 5.12 shows the block diagram of the project.

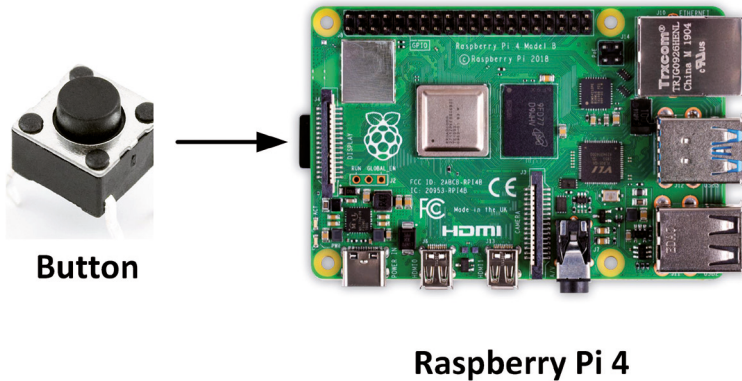


Figure 5.12 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 5.13. The button is connected to port GPIO 2 of the Raspberry Pi such that the output of the button is at logic 1 and goes to logic 0 when the button is pressed (i.e. when an external event occurs).

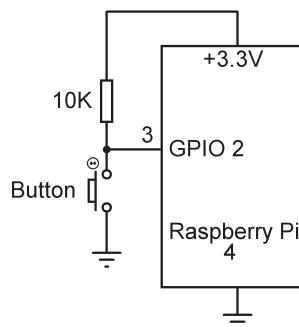


Figure 5.13 Circuit diagram of the project

Program Listing: The program listing is shown in Figure 5.14 (program: **events.py**). In this program, a queue is created with the name **q**. The button is configured as an input port. At the beginning of the child process, the variable **count** is initialized to 0. The child process waits until the button is pressed. i.e. until an external event occurs. When the button is pressed, the variable **count** is incremented by 1 and its value is put into the queue. The child process then waits until the button is released. The parent process checks the state of the queue and if the queue is not empty, the event count is read and displayed on the screen.

```
#-----
#                               MULTITASKING EVENT COUNTER
#
# This is a multitasking event counter project. A button is
# connected to port GPIO 2 of the Raspberry Pi. External events
# are assumed to occur when the button is pressed. The child process
# counts the events and sends to the parent process where the total
# number of events at any time is displayed
#
# Author: Dogan Ibrahim
# File  : events.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO          # Import RPi
import os                        # Import os
import time                      # Import time
import multiprocessing           # Import multiprocessing
q = multiprocessing.Queue()      # Create a queue
GPIO.setwarnings(False)         # Disable warnings
GPIO.setmode(GPIO.BCM)

Button = 2                      # Button at GPIO 2
GPIO.setup(Button, GPIO.IN)     # Button is an input

def fork_child():                # Function to fork
    pid = os.fork()              # Create a child
    if pid == 0:                 # If child process
        count=0                  # Set count to 0
        while True:             # Do forever
            while GPIO.input(Button) == 1: # Button is not pressed
                pass
            count = count + 1      # Increment count
            q.put(count)          # Send count to queue
            while GPIO.input(Button) == 0: # Button is not released
                pass
    else:                         # If parent process
        while True:
```

```

if not q.empty():                # If queue is not empty
    cnt = q.get()                # Get event count
    print("Event count = %d" % cnt) # Display event cnt

fork_child()                     # Call function

```

Figure 5.14 Program: *events.py*

Figure 5.15 shows an output displayed on the screen.

```

pi@raspberrypi:~ $ python3 events.py
Event count = 1
Event count = 2
Event count = 3
Event count = 4
Event count = 5
■

```

Figure 5.15 Output from the program

5.3.5 Project 5 – LED flashing and LED control with a button

Description: In this project, two LEDs named **LEDFlash** and **LEDButton** are connected to ports GPIO 2 and GPIO 3 of the Raspberry Pi through current limiting resistors respectively. At the same time, a button is connected to port GPIO 4. **LEDFlash** flashes every second. **LEDButton** is turned ON independent of **LEDFlash** when the button is pressed and is turned OFF when the button is released.

Block Diagram: Figure 5.16 shows the block diagram of the project.

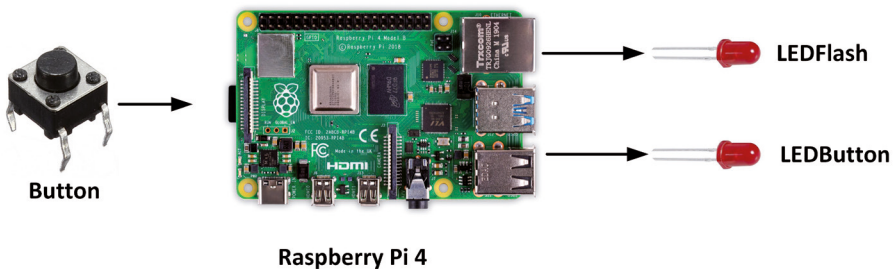


Figure 5.16 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 5.17. The output state of the button is normally at logic 1 and goes to logic 0 when the button is pressed.

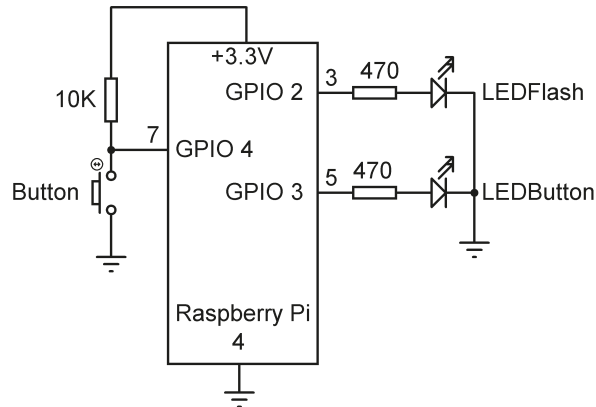


Figure 5.17 Circuit diagram of the project

Program Listing: The program listing is shown in Figure 5.18 (program: **LEDButton.py**). At the beginning of the program, the modules required are imported, **LEDFlash**, **LEDButton** and **Button** are assigned to 2, 3, and 4 respectively to correspond to ports GPIO 2, GPIO 3, and GPIO 4. Ports GPIO 2 and GPIO 3 are configured as outputs, while GPIO 4 is configured as input. The child process flashes the LED every second. Inside the parent process, **LEDButton** is turned ON when the button is pressed and turned OFF when the button is released.

```
#-----
#                               LED FLASHING AND LED BUTTON CONTROL
#
# In this program two LEDs,named LEDFlash and LEDButton are connected
# to GPIO 2 and GPIO 3 respectively. In addition, a button is
# conencted to GPIO 4. LEDFlash flashes every second. Pressing the
# button turns LEDButton ON, and releasing the button turns OFF
# LEDButton independent of LEDFlash.
#
# Author: Dogan Ibrahim
# File  : LEDButton.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO                # Import RPi
import os                               # Import os
import time                             # Import time
GPIO.setwarnings(False)                # Disable warnings
GPIO.setmode(GPIO.BCM)

LEDFlash = 2                           # LEDFlash at GPIO 2
LEDButton = 3                          # LEDButton at GPIO 3
Button = 4                             # Button at GPIO 4
GPIO.setup(LEDFlash, GPIO.OUT)         # LEDFlash is output
```

```
GPIO.setup(LEDButton, GPIO.OUT)           # LEDButton is output
GPIO.setup(Button, GPIO.IN)               # Button is an input

def fork_child():                          # Function to fork
    pid = os.fork()                        # Create a child
    if pid == 0:                           # If child process
        while True:                       # Do forever
            GPIO.output(LEDFlash, 1)       # LEDFlash ON
            time.sleep(1)                  # Wait 1 second
            GPIO.output(LEDFlash, 0)       # LEDFlash is OFF
            time.sleep(1)                  # Wait 1 second
    else:
        while True:                       # If parent process
            while GPIO.input(Button) == 1: # Wait button press
                pass
            GPIO.output(LEDButton, 1)      # Turn ON LEDButton
            while GPIO.input(Button) == 0: # Wait button release
                pass
            GPIO.output(LEDButton, 0)      # Turn OFF LEDButton

fork_child()                              # Call function
```

Figure 5.18 Program: LEDButton.py

5.3.6 Project 6 – Synchronizing the parent and child processes - multitasking event counter

Description: This project is similar to Project 3, but here the parent and child processes are synchronized. When a new event occurs, the child process puts the event count into the queue and sets a flag to inform the parent process that a new count is available. The parent process displays the total count as in Project 3.

Block Diagram: The block diagram of the project is as shown in Figure 5.12

Circuit Diagram: The circuit diagram of the project is as shown in Figure 5.13 where the button is connected to port GPIO 2 of the Raspberry Pi.

Program Listing: The program listing is shown in Figure 5.19 (program: **events_sync.py**). In this program, a queue is created with the name **q**, and an Event is created with name **e** (the details of events will be described in detail in a later Chapter). The button is configured as an input port. At the beginning of the child process, variable **count** is initialized to 0. The child process waits until the button is pressed. i.e. until an external event occurs. When the button is pressed, variable **count** is incremented by 1 and its value is put into the queue. Then, function **e.set()** is called to set an event flag. The child process then waits until the button is released. The parent process calls to function **e.wait()** which suspends the process until the event flag is set by the child process. The total event count

is then read by the parent process by calling to function **q.get()** and is displayed on the screen.

```
#-----
#           MULTITASKING EVENT COUNTER
#
# This is a multitasking event counter project. A button is
# connected to port GPIO 2 of the Raspberry Pi. External events
# are assumed to occur when the button is pressed. The child process
# counts the events and sends to the parent process where the total
# number of events at any time is displayed. The parent and the child
# processes are synchronized when an event occurs
#
# Author: Dogan Ibrahim
# File  : events_sync.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO          # Import RPi
import os                        # Import os
import time                      # Import time
import multiprocessing           # Import multiprocessing
q = multiprocessing.Queue()      # Create a queue
e = multiprocessing.Event()      # Create an event
GPIO.setwarnings(False)         # Disable warnings
GPIO.setmode(GPIO.BCM)

Button = 2                       # Button at GPIO 2
GPIO.setup(Button, GPIO.IN)      # Button is an input

def fork_child():                # Function to fork
    pid = os.fork()              # Create a child
    if pid == 0:                 # If child process
        count=0                  # Set count to 0
        while True:             # Do forever
            while GPIO.input(Button) == 1: # Button is not pressed
                pass
            count = count + 1      # Increment count
            q.put(count)          # Send count to queue
            e.set()               # Set event
            while GPIO.input(Button) == 0: # Button is not released
                pass
    else:
        while True:              # If parent process
            e.wait()             # Wait for event
            cnt = q.get()         # Get event count
```

```

print("Event count = %d" % cnt) # Display event cnt

fork_child()                      # Call function

```

Figure 5.19 Program: *events_sync.py*

Figure 5.20 shows an output displayed on the screen.

```

pi@raspberrypi:~ $ python3 events_sync.py
Event count = 1
Event count = 2
Event count = 3
Event count = 4
Event count = 5
Event count = 6
■

```

Figure 5.20 Output from the program

5.3.7 Project 7 – Up/down counter

Description: This is a multitasking up/down counter project. Two buttons are used with the names: **UP** and **DOWN**. The **UP** button is controlled by a child process, and **DOWN** button by the parent process. Count starts from 0 and pressing **UP** increments the count by 1, pressing **DOWN** decrements the count by 1. When the count reaches 0 it stays at 0. The total count at any time is displayed on the screen.

Block Diagram: The block diagram of the project is shown in Figure 5.21.

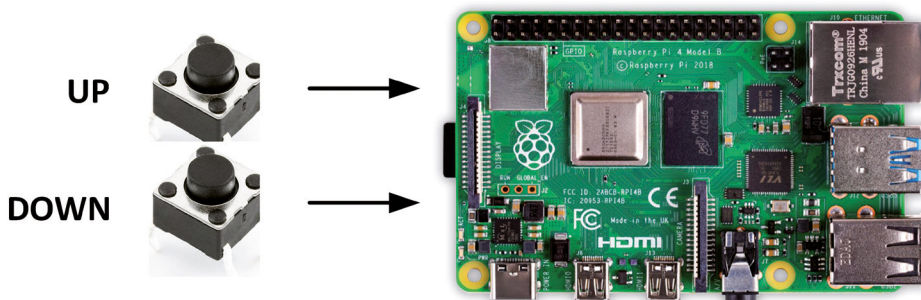


Figure 5.21 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 5.22. The buttons are connected to the Raspberry Pi as follows:

Button	GPIO port
UP	2
DOWN	3

The output state of a button is at logic 1 and goes to logic 0 when the button is pressed.

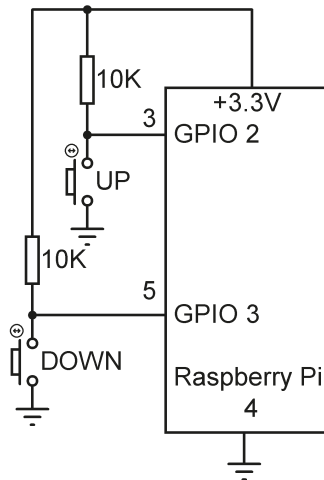


Figure 5.22 Circuit diagram of the project

Program Listing: The program listing is shown in Figure 5.23 (program: **counter.py**). At the beginning of the program, a queue is created with the name **q**, and an event flag is created with the name **e**. The modules used in the program are imported into the program. Buttons **UP** and **DOWN** are assigned to 2 and 3 of the corresponding GPIO ports and are configured as outputs.

Inside the child process, the process waits until button **UP** is pressed, and the latest value of the count is received from the queue and is incremented by 1 and put back into the queue. At the same time, the event flag is set so that the parent process can continue. The parent process receives the latest value of the count, decrements it by 1, and puts it back into the queue. If the count becomes less than 0 then it is set to 0. The total count is displayed on the screen.

```
#-----
#                               UP/DOWN COUNTER
#
# This is an UP/DOWN counter project. 2 buttons are used named as
# UP and DOWN. UP increments the count by 1, and DOWN decrements the
# count by 1. . The child process controls the UP button, while the
# parent process controls the DOWN button. The total count at any time
# is displayed on the screen
#
# Author: Dogan Ibrahim
# File  : counter.py
# Date  : May 2020
#-----
import RPi.GPIO as GPIO          # Import RPi
import os                        # Import os
import multiprocessing           # Import multiprocessing
```

```
q = multiprocessing.Queue()           # Create a queue
e = multiprocessing.Event()           # Create an event
GPIO.setwarnings(False)               # Disable warnings
GPIO.setmode(GPIO.BCM)

UP = 2                                # UP button at GPIO 2
DOWN = 3                              # DOWN button at GPIO 3

GPIO.setup(UP, GPIO.IN)               # UP button is input
GPIO.setup(DOWN, GPIO.IN)            # DOWN button is input

def Start_Child():                    # Function to fork
    pid1 = os.fork()                  # Create a child
    if pid1 == 0:                     # If child process
        while True:                  # Do forever
            while GPIO.input(UP) == 1: # UP is not pressed
                pass
            e.wait()                  # Wait for event flag
            count = q.get()           # Get latest count
            count = count + 1         # Increment count
            q.put(count)              # Latest count to queue
            e.set()                   # Set event flag
            while GPIO.input(UP) == 0: # UP is not released
                pass
            print("Total count = %d" % count) # Display total count
    else:                             # Parent process
        q.put(0)                     # count=0 at beginning
        e.set()                       # Set event flag
        while True:                  # Do forever
            while GPIO.input(DOWN) == 1: # DOWN is not pressed
                pass
            e.wait()                  # Wait for event flag
            cnt = q.get()              # GEt latest count
            cnt = cnt - 1              # Decrement count
            if cnt < 0:                # If negative
                cnt = 0
            q.put(cnt)                # Send count to queue
            e.set()                   # Set event flag
            while GPIO.input(DOWN) == 0: # DOWN not released
                pass
            print("Total count = %d" % cnt) # Display total count

Start_Child()                         # Call Start_Child
```

Figure 5.23 Program: counter.py

To run the program enter the following command:

```
pi@raspberrypi:~ $ python3 counter.py
```

Figure 5.24 shows an output from the program as the UP and DOWN buttons are pressed.

```
pi@raspberrypi:~ $ python3 counter.py
Total count = 1
Total count = 2
Total count = 3
Total count = 4
Total count = 5
Total count = 6
Total count = 7
Total count = 8
Total count = 9
Total count = 8
Total count = 7
Total count = 6
Total count = 5
```

Figure 5.24 Output from the program

5.4 Summary

In this chapter, we learned how to develop simple multitasking projects using process forks. In the next chapter, we will look at threads and how we can develop more complex multitasking applications.

Chapter 6 • Raspberry Pi multitasking projects - using threads

6.1 Overview

In the last chapter, we looked at the concepts of multitasking using process forks. In this chapter, we will learn how to develop multitasking applications using threads.

Fork is a new process that looks exactly like the parent process, but it has its own process ID and memory. It executes independently from the parent process. The child process has a copy of all the pages corresponding to the parent process, and this is loaded into separate memory space by the operating system for the child process. As we saw in the previous chapter, after a child process is created, the variables of the parent and the child processes are not shared. Development of fork based multitasking applications are easy in general and the code developed is easily maintained. Forking is safe and secure because the child processes run in their own virtual address spaces. If for example, one child process crashes, it does not affect any other processes in the system. Threads are in general harder to debug than forks and are not as portable. Forking is faster than threading, especially on a single CPU. Forks have longer startup and stopping times. Also, communication and synchronization between the forked processes are not easy to manage. The management of multitasking is easier with threads as we can easily end, suspend, and resume threads from the parent. When a parent exits in a forked child, we get a ghost process that cannot be managed from the parent.

6.2 Threads

Threads are another way used to create multitasking applications. They are like sub-processes, but the main difference between a thread and process is that the threads created by an application share the same memory space. Threads require less overhead than forking or spawning a new process. The system does not have to initialise a new system virtual memory space and a new environment with a thread. In general, threads are more effective on multi-processor or multi-core systems. Because all threads within a process share the same address space, communication and thread synchronisation is easier to manage and at the same time, the overhead is reduced.

Threads have unique Thread IDs, a unique set of registers and stack pointer, local variables, and priorities. Threads in a process share the following:

- Most of the data
- Open files
- Signals
- Working directory
- Process instructions
- Working directory

Because threads share the same memory space, sharing data between them is easy and very fast. This can cause race problems if multiple threads attempt to work on the same data. The programmer has to make sure that the shared data is accessed properly and safely. There is no process level context switching in multi-threaded applications and as a result of this, threads give higher speeds. Threads are fast to start and terminate.

Threads run inside the same main process, in parallel with the main process. Usually, we create functions inside a main program and then activate these functions to run as independent parallel tasks, all as part of the main program. Threads are easy to program since they can simply be functions in a program and the programmer does not have to worry about creating child processes using fork or exec functions.

Although threads seem to be easy to program and run, they have a major disadvantage in that they are not completely independent processes. There is only one standard input (**sys.stdin**) and only one standard output (**sys.stdout**) and usage of input or output functions within the threads could easily cause conflicts. Also, threads have full access to shared process resources such as global memory and this requires careful planning to avoid any read or write conflicts.

6.3 Forking or Threads?

The answer to this question depends on many factors. Threading is more light-weight than forking and has lower startup and shutdown costs. Interprocess communication is harder and slower in forking than in threading and threading should be chosen if the application requires a large number of interprocess communication. Threads have the disadvantage that if a thread crashes, it may take down all other threads in the process. Also, a faulty or incorrect pointer in a thread can easily corrupt the memory of the parent process or another thread within the same address space.

Running several threads in a program is like running several different programs concurrently, but with the following differences:

- Threads within a process share the same data space and therefore it is easier for the threads to communicate with each other than it is with independent programs. Because threads share the same memory space, the programmer should take extra care to avoid multiple threads to work on the shared data at the same time
- Threads do not have much memory overheads as independent programs
- Threads can be interrupted
- Threads can be suspended while other higher priority threads are running

6.4 Using threads

The function:

thread.start_new(name, arguments)

is used to start a new thread. The child thread starts immediately and calls the function with the passed list of arguments. Here, name is usually the name of a function inside the main program which will start the thread. The argument field is optional and provides the means of passing data to the thread. This field should be a tuple. The newly created thread exists when the function returns (i.e exits). Also, the entire program exits when the main thread (i.e. the main program) exits.

Example

An example program is given below in Figure 6.1 (program: **exthread.py**). In this program, two functions named **Thread1** and **Thread2** are declared and these functions are created as threads by calling to function **thread.start_new()**. There are no arguments passed to these threads in this example. **Thread1** displays the message **I am Thread1...** twice. Similarly, **Thread2** displays the message **I am Thread2...** twice. Both threads wait for one second before they continue. The main program waits forever so that the threads complete their tasks, since terminating the main program will also terminate all the threads.

```
#-----
#           EXAMPLE PROGRAM WITH 2 THREADS
#           =====
#
# In this program two threads are created where each thread
# displays a message
#
# Author: Dogan Ibrahim
# File  : exthread.py
# Date  : MAr 2020
#-----
import _thread
import time

#
# Thread 1
#
def Thread1():
    k = 0                # Initialize k
    while k < 2:         # Do twice
        print("I am Thread1...")    # Display message
        time.sleep(1)           # Wait 1 second
        k = k + 1           # Increment k

#
# Thread 2
#
def Thread2():
    j = 0                # Initialize j
    while j < 2:         # Do twice
        print("I am Thread2...")    # Display message
        time.sleep(1)           # Wait 1 second
        j = j + 1           # Increment j

_thread.start_new_thread(Thread1, ())    # Start Thread1
_thread.start_new_thread(Thread2, ())    # Start Thread2
```

```
while True:
    pass
```

Figure 6.1 Program: *exthread.py*

An example run of the program is shown in Figure 6.2.

```
pi@raspberrypi:~ $ python3 exthread.py
I am Thread1...
I am Thread2...
I am Thread1...
I am Thread2...
```

Figure 6.2 Example run of the program

Example

Another example program is given in Figure 6.3 (program: **exthread2.py**) which shows how data can be passed to a thread through its arguments. In this example, **Thread1** displays the message **This is Thread1** twice, and **Thread2** displays the message **This is Thread2** three times.

```
#-----
#           EXAMPLE PROGRAM WITH 2 THREADS
#           =====
#
# In this program two threads are created where each thread
# displays a message
#
# Author: Dogan Ibrahim
# File  : exthread2.py
# Date  : MAY 2020
#-----
import _thread
import time

#
# Thread 1
#
def Thread1(msg, cnt):
    k = 0                                # Initialize k
    while k < cnt:                       # Do cnt (2) times
        print(msg)                      # Display message
        time.sleep(1)                  # Wait 1 second
        k = k + 1                      # Increment k

#
# Thread 2
#
```

```
def Thread2(msg, cnt):
    j = 0                                # Initialize j
    while j < cnt:                       # Do cnt (3) times
        print(msg)                      # Display message
        time.sleep(1)                  # Wait 1 second
        j = j + 1                      # Increment j

_thread.start_new_thread(Thread1, ("This is Thread1", 2,))
_thread.start_new_thread(Thread2, ("This is Thread2",3,))

while True:
    pass
```

Figure 6.3 Program: exthread2.py

An example run of the program is shown in Figure 6.4.

```
pi@raspberrypi:~ $ python3 exthread2.py
This is Thread2
This is Thread1
This is Thread2
This is Thread1
This is Thread2
```

Figure 6.4 Example run of the program

The program can run in the foreground by entering the command:

```
pi@raspberrypi:~ $ python3 exthread2.py
```

Notice we can also run the program in the background by inserting the **&** character at the end of the command.

In the above program, notice the main parent code waits forever in a while loop and we have to type Cntrl+C keys to stop the program. We can modify the code and wait until both threads complete their tasks and exit normally and cleanly. The modified program is given in the following example.

Example

This example is similar to the previous one, but here a global variable called **ThreadCount** is declared and initialized to zero. This variable is incremented when a thread completes its task. The main program code waits until the **ThreadCount** reaches 2 (both threads finished) and then terminates normally, releasing the keyboard. Figure 6.5 shows the modified program listing (program: **exthread3.py**).

```
#-----
#
#           EXAMPLE PROGRAM WITH 2 THREADS
#           =====
#
# In this program two threads are created where each thread
```

```
# displays a message. The program is terminated when both
# threads terminate
#
# Author: Dogan Ibrahim
# File : exthread3.py
# Date : MAy 2020
#-----
import _thread
import time

ThreadCount = 0

#
# Thread 1
#
def Thread1(msg, cnt):
    global ThreadCount
    k = 0                                # Initialize k
    while k < cnt:                       # Do cnt (2) times
        print(msg)                      # Display message
        time.sleep(1)                  # Wait 1 second
        k = k + 1                      # Increment k
    ThreadCount = ThreadCount + 1       # Increment ThreadCount

#
# Thread 2
#
def Thread2(msg, cnt):
    global ThreadCount
    j = 0                                # Initialize j
    while j < cnt:                       # Do cnt (3) times
        print(msg)                      # Display message
        time.sleep(1)                  # Wait 1 second
        j = j + 1                      # Increment j
    ThreadCount = ThreadCount + 1       # Increment ThreadCount

_thread.start_new_thread(Thread1, ("This is Thread1", 2,))
_thread.start_new_thread(Thread2, ("This is Thread2",3,))

while ThreadCount != 2:                 # Wait until both finish
    pass
print("End of program")
```

Figure 6.5 Program: exthread3.py

An example run of the program is shown in Figure 6.6.

```
pi@raspberrypi:~ $ python3 exthread3.py
This is Thread2
This is Thread1
This is Thread2
This is Thread1
This is Thread2
End of program
pi@raspberrypi:~ $ █
```

Figure 6.6 Example run of the program

Some projects are given in the remaining parts of this chapter to show how multitasking projects can be developed using thread calls.

6.4.1 Project 1 – Two LEDs flashing at different rates

Description: In this project, two LEDs are connected to the Raspberry Pi. Two tasks are created using threads such that one of the LEDs flash every second, while the other one flashes every 250 milliseconds.

Block Diagram: The block diagram of the project is as shown in Figure 5.1.

Circuit Diagram: The circuit diagram of the project is as shown in Figure 5.2

Program Listing: Figure 6.7 shows the program listing (program: **two_leds_thread.py**). At the beginning of the program, modules RPi, thread, and time are imported into the program. Variables LED2 and LED3 are assigned to 2 and 3 which correspond to the GPIO pin numbers. The GPIO mode is then set to BCM and the two LED ports are configured as outputs. There are two threads in the program defined as functions with the names **Flash_LED2** and **Flash_LED3**. Thread **Flash_LED2** flashes the LED at port GPIO 2 every second. Similarly, thread **Flash_LED3** flashes the LED at port GPIO 3 every 250 milliseconds. The threads are started by the following commands:

```
_thread.start_new_thread(Flash_LED2, ())
_thread.start_new_thread(Flash_LED3, ())
```

As a result, you should see the two LEDs flashing at different rates. The program is started using the command:

```
pi@raspberrypi:~ $ python3 two_leds_thread.py
```

Notice that the program runs in the foreground and therefore the keyboard is not available while the program is running. We can run the program in the background by inserting **&** character at the end of the command.

```
#-----
#           TWO LEDS FLASHING AT DIFFERENT RATES
#
# In this project two LEDs are connected to the Raspberry
# Pi. The LEDs flash at different rates using threading
#
# Author: Dogan Ibrahim
# File  : two_leds_thread.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)

LED2 = 2                # LED at GPIO 2
LED3 = 3                # LED at GPIO 3
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED2, GPIO.OUT)    # LED is output
GPIO.setup(LED3, GPIO.OUT)    # LED is output

#
# Thread to flash LED2
#
def Flash_LED2():          # Thread Flash_LED2
    while True:            # Do forever
        GPIO.output(LED2, 1)    # LED2 ON
        time.sleep(1)          # Wait 1 second
        GPIO.output(LED2, 0)    # LED2 OFF
        time.sleep(1)          # Wait 1 second

#
# Thread to flash LED3
#
def Flash_LED3():          # Thread Flash_LED3
    while True:            # Do forever
        GPIO.output(LED3, 1)    # LED3 ON
        time.sleep(0.250)       # Wait 250 ms
        GPIO.output(LED3, 0)    # LED3 OFF
        time.sleep(0.250)       # Wait 250 ms

_thread.start_new_thread(Flash_LED2, ())
```

```
_thread.start_new_thread(Flash_LED3, ())
```

```
while True:
    pass
```

Figure 6.7 Program: two_leds_thread.py

Modified program

In the program in Figure 6.7, the program runs continuously flashing the two LEDs and the keyboard is not available while the program is running. In this modified version of the program (program: **two_leds_thread2.py**) shown in Figure 6.8, the LEDs flash for 10 seconds and then the program terminates.

```
#-----
#           TWO LEDS FLASHING AT DIFFERENT RATES
#
# In this project two LEDs are connected to the Raspberry
# Pi. The LEDs flash at different rates using threading.The
# program terminates after 10 seconds
#
# Author: Dogan Ibrahim
# File  : two_leds_thread2.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)

LED2 = 2                # LED at GPIO 2
LED3 = 3                # LED at GPIO 3
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED2, GPIO.OUT)    # LED is output
GPIO.setup(LED3, GPIO.OUT)    # LED is output

#
# Thread to flash LED2
#
def Flash_LED2():          # Thread Flash_LED2
    while True:            # Do forever
        GPIO.output(LED2, 1)    # LED2 ON
        time.sleep(1)           # Wait 1 second
        GPIO.output(LED2, 0)    # LED2 OFF
        time.sleep(1)           # Wait 1 second

#
```

```

# Thread to flash LED3
#
def Flash_LED3():
    while True:
        GPIO.output(LED3, 1)
        time.sleep(0.250)
        GPIO.output(LED3, 0)
        time.sleep(0.250)

# Thread Flash_LED3
# Do forever
# LED3 ON
# Wait 250 ms
# LED3 OFF
# Wait 250 ms

_thread.start_new_thread(Flash_LED2, ())
_thread.start_new_thread(Flash_LED3, ())

#
# Stop after 10 seconds
#
cnt = 0
while cnt < 10:
    time.sleep(1)
    cnt = cnt + 1
print("End of program")
# Set cnt to 0
# Do 10 times
# Wait 1 second
# Increment cnt
# End of program

```

Figure 6.8 Program: *two_leds_thread2.py*

6.4.2 Project 2 – Up/down counter

Description: This is a multitasking up/down counter project. Three buttons are used with the names: **UP**, **DOWN**, and **CLEAR**. The count starts from 0 and pressing **UP** increments the count by 1, pressing **DOWN** decrements the count by 1. When the count reaches 0 it stays at 0. Pressing button **CLEAR** clears the count to 0. The total count at any time is displayed on the screen.

Block Diagram: The block diagram of the project is shown in Figure 6.9.

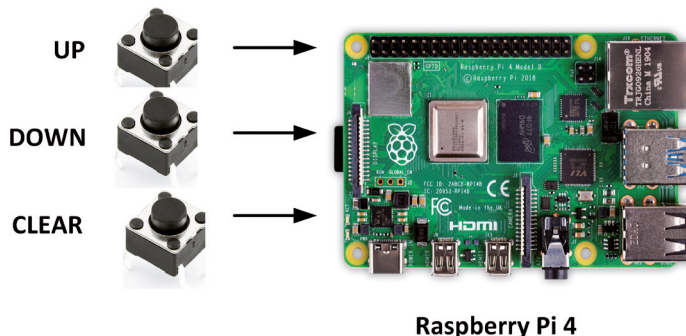


Figure 6.9 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 6.10. The buttons are connected to the Raspberry Pi as follows:

Button	GPIO port
UP	2
DOWN	3
CLEAR	4

The output state of a button is at logic 1 and goes to logic 0 when the button is pressed.

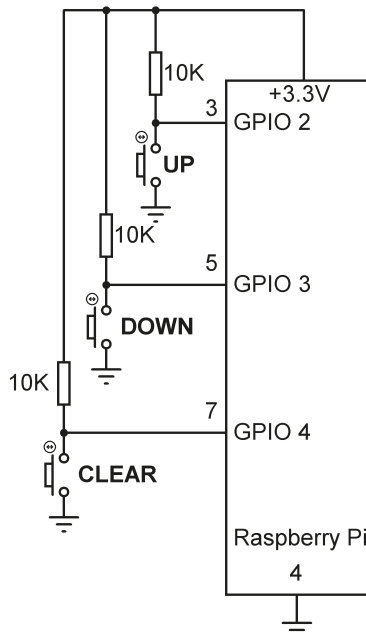


Figure 6.10 Circuit diagram of the project

Program Listing: The program listing is shown in Figure 6.11 (program: **counter_thread.py**). At the beginning of the program, the modules used in the program are imported. Buttons **UP**, **DOWN**, and **CLEAR** are then assigned to 2, 3, and 4 which corresponds to the GPIO port numbers. These ports are configured as inputs. Three threads are created in the program, named **Button_UP**, **Button_DOWN**, and **Button_CLEAR**. Taking **Button_UP** as an example, this thread waits until the **UP** button is pressed. When it is pressed, variable **count** is incremented by 1. The thread then waits until the button is released. The other two threads are similar, but **Button_DOWN** decrements the **count**, and **Button_CLEAR** clears the **count**. The total **count** at any time is displayed on the screen. Notice the display shows only if the value of **count** changes. The program terminates when the total count reaches 100 and displays the message **End of program**.

```
#-----
#                               UP/DOWN COUNTER USING THREADS
#                               =====
#
# This is an up/down counter project. Three buttons named
# UP, DOWN, CLEAR are connected to the Raspberry Pi. Pressing
# UP increments the count, pressing DOWN decrements the count
# and pressing CLEAR clears the count to 0. The total count at
# any time is displayed on the screen. The program terminates
# when count becomes 100
#
# Author: Dogan Ibrahim
# File  : conter_thread.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
GPIO.setwarnings(False)

UP = 2                                # Button UP at GPIO 2
DOWN = 3                              # Button DOWN at GPIO 3
CLEAR = 4                             # Button CLEAR at GPIO 4
GPIO.setmode(GPIO.BCM)               # GPIO mode BCM
GPIO.setup(UP, GPIO.IN)               # UP is input
GPIO.setup(DOWN, GPIO.IN)             # DOWN is input
GPIO.setup(CLEAR, GPIO.IN)            # CLEAR is input

count = 0                             # Count is 0 at beginning
oldcount = count

#
# Thread to control button UP
#
def Button_UP():                       # Thread Button_UP
    global count
    while True:                       # Do forever
        while GPIO.input(UP) == 1:   # UP not pressed
            pass
        count = count + 1             # Increment count
        while GPIO.input(UP) == 0:   # UP not released
            pass

#
# Thread to control button DOWN
#
def Button_DOWN():                     # Thread Button_DOWN
```

```
global count
while True:                                # Do forever
    while GPIO.input(DOWN) == 1:           # DOWN not pressed
        pass
    count = count - 1                       # Decrement count
    if count < 0:                           # If negative
        count = 0                         # Set to 0
    while GPIO.input(DOWN) == 0:           # DOWN not released
        pass

#
# Thread to control button CLEAR
#
def Button_CLEAR():                        # Thread Button_CLEAR
    global count
    global oldcount
    while True:                            # Do forever
        while GPIO.input(CLEAR) == 1:     # CLEAR not pressed
            pass
        count = 0                         # Clear count
        while GPIO.input(CLEAR) == 0:     # CLEAR not released
            pass

#
# Create the threads
#
_thread.start_new_thread(Button_UP, ())
_thread.start_new_thread(Button_DOWN, ())
_thread.start_new_thread(Button_CLEAR, ())

#
# Stop if count equals 100
#
while count <= 100:                        # while count <= 100
    if count != oldcount:                  # If count changed
        print("Count = %d" % count)       # Display count
        oldcount = count

print("End of program")                    # End of program
```

Figure 6.11 Program: counter_thread.py

A typical output from the program is shown in Figure 6.12.

```

pi@raspberrypi:~ $ python3 counter_thread.py
Count = 1
Count = 2
Count = 3
Count = 4
Count = 3
Count = 2
Count = 3
Count = 4
Count = 5

```

Figure 6.12 Typical output from the program

6.4.3 Project 3 – Setting the LED flashing rate from the keyboard

Description: In this project, an LED is connected to the Raspberry Pi. The flashing rate of the LED is dynamically set from the keyboard while the LED is flashing.

Circuit Diagram: The circuit diagram of the project is similar to Figure 5.2, but only one LED is used and is connected to Raspberry Pi port GPIO 2.

Program Listing: Two threads are used in this project. Thread **Flash** flashes the LED, while thread **Rate** changes the flashing rate while the LED is flashing. Initially, the flashing rate is set to 1 second. Figure 6.13 shows the program listing (program: **LEDrate_thread.py**). At the beginning of the program the modules used in the program are imported, LED is assigned number 2 which corresponds to GPIO port 2, and the port is configured as an output. Thread **Flash** flashes the LED with the rate specified by variable **dly**, which is set to 1 at the beginning of the program. Thread **Rate** displays the message **Enter LED flashing rate (secs):** and prompts the user to enter the required flashing rate in seconds. The value received is stored in variable **dly** which changes the LED flashing rate.

```

#-----
#           CHANGE LED FLASHING RATE FROM KEYBOARD
#           =====
#
# In this program an LED is connected to port GPIO 2 of the
# RASpberry Pi. The LED initially flashes every second. The
# flashing rate can be changed from the keyboard while the
# LED is flashing
#
# Author: Dogan Ibrahim
# File  : LEDrate_thread.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)

```

```
LED = 2                                # LED at GPIO 2
GPIO.setmode(GPIO.BCM)                 # GPIO mode BCM
GPIO.setup(LED, GPIO.OUT)               # LED is output
dly = 1                                # Default flashing rate

#
# Thread to flash the LED
#
def Flash():                             # Thread Flash
    global dly
    while True:                         # Do forever
        GPIO.output(LED, 1)             # LED ON
        time.sleep(dly)                 # Wait dly seconds
        GPIO.output(LED, 0)             # LED OFF
        time.sleep(dly)                 # Wait dly seconds

#
# Thread to change the LED flashing rate
#
def Rate():                              # Thread Rate
    global dly
    while True:                         # Do forever
        dly = float(input("Enter LED flashing rate (secs): "))

#
# Create the threads
#
_thread.start_new_thread(Flash, ())
_thread.start_new_thread(Rate, ())

while True:
    pass
```

Figure 6.13 Program: LEDrate_thread.py

Figure 6.14 shows an example run of the program.

```
pi@raspberrypi:~ $ python3 LEDrate_thread.py
Enter LED flashing rate (secs): 2
Enter LED flashing rate (secs): 0.2
Enter LED flashing rate (secs): 0.1
Enter LED flashing rate (secs): 0.5
Enter LED flashing rate (secs): █
```

Figure 6.14 Example run of the program

6.4.4 Project 4 – Setting the LED flashing rate using a button

Description: In this project, an LED and two buttons are connected to the Raspberry Pi. By default the LED flashes every second. The buttons are named **UP** and **DOWN**. Each time **UP** is pressed, the flashing rate is incremented by 200 milliseconds. Similarly, each time

DOWN is pressed, the flashing rate is decremented by 200 milliseconds. If the flashing rate becomes less than 0 then it is set to 0.

Block Diagram: Figure 6.15 shows the block diagram of the project.

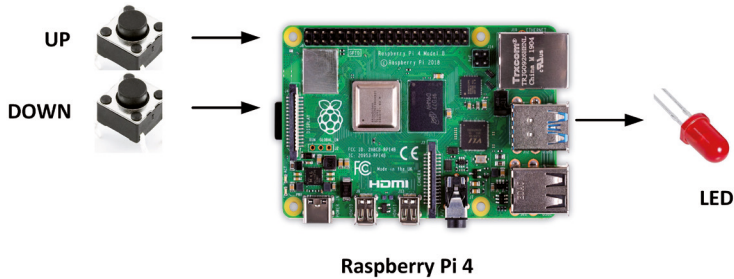


Figure 6.15 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 6.16. Buttons UP and DOWN are connected to GPIO 2 and GPIO 3 respectively. The LED is connected to GPIO 4.

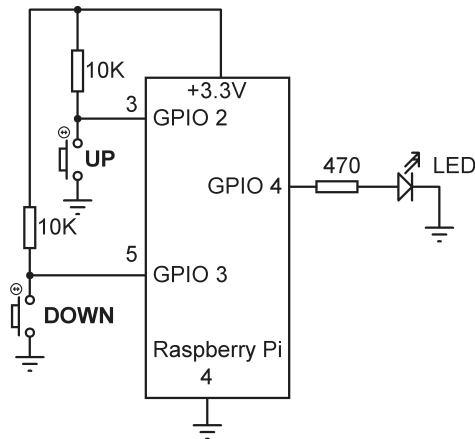


Figure 6.16 Circuit diagram of the project

Program Listing: Figure 6.17 shows the program listing (program: **rate_thread.py**). At the beginning of the program the modules used are imported to the program, buttons **UP**, **DOWN**, and the **LED** are assigned to 2, 3, and 4 to correspond to GPIO 2, GPIO 3, and GPIO 4 respectively. The buttons are configured as inputs and the LED is configured as output. There are three threads in this program: **Flash**, **UP_Button**, and **DOWN_Button**. Thread **Flash** flashes the LED with the rate specified by global variable **dly**. Thread **UP_Button** increases the flashing rate by 200ms when button **UP** is pressed. Similarly, thread **DOWN_Button** decrements the flashing rate by 200ms when button **DOWN** is pressed.

```
#-----
#           CHANGE LED FLASHING RATE USING BUTTONS
#           =====
#
# In this program an LED and two buttons are connected to the
# Raspberry Pi. The default LED flashing rate is 1 second.The
# buttons are named UP and DOWN. Pressing UP increases the flashing
# rate by 200ms. Similarly, pressing DOWN decreases the flashing
# rate by 200ms.
#
# Author: Dogan Ibrahim
# File  : rate_thread.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)

UP = 2                # Button UP at GPIO 2
DOWN = 3              # Button DOWN at GPIO 3
LED = 4               # LED at GPIO 4
GPIO.setmode(GPIO.BCM) # GPIO mode BCM
GPIO.setup(UP, GPIO.IN) # UP is input
GPIO.setup(DOWN, GPIO.IN) # DOWN is input
GPIO.setup(LED, GPIO.OUT) # LED is output
dly = 1.0             # Default flashing rate

#
# Thread to flash the LED
#
def Flash():          # Thread Flash
    global dly
    while True:       # Do forever
        GPIO.output(LED, 1) # LED ON
        time.sleep(dly)    # Wait dly seconds
        GPIO.output(LED, 0) # LED OFF
        time.sleep(dly)    # Wait dly seconds
#
# Thread to increase the flashing rate
#
def UP_Button():      # Thread UP_Button
    global dly
    while True:       # Do forever
        while GPIO.input(UP) == 1: # UP not pressed
            pass
```

```

        dly = dly + 0.2                # Increase flashing rate
        while GPIO.input(UP) == 0: # UP not released
            pass

#
# Thread to decrease the flashing rate
#
def DOWN_Button():                    # Thread DOWN_Button
    global dly
    while True:                      # Do forever
        while GPIO.input(DOWN) == 1: # DOWN not pressed
            pass
        dly = dly - 0.2              # Decrease flashing rate
        if dly < 0:
            dly = 0
        while GPIO.input(DOWN) == 0: # DOWN not released
            pass

#
# Create the threads
#
_thread.start_new_thread(Flash, ())
_thread.start_new_thread(UP_Button, ())
_thread.start_new_thread(DOWN_Button, ())

while True:
    pass

```

Figure 6.17 Program: rate_thread.py

6.4.5 Project 5 – Two-digit 7-segment display seconds counter

Description: In this project, a 7-segment 2-digit multiplexed LED display is used as a counter to count up every second from 0 to 99. Multi-digit 7-segment displays require continuous refreshing of their digits so that the human eye sees the digits as steady and non-flashing. This technique is used to enable each digit for a short time (e.g. 10ms) so that the human eye sees both digits ON at any time. This process requires the digits to be enabled alternately and continuously. The processor cannot perform any other tasks and is busy refreshing the digits all of the time. One technique used in non-multitasking systems is timer interrupts and refreshing the digits in the timer interrupt service routines. In this project, we will be employing a multitasking approach to refresh display digits so that the processor can carry out other tasks. The project aims to show how the digits of a multiplexed 2-digit 7-segment LED display can be refreshed in a task, while another task sends data to the display to count up in seconds from 00 to 99.

7-segment LED displays: Displaying data is one of the fundamental output activities of any microcontroller system. For example, displays are used to show sensor data such as temperature, humidity, pressure, etc. Several types of display devices can be used in microcontroller-based systems. LCDs and 7-segment displays are probably two of the most commonly used display devices. There are several types of LCDs, such as text-based LCD, graphics LCDs, colour LCDs, touch screen LCDs, etc. 7-segment displays are used to display numeric or alphanumeric values, and they can have one or more digits. One digit displays can only display numbers from 0 to 9. Two-digit displays can display numbers from 0 to 99, three-digit displays numbers from 0 to 999, and so on. In this project, a two-digit 7-segment display is used.

As shown in Figure 6.18, a 7-segment LED display consists of 7 LEDs connected such that numbers from 0 to 9 and some letters can be displayed. Segments are identified by letters from **a** to **g**. Figure 6.19 shows the segment names of a typical 7-segment display.

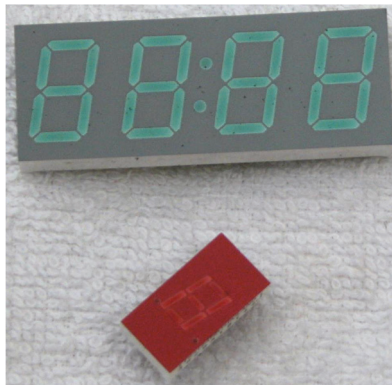


Figure 6.18 Some 7-segment displays

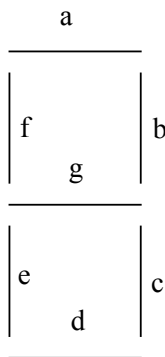


Figure 6.19 Segment names of a 7-segment display

Figure 6.20 shows how numbers from 0 to 9 can be obtained by turning ON or OFF different segments of the display.

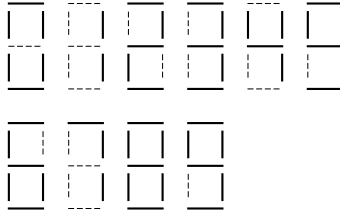


Figure 6.20 Displaying numbers 0 – 9

Multiplexed 7-segment LED displays are available in two different configurations: **common cathode** and **common anode**. As shown in Figure 6.21, in common cathode configuration, all cathodes of all segment LEDs are connected to ground. The segments are then turned ON by applying a logic 1 to the required segment LED via current limiting resistors. In common cathode configuration, the 7-segment LED is connected to the microcontroller in current sourcing mode.

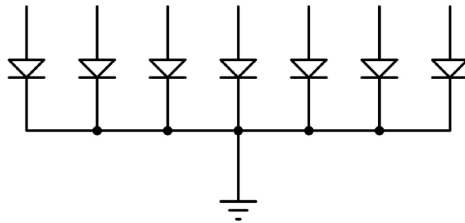


Figure 6.21 Common cathode 7-segment LED display

In a common anode configuration, the anode terminals of all the LEDs are connected as shown in Figure 6.22. This common point is then normally connected to the supply voltage. A segment is turned ON by connecting its cathode terminal to logic 0 via a current limiting resistor. In common anode configuration, the 7-segment LED is connected to the microcontroller in current sinking mode.

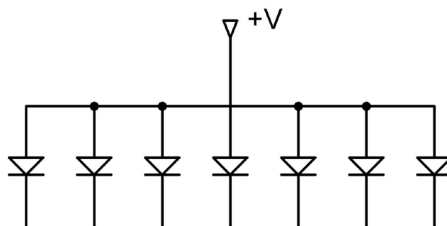


Figure 6.22 Common anode 7-segment LED display

In multiplexed LED applications (for example, see Figure 6.23 for a 2-digit multiplexed LED display), the LED segments of all the digits are tied together and the common pins of each digit are turned ON separately by the microcontroller. By displaying each digit for several

milliseconds the eye can not differentiate that the digits are not ON all the time. This way we can multiplex any number of 7-segment displays together. For example, to display number 57, we have to send 5 to the first digit and enable its common pin. After a few milliseconds, number 7 is sent to the second digit and the common point of the second digit is enabled. When this process is continuously repeated, the user sees it as if both displays are ON continuously.

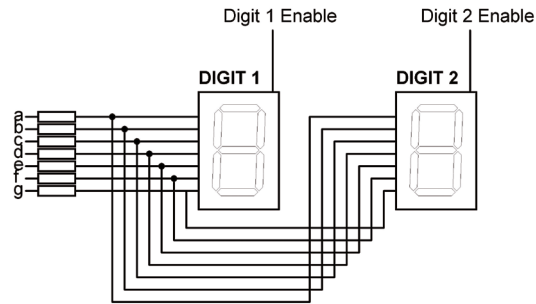


Figure 6.23 2-digit multiplexed 7-segment LED display

Some manufacturers provide multiplexed multi-digit displays in single packages. For example, we can purchase 2,4, or 8 digit multiplexed displays in a single package. The display used in this project is the DC56-11EWA which is a red colour, 0.56 inch height common-cathode two-digit multiplexed display with 18 pins, where the pin configuration is shown in Table 6.1. This display can be controlled from the microcontroller as follows:

- Send the segment bit pattern for digit 1 to segments a to g
- Enable digit 1
- Wait for a few milliseconds
- Disable digit 1
- Send the segment bit pattern for digit 2 to segments a to g
- Enable digit 2
- Wait for a few milliseconds
- Disable digit 2
- Repeat the above process continuously

Pin no	Segment
1,5	e
2,6	d
3,8	c
14	digit 1 Enable
17,7	g
15,10	b
16,11	a
18,12	f
13	digit 2 Enable
4	decimal Point1
9	decimal Point 2

Table 6.1 Pin configuration of DC56-11EWA dual display

The segment configuration of DC56-11EWA display is shown in Figure 6.24. In a multiplexed display application, the segment pins of corresponding segments are connected. For example, pins 11 and 16 are connected as the common **a** segment. Similarly, pins 15 and 10 are connected as the common **b** segment and so on.

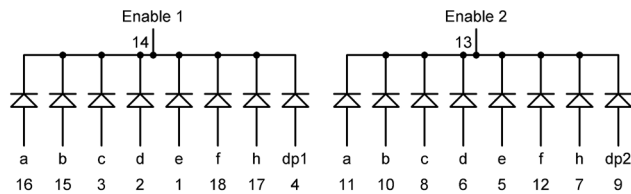


Figure 6.24 DC56-11EWA display segment configuration

Block Diagram: Figure 6.25 shows the block diagram of the project.

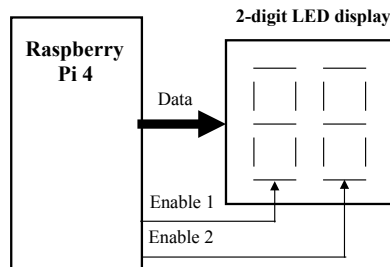


Figure 6.25 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 6.26. In this project, the following pins of the Raspberry Pi are used to interface with the 7-segment LED display:

7-Segment Display pin	Raspberry Pi GPIO	Physical pin no
a	2	3
b	3	5
c	4	7
d	17	11
e	27	13
f	22	15
g	10	19
E2	9 (via transistor)	21
E1	11 (via transistor)	23

7-segment display segments are driven from the port pins through 470 Ohm current limiting resistors. Digit enable pins E1 and E2 are driven from port pins GPIO 11 and GPIO 9 respectively through two BC108 type NPN transistors (any other NPN transistor can be used here), used as switches. The collectors of these transistors drive the segment digits. The segments are enabled when the base of the corresponding transistor is set to logic 1. Notice the following pins of the display are connected to form a multiplexed display:

16 and 11, 15 and 10, 3 and 8, 2 and 6, 1 and 5, 17 and 7, 18 and 12

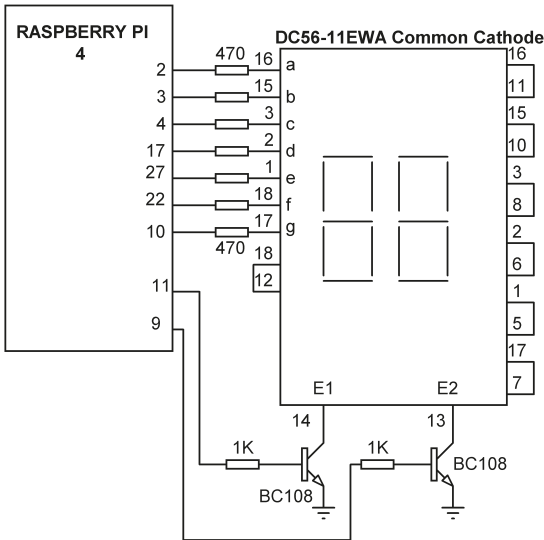


Figure 6.26 Circuit diagram of the project

Program Listing: Before driving the display, we have to know the relationship between the numbers to be displayed and the corresponding segments to be turned ON, and this is shown below:

Number to be displayed	LED bit pattern (a,b,c,d,e,f,g)
1	1,1,1,1,1,1,0
2	0,1,1,0,0,0,0
3	1,1,0,1,1,0,1
4	1,1,1,1,0,0,1
5	0,1,1,0,0,1,1
6	1,0,1,1,0,1,1
7	1,0,1,1,1,1,1
8	1,1,1,0,0,0,0
9	1,1,1,1,1,1,1
10	1,1,1,1,0,1,1

Figure 6.27 shows the program listing (program: **SevenCount.py**). At the beginning of the program, the connections between the LED segments and GPIO pins are defined in list variable **LED_Segments**. Also, the connections between the LED digits and the GPIO pins are defined in the list variable **LED_Digits**. These GPIO pins are then configured as outputs and are all cleared to 0. Variable **count** is initialized to zero at the beginning of the program. The program consists of two threads, named: **Refresh** and **Up_Count**. Thread **Refresh** displays the number in variable **count** and refreshes the multiplexed LED continuously every millisecond so that the digits seem to be ON all the time. If the number to be displayed is less than 10, a 0 is inserted in front of the number so that the numbers 0 to 9 are displayed as 00 to 09. Thread **Up_Count** simply increment variable count every second. When count reaches 100 it is cleared back to 0 and the counting continues.

```
#-----
#           2 DIGIT SEVEN SEGMENT LED COUNTER
#           =====
#
# This is a 2 digit 7-segment LED counter program. The program
# counts up every second from 0 to 99 continuously. The LED matrix
# is refreshed in a thread. The connections between the 7-segment
# LED and the RaspberryPi are as follows:
#
# 7-Segment LED      GPIO
#   a                2
#   b                3
#   c                4
#   d               17
#   e               27
#   f               22
#   g               10
#   E2 (digit)       9
#   E1 (digit)      11
#
# Author: Dogan Ibrahim
# File  : SevenCount.py
```

```
# Date : May 2020
#-----
import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)           # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (9, 11)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)      # Segments are outputs
    GPIO.output(seg, 0)           # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)      # Digits are outputs
    GPIO.output(dig, 0)           # Clear all digits

LED_Bits = {
'0':(1,1,1,1,1,1,0),             # 0
'1':(0,1,1,0,0,0,0),             # 1
'2':(1,1,0,1,1,0,1),             # 2
'3':(1,1,1,1,0,0,1),             # 3
'4':(0,1,1,0,0,1,1),             # 4
'5':(1,0,1,1,0,1,1),             # 5
'6':(1,0,1,1,1,1,1),             # 6
'7':(1,1,1,0,0,0,0),             # 7
'8':(1,1,1,1,1,1,1),             # 8
'9':(1,1,1,1,0,1,1)}             # 9

count = 0                         # Initialzie count

#
# Thread to refresh the 7-segment LED
#
def Refresh():                    # Thread Refresh
    global count
    while True:                  # Do forever
        cnt = str(count)         # into string
        if len(cnt) < 2:
            cnt = "0" + cnt      # Make sure 2 digits
            for dig in range(2): # Do for 2 digits
```

```

        for loop in range(0,7):          # Do for all segments
            GPIO.output(LED_Segments[loop], LED_Bits[cnt[dig]][loop])
        GPIO.output(LED_Digits[dig], 0)
        time.sleep(0.001)
        GPIO.output(LED_Digits[dig], 1)

#
# Thread to count up every second
#
def UP_Count():                          # Thread UP_Count
    global count
    while True:                          # Do forever
        time.sleep(1)                    # Wait a second
        count = count + 1                # Increment count
        if count == 100:                 # If count = 100
            count = 0

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(UP_Count, ())

while True:
    pass

```

Figure 6.27 Program: SevenCount.py

Modified program

In the program shown in Figure 6.27, numbers less than 10 are displayed with a leading 0 (e.g. 05). We can remove the leading zero by blanking all segments of the left-hand digit if the number is less than 10 so that for example number five is displayed as 5 and not as 05. The modified program listing (program: **SevenCount2.py**) is shown in Figure 6.28. Here, list **LED_Bits** is modified by adding a blank line where all the segment bits are set to 0. Additionally, a blank character is inserted to the front of string **cnt** if the number is less than 10 so that the leading digit is blanked.

```

#-----
#           2 DIGIT SEVEN SEGMENT LED COUNTER
#           =====
#
# This is a 2 digit 7-segment LED counter program. The program
# counts up every second from 0 to 99 continuously. The LED matrix
# is refreshed in a thread. The connections between the 7-segment
# LED and the RaspberryPi are as follows:
#

```

```

# 7-Segment LED      GPIO
#    a                2
#    b                3
#    c                4
#    d               17
#    e               27
#    f               22
#    g               10
#    E2 (digit)       9
#    E1 (digit)      11
#
# In this version of the program leading 0s are blanked if the
# number to be displayed is less than 10
#
# Author: Dogan Ibrahim
# File  : SevenCount2.py
# Date  : May 2020
#-----
import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)           # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (9, 11)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)      # Segments are outputs
    GPIO.output(seg, 0)           # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)      # Digits are outputs
    GPIO.output(dig, 0)           # Clear all digits

LED_Bits = {
    ' ': (0,0,0,0,0,0,0),         # Blank
    '0': (1,1,1,1,1,1,0),         # 0
    '1': (0,1,1,0,0,0,0),         # 1
    '2': (1,1,0,1,1,0,1),         # 2
    '3': (1,1,1,1,0,0,1),         # 3
    '4': (0,1,1,0,0,1,1),         # 4
    '5': (1,0,1,1,0,1,1),         # 5

```

```

'6':(1,0,1,1,1,1,1),          # 6
'7':(1,1,1,0,0,0,0),          # 7
'8':(1,1,1,1,1,1,1),          # 8
'9':(1,1,1,1,0,1,1)}          # 9

count = 0                       # Initialzie count

#
# Thread to refresh the 7-segment LED
#
def Refresh():                  # Thread Refresh
    global count
    while True:                # Do forever
        cnt = str(count)       # into string
        if len(cnt) < 2:
            cnt = " " + cnt     # Make sure 2 digits
        for dig in range(2):    # Do for 2 digits
            for loop in range(0,7): # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[cnt[dig]][loop])
            GPIO.output(LED_Digits[dig], 0)
            time.sleep(0.001)
            GPIO.output(LED_Digits[dig], 1)

#
# Thread to count up evey second
#
def UP_Count():                 # Thread UP_Count
    global count
    while True:                # Do forever
        time.sleep(1)          # Wait a second
        count = count + 1      # Increment count
        if count == 100:       # If count = 100
            count = 0

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(UP_Count, ())

while True:
    pass

```

Figure 6.28 Program: SevenCount2.py

6.4.6 Project 6 – Two digit 7-segment temperature display

Description: In this project, the ambient temperature is read using a temperature sensor chip. It is then displayed on a 7-segment 2-digit multiplexed LED display as an integer number every minute. Two threads are used in the program, where one thread reads the temperature and another thread displays it on the 7-segment LED while at the same time refreshes the LED continuously.

Block Diagram: Figure 6.29 shows the block diagram of the project. In this project, a DHT11 temperature and humidity sensor chip is used to read the ambient temperature.

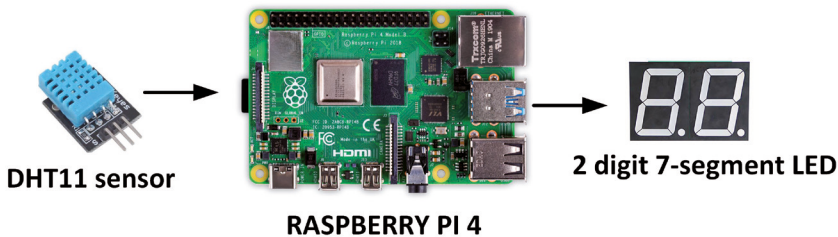


Figure 6.29 Block diagram of the project

Circuit diagram: A DHT11 type temperature and humidity sensor chip (see Figure 6.30) is used in this project. This is normally a 3-pin sensor (there is also a 4-pin version of this sensor where one of the pins is not used) with pins GND, +V, and Data. GND and +V are connected to the ground and the +3.3V power supply pins of the Raspberry Pi. The Data pin must be connected to +V through a 10K resistor. In this project, a 3-pin DHT11 from Elektor is used with a built-in 10K pull-up resistor. As shown in Figure 6.31, the data pin of the sensor is named as **S** and is connected to GPIO 26 of the Raspberry Pi. The 7-segment LED is connected to the Raspberry Pi as in the previous project.

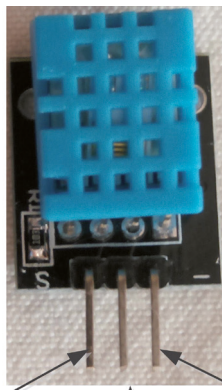


Figure 6.30 DHT11 sensor

DHT11 uses a capacitive humidity sensor and a thermistor to measure ambient temperature. Data output is available from the chip around every two seconds. The basic features of DHT11 are:

- 3 to 5V operation
- 2.5mA current consumption (during a conversion)
- Temperature reading in the range 0-50°C with an accuracy of $\pm 2^{\circ}\text{C}$
- Humidity reading in the range 20-80% with 5% accuracy
- Breadboard compatible with 0.1-inch pin spacings

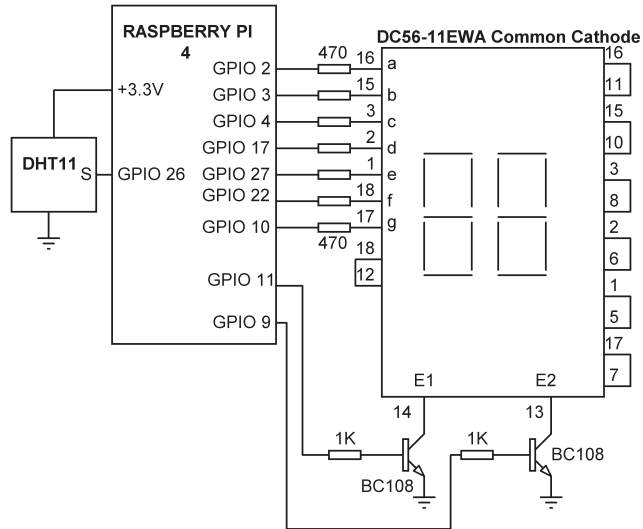


Figure 6.31 Circuit diagram of the project

Construction: The project was constructed on a breadboard with female-male jumper cables used to connect the sensor to the GPIO ports of the Raspberry Pi.

Program listing: In this program, the **Adafruit DHT11 library (Adafruit_DHT)** module is used. This module should be installed to the Raspberry Pi before it can be used. The instructions for this are as follows:

Go to command mode and enter:

```
pi@raspberrypi:~ $ git clone https://github.com/adafruit/Adafruit_Python_DHT.git
```

Change directory to:

```
pi@raspberrypi:~ $ cd Adafruit_python_DHT
pi@raspberrypi:~/Adafruit_python_DHT $
```

Enter the following commands:

```
pi@raspberrypi:~/Adafruit_python_DHT $ sudo apt-get install build-essential python-dev
```

Figure 6.32 shows the program listing (program: **dht11.py**). At the beginning of the program, the modules used in the program are imported including the **Adafruit_DHT** library. This library supports both DHT11 and DHT22 type sensors. Here, the sensor type is specified as DHT11 by the statement **Adafruit_DHT.DHT11**. Variable **S** is set to 26 which corresponds to GPIO 26. The parts of the code that initialises the 7-segment variables and segments is the same as in the previous project and is not repeated here. Also, thread **Refresh** is the same as in the previous project. Thread **Get_Temp** reads the humidity and the temperature from the DHT11 sensor every minute. In this project, only the temperature readings are used and stored in variable **temperature**. The temperature is then converted into an integer number and is stored in variable **temp** which is used by thread **Refresh** to display the temperature.

```
#-----
#           DISPLAY TEMPERATURE ON 7-SEGMENT LED
#           =====
#
# In this program the ambient temperature is displayed on a 2 digit
# 7-segment LED every minute. DHT11 type temperature sensor is used
# in this project and is connected to GPIO 26. The 7-segment LED is
# connected to the RAspberry Pi as follows:
#
# 7-Segment LED    GPIO
#   a              2
#   b              3
#   c              4
#   d             17
#   e             27
#   f             22
#   g             10
#   E2 (digit)     9
#   E1 (digit)    11
#
#
# Author: Dogan Ibrahim
# File  : dht11.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time

import Adafruit_DHT                                # Adafruit DHT11 library
```

```

sensor = Adafruit_DHT.DHT11
S = 26                                # DHT11 on GPIO 26
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (9, 11)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)          # Segments are outputs
    GPIO.output(seg, 0)                # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)          # Digits are outputs
    GPIO.output(dig, 0)                # Clear all digits

LED_Bits = {
    ' ': (0,0,0,0,0,0,0),             # Blank
    '0': (1,1,1,1,1,1,0),             # 0
    '1': (0,1,1,0,0,0,0),             # 1
    '2': (1,1,0,1,1,0,1),             # 2
    '3': (1,1,1,1,0,0,1),             # 3
    '4': (0,1,1,0,0,1,1),             # 4
    '5': (1,0,1,1,0,1,1),             # 5
    '6': (1,0,1,1,1,1,1),             # 6
    '7': (1,1,1,0,0,0,0),             # 7
    '8': (1,1,1,1,1,1,1),             # 8
    '9': (1,1,1,1,0,1,1)}             # 9

temp = 0                              # Initialzie count

#
# Thread to refresh the 7-segment LED
#
def Refresh():                          # Thread Refresh
    global temp
    while True:                        # Do forever
        cnt = str(temp)                # into string
        if len(cnt) < 2:
            cnt = " " + cnt            # Make sure 2 digits
        for dig in range(2):           # Do for 2 digits
            for loop in range(0,7):    # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[cnt[dig]][loop])

```

```
GPIO.output(LED_Digits[dig], 0)
time.sleep(0.001)
GPIO.output(LED_Digits[dig], 1)

#
# Thread to get the ambient temperature from DHT11 every minute
#
def Get_Temp():                                # Thread Get_Temp
    global temp
    while True:                                # Do forever
        humidity,temperature = Adafruit_DHT.read_retry(sensor, S)
        temp = int(temperature)
        time.sleep(60)

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(Get_Temp, ())

while True:
    pass
```

Figure 6.32 Program: dht11.py

6.4.7 Project 7 – Square waveform generator with 7-segment LED display and keyboard

Description: In this project, square waveform is generated. The frequency can be set from the keyboard while the waveform is being displayed. The selected frequency can be set between 1 and 99Hz and is displayed on the 7-segment LED.

Circuit diagram: The 7-segment LED is connected to the Raspberry Pi as in Project 5 (Figure 6.26). The generated waveform is sent to port pin GPIO 26.

Program listing: In this program, the square waveform is generated using a PWM (Pulse Width Modulation) channel of the Raspberry Pi. PWM waveforms are frequently used in power control applications. The waveform is a positive only square waveform with variable ON and OFF times. As shown in Figure 6.33, the total of the ON and OFF times is known as the **Period** of the waveform. The ratio of the ON time to the period is known as **Duty Cycle** and is represented as a percentage. i.e.

$$\text{Duty Cycle} = (T / P) \times 100\%$$

where, T is the ON time, and P is the period (ON time + OFF time).

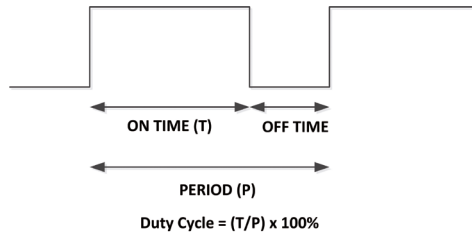


Figure 6.33 PWM waveform

By varying the duty cycle from 0% to 100% we can easily control a load, e.g. a motor. For example, at a 50% duty cycle, the load receives half of the total power. Similarly, at 100% duty cycle the load receives full power.

The Raspberry Pi Python PWM library supports the following commands:

- | | |
|---|--|
| <code>P = GPIO.PWM(channel, frequency)</code> | - Configure channel for PWM with specified frequency |
| <code>p.start(DC)</code> | - Start PWM with specified Duty Cycle |
| <code>p.stop()</code> | - Stop PWM |
| <code>p.ChangeFrequency(frequency)</code> | - Change PWM frequency |
| <code>p.ChangeDutyCycle(DC)</code> | - Change Duty Cycle |

In this program, GPIO 26 is chosen as the PWM channel. Figure 6.34 shows the program listing (program: **square.py**). The program consists of two threads, named: **Refresh** and **Change_Freq**. Thread **Refresh** refreshes the 7-segment LED display and shows the set frequency of the PWM waveform. Thread **Change_Freq** generates the PWM waveform at GPIO 26 of the Raspberry Pi. The Duty Cycle of the waveform is set to 50% using the function call:

```
p.start(50)
```

The frequency of the waveform is initially assumed to be 50Hz and is set by the function call (notice that frequency must be entered in Hz):

```
p = GPIO.PWM(26, freq)
```

where variable **freq** is set to 50 at the beginning of the program. The user is prompted to set the frequency after displaying the message: **Enter PWM frequency (Hz):** Frequency can be changed by the function call:

```
p.ChangeFrequency(freq)
```

```
#-----
#           GENERATE SQUARE WAVEFORM
#           =====
#
# This program generates square waveforms using the PWM module of
# the Raspberry Pi. The frequency of the waveform can be set between
# 1 and 99Hz through the keyboard. The set frequency is displayed
# on the 7-segment LED display. The waveform is output at GPIO 26 of
# the Raspberry Pi.
# Connections of the 7-segment LED to the Raspberry Pi are as follows:
#
# 7-Segment LED   GPIO
#   a             2
#   b             3
#   c             4
#   d            17
#   e            27
#   f            22
#   g            10
#   E2 (digit)    9
#   E1 (digit)   11
#
# Author: Dogan Ibrahim
# File  : square.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (9, 11)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)          # Segments are outputs
    GPIO.output(seg, 0)                 # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)           # Digits are outputs
    GPIO.output(dig, 0)                 # Clear all digits
```

```

LED_Bits =[
' ': (0,0,0,0,0,0,0),          # Blank
'0': (1,1,1,1,1,1,0),          # 0
'1': (0,1,1,0,0,0,0),          # 1
'2': (1,1,0,1,1,0,1),          # 2
'3': (1,1,1,1,0,0,1),          # 3
'4': (0,1,1,0,0,1,1),          # 4
'5': (1,0,1,1,0,1,1),          # 5
'6': (1,0,1,1,1,1,1),          # 6
'7': (1,1,1,0,0,0,0),          # 7
'8': (1,1,1,1,1,1,1),          # 8
'9': (1,1,1,1,0,1,1)}

freq = 50                        # Initialize frequency

#
# Thread to refresh the 7-segment LED
#
def Refresh():                   # Thread Refresh
    global freq
    while True:                 # Do forever
        f = str(freq)           # into string
        if len(f) < 2:
            f = " " + f         # Make sure 2 digits
        for dig in range(2):     # Do for 2 digits
            for loop in range(0,7): # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[f[dig]][loop])
            GPIO.output(LED_Digits[dig], 0)
            time.sleep(0.001)
            GPIO.output(LED_Digits[dig], 1)

#
# Thread change the PWM frequency
#
def Change_Freq():               # Thread Change_Freq
    global freq
    PWM_Port = 26                # PWM on port 26
    GPIO.setup(PWM_Port, GPIO.OUT) # Configure as output
    p = GPIO.PWM(PWM_Port, freq)  # Initialize
    p.start(50)                   # Start PWM with DC=50%
    while True:                   # Do forever
        freq = int(input("Enter PWM frequency (Hz): "))
        p.ChangeFrequency(freq)

```

```

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(Change_Freq, ())

while True:
    pass

```

Figure 6.34 Program: square.py

6.4.8 Project 8 – Square waveform generator with 7-segment LED display and buttons

Description: This project is similar to Project 7, but here 4 buttons are used to set the frequency of the PWM waveform instead of the keyboard. As a result, the project is portable and is independent of the keyboard. The buttons are named: **UP10**, **UP1**, **DOWN10**, and **DOWN1**. Frequency is initially set to 50Hz, but it can be set to any integer value from 1 to 99Hz. Buttons **UP10** and **UP1** increment the frequency in 10s and 1 unit respectively. For example, pressing button **UP10** will increment frequency to 60Hz. Pressing **UP1** will increment it to 61Hz and so on. Similarly, pressing **DOWN10** and **DOWN1** will decrement frequency in 10s and 1 unit respectively. The frequency can not be below 1Hz or above 99Hz. The 7-segment LED displays the set frequency.

Block Diagram: The block diagram of the project is shown in Figure 6.35.

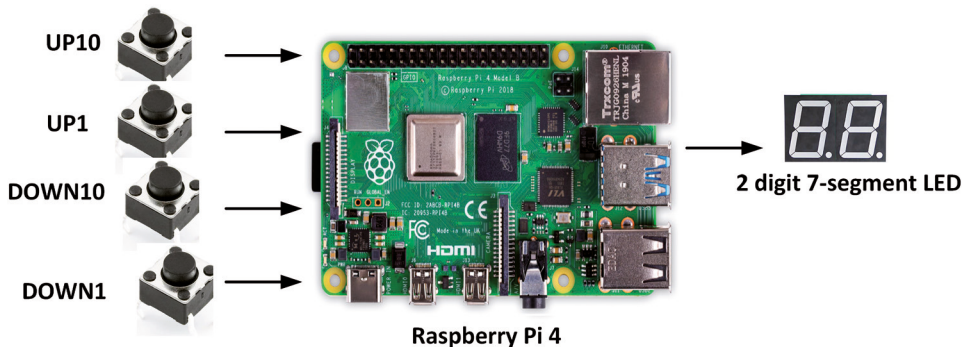


Figure 6.35 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 6.36. The buttons are connected to the Raspberry Pi as follows:

Button	GPIO port	Physical pin
UP10	12	32
UP1	16	36
DOWN10	20	38
DOWN1	21	40

The output state of a button is at logic 1 and goes to logic 0 when the button is pressed.

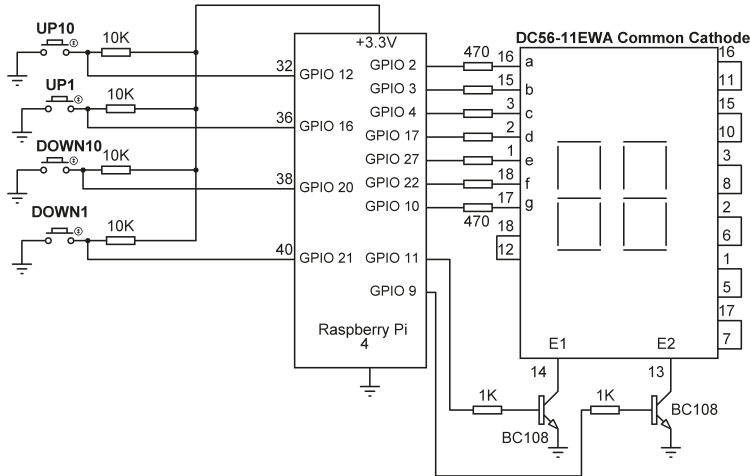


Figure 6.36 Circuit diagram of the project

Program Listing: The program listing is shown in Figure 6.37 (program: **square_buttons.py**). At the beginning of the program, the modules used in the program are imported. Then the 7-segment LED segment and digit variables are defined as in the previous 7-segment LED projects. The default starting frequency is set to 50Hz. Buttons **UP10**, **UP1**, **DOWN10**, and **DOWN1** are set to 12, 16, 20, and 21 respectively to correspond to the GPIO pins. These port pins are configured as inputs. The program consists of 6 threads:

Thread **Refresh** refreshes the 7-segment LED and displays the set frequency value, which is in variable called **freq**.

Thread **Freq_UP10** increments the frequency by 10Hz whenever button **UP10** is pressed. If the frequency goes above 99, it is set to 99. Notice in the previous projects the button pressing action was detected by the following statement:

```
while GPIO.input(UP10) == 1:
    pass
```

The above statement continuously checks the state of the button until it is pressed. This statement consumes too much CPU time as it is running continuously and as a result, the refreshing of the 7-segment LED display may be affected, causing the digits to flicker. In this program, instead of using the **pass** statement, a small delay (0.2 seconds) is used so

that the thread is suspended and releases the CPU time to other threads in the system. As a result, thread **Refresh** has more CPU time, thus eliminating flickering of the digits.

Thread **Freq_UP1** is similar to **Freq_UP10** but here the frequency is incremented by 1Hz.

Thread **Freq_DOWN10** decrements the frequency by 10Hz. If it becomes less than 0 it is set to 1Hz. Similarly, thread **Freq_DOWN1** decrements the frequency by 1Hz.

Thread **Generate_PWM** generates the PWM waveform at GPIO 26. Here, port GPIO 26 is initially configured as an output and the PWM is initialized. The PWM waveform is then generated with a default frequency of 50Hz and the Duty Cycle of 50%. The thread checks if the frequency is changed by the user and if so calls function **ChangeFrequency()** to change the frequency to the required value.

```
#-----
#           GENERATE SQUARE WAVEFORM
#           =====
#
# This program generates square waveforms using the PWM module of
# the Raspberry Pi. The frequency of the waveform can be set between
# 1 and 99Hz through 4 buttons named UP10,UP1,DOWN10 and DOWN1.
# Pressing UP10 or DOWN10 increments or decrements teh freqeency
# by 10Hz respectively. Similarly, pressing UP1 and DOWN1 decrease
# teh frequency by 1Hz. The frequency si set to 50HZ at startup.
# Connections of the 7-segment LED to the Raspberry Pi are as follows:
#
# 7-Segment LED      GPIO
#   a                2
#   b                3
#   c                4
#   d               17
#   e               27
#   f               22
#   g               10
#   E2 (digit)       9
#   E1 (digit)      11
#
# Author: Dogan Ibrahim
# File  : square_buttons.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM
```

```
LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (9, 11)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)          # Segments are outputs
    GPIO.output(seg, 0)                 # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)          # Digits are outputs
    GPIO.output(dig, 0)                 # Clear all digits

LED_Bits = {
    ' ': (0,0,0,0,0,0,0),              # Blank
    '0': (1,1,1,1,1,1,0),              # 0
    '1': (0,1,1,0,0,0,0),              # 1
    '2': (1,1,0,1,1,0,1),              # 2
    '3': (1,1,1,1,0,0,1),              # 3
    '4': (0,1,1,0,0,1,1),              # 4
    '5': (1,0,1,1,0,1,1),              # 5
    '6': (1,0,1,1,1,1,1),              # 6
    '7': (1,1,1,0,0,0,0),              # 7
    '8': (1,1,1,1,1,1,1),              # 8
    '9': (1,1,1,1,0,1,1)}              # 9

freq = 50                              # Initialize frequency
oldfreq = freq
UP10 = 12                               # Button UP10
UP1 = 16                                # Button UP1
DOWN10 = 20                             # Button DOWN10
DOWN1 = 21                              # Button DOWN1
GPIO.setup(UP10, GPIO.IN)               # UP10 is input
GPIO.setup(UP1, GPIO.IN)                # UP1 is input
GPIO.setup(DOWN10, GPIO.IN)             # DOWN10 is input
GPIO.setup(DOWN1, GPIO.IN)              # DOWN1 is input
#
# Thread to refresh the 7-segment LED
#
def Refresh():                           # Thread Refresh
    global freq
    while True:                          # Do forever
        f = str(freq)                   # into string
        if len(f) < 2:                  # Make sure 2 digits
            f = " " + f
```

```

        for dig in range(2):                # Do for 2 digits
            for loop in range(0,7):          # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[f[dig]][loop])
            GPIO.output(LED_Digits[dig], 0)
            time.sleep(0.001)
            GPIO.output(LED_Digits[dig], 1)

#
# Thread UP10
#
def Freq_UP10():                            # Thread Freq_UP10
    global freq
    while True:                             # Do forever
        while GPIO.input(UP10) == 1:       # UP10 not pressed
            time.sleep(0.2)
        freq = freq + 10                    # Increment freq
        if freq > 99:
            freq = 99
        while GPIO.input(UP10) == 0:       # UP10 not released
            time.sleep(0.2)

#
# Thread UP1
#
def Freq_UP1():                             # Thread UP1
    global freq
    while True:                             # Do forever
        while GPIO.input(UP1) == 1:       # UP1 not pressed
            time.sleep(0.2)
        freq = freq + 1                    # Increment freq
        if freq > 99:
            freq = 99
        while GPIO.input(UP1) == 0:       # UP1 not released
            time.sleep(0.2)

#
# Thread DOWN10
#
def Freq_DOWN10():                         # Thread DOWN10
    global freq
    while True:                             # Do forever
        while GPIO.input(DOWN10) == 1:    # DOWN10 not pressed
            time.sleep(0.2)
        freq = freq - 10                   # Decrement freq
        if freq < 0:
            freq = 1

```

```
        while GPIO.input(DOWN10) == 0:      # DOWN10 not released
            time.sleep(0.2)

#
# Thread DOWN1
#
def Freq_DOWN1():                          # Thread DOWN1
    global freq
    while True:                            # Do forever
        while GPIO.input(DOWN1) == 1:      # DOWN1 not pressed
            time.sleep(0.2)
            freq = freq - 1                 # Decrement freq
            if freq < 0:
                freq = 1
        while GPIO.input(DOWN1) == 0:      # DOWN1 not released
            time.sleep(0.2)

#
# Thread Generate_PWM
#
def Generate_PWM():                        # Thread Generate_PWM
    global freq
    global oldfreq
    PWM_Port = 26                          # PWM port
    GPIO.setup(PWM_Port, GPIO.OUT)         # Configure as output
    p =GPIO.PWM(PWM_Port, freq)            # Initialize PWM
    p.start(50)                            # Start with DC=50%
    while True:                            # Change freq
        if freq != oldfreq:
            p.ChangeFrequency(freq)
            oldfreq = freq
        time.sleep(0.5)

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(Freq_UP10, ())
_thread.start_new_thread(Freq_UP1, ())
_thread.start_new_thread(Freq_DOWN10, ())
_thread.start_new_thread(Freq_DOWN1, ())
_thread.start_new_thread(Generate_PWM, ())

while True:
    pass
```

Figure 6.37 Program: square_buttons.py

Figure 6.38 shows the waveform generated on a digital oscilloscope where the set frequency was 50Hz. Here, the horizontal time scale was 10 ms/division and the vertical voltage scale was 2 V/division. The measured frequency of the waveform was 49.7Hz.



Figure 6.38 Generated PWM waveform

6.4.9 Project 9 – Four-digit 7-segment display seconds counter

Description: In this project, a 7-segment 4-digit multiplexed LED display is used as a counter to count up every second from 0 to 9999. The project is very similar to Project 5, but here 4 digits are used instead of 2.

The operation of a 4-digit multiplexed display (Figure 6.39) is similar to the 2-digit display, where the LED segments of the digits are tied together and the common pins of each digit are turned ON separately by the microcontroller. By displaying each digit for several milliseconds, the eye can not differentiate that the digits are not ON all the time. This way we can multiplex any number of 7-segment displays together. For example, to display number 5734, we have to send 5 to the first digit and enable its common pin. After a few milliseconds, number 7 is sent to the second digit and the common point of the second digit is enabled, and so on. When this process is repeated, the user sees as if both displays are ON continuously.



Figure 6.39 4-digit multiplexed 7-segment LED display

The display used in this project is the DC56-11EWA which is a red colour, 0.56 inch height common-cathode two-digit multiplexed display having 18 pins, where the pin configuration is shown in Table 6.1. Two such display modules are used to construct a 4 digit display. Each module has E1 and E2 enable pins.

The segment configuration of DC56-11EWA display is shown in Figure 6.24. In a multiplexed display application, the segment pins of corresponding segments are connected. For example, pins 11 and 16 are connected as the common **a** segment. Similarly, pins 15 and 10 are connected as the common **b** segment and so on.

Block Diagram: Figure 6.40 shows the block diagram of the project.

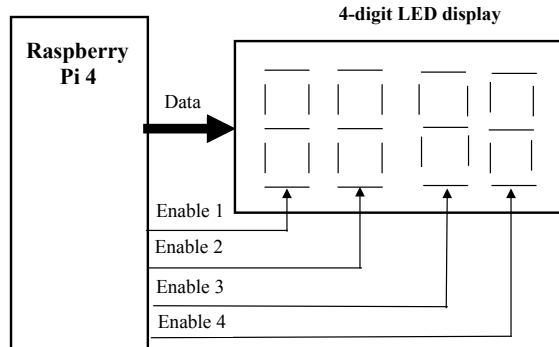


Figure 6.40 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 6.41. In this project, the following pins of the Raspberry Pi are used to interface with the 7-segment LED display:

7-Segment Display pin	Raspberry Pi GPIO	Physical pin no
a	2	3
b	3	5
c	4	7
d	17	11
e	27	13
f	22	15
g	10	19
E1 (first module)	11 (via transistor)	23
E2 (first module)	9 (via transistor)	21
E1 (second module)	21 (via transistor)	40
E2 (second module)	20 (via transistor)	38

7-segment display segments are driven from the port pins through 470 Ohm current limiting resistors. Digit enable pins E1, E2 of the first module and E1, E2 of the second module are driven from port pins GPIO 11, GPIO 9, GPIO 21, and GPIO 20 respectively through four BC108 type NPN transistors (any other NPN transistor can be used here), used as switches. The collectors of these transistors drive the segment digits. The segments are enabled

when the base of the corresponding transistor is set to logic 1. Notice that the following pins of the display are connected to form a multiplexed display:

16 and 11, 15 and 10, 3 and 8, 2 and 6, 1 and 5, 17 and 7, 18 and 12

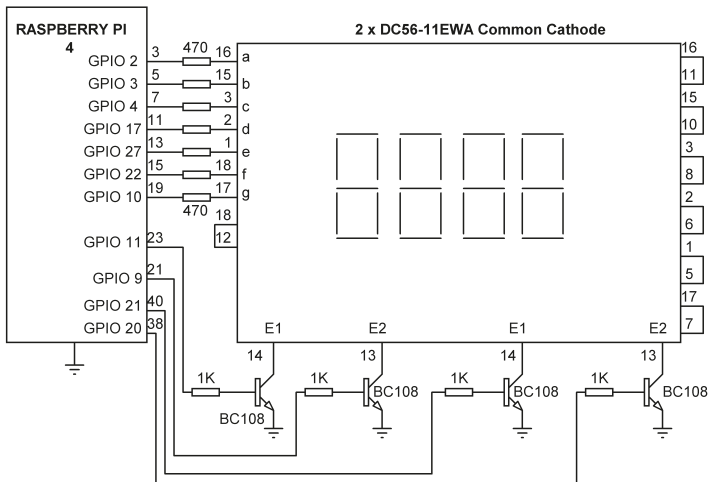


Figure 6.41 Circuit diagram of the project

Program Listing: Figure 6.42 shows the program listing (program: **SevenCount4.py**). At the beginning of the program, the modules are imported. The program is very similar to the one with 2 digits. Here, the list **LED_Digits** contains 4 numbers which are the digit enable pins of the two 7-segment LED modules. The program consists of two threads. Thread **Refresh** is very similar to the one with 2 digits, except that we had to make sure that the number to be displayed (**count**) consists of 4 digits. If the number is less than 4 digits, spaces are inserted in front of it to blank the display for the digit positions. Notice that digit count runs from 0 to 4 and not from 0 to 2 which was the case with the 2-digit display. The delay between the digit enables is reduced to 0.5 ms (it was 1 ms with the 2 digit display) since we have 4 digits and a faster refresh rate is required. Variable **count** starts from 0 at the beginning of the program by default. It is incremented by 1 in Thread **Up_Count**. When the count reaches 10000, it is reset back to 0.

```
#-----
#           4 DIGIT SEVEN SEGMENT LED SECONDS COUNTER
#           =====
#
# This is a 4 digit 7-segment LED seconds counter program. The
# program counts up every second from 0 to 9999 continuously.
# The LED matrix is refreshed in a thread. The connections between
# the 7-segment LED and the RaspberryPi are as follows:
#
# 7-Segment LED   GPIO
#   a              2
```

```
#      b      3
#      c      4
#      d     17
#      e     27
#      f     22
#      g     10
#  E1 module1 11
#  E2 module1  9
#  E1 module2 21
#  E2 module2 20
#
# Author: Dogan Ibrahim
# File  : SevenCount4.py
# Date  : May 2020
#-----
import RPi.GPIO as GPIO
import _thread
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)          # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (11, 9, 21, 20)

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)    # Segments are outputs
    GPIO.output(seg, 0)          # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)    # Digits are outputs
    GPIO.output(dig, 0)          # Clear all digits

LED_Bits = {
    ' ': (0,0,0,0,0,0,0,0),      # Blank
    '0': (1,1,1,1,1,1,0,0),      # 0
    '1': (0,1,1,0,0,0,0,0),      # 1
    '2': (1,1,0,1,1,0,1,1),      # 2
    '3': (1,1,1,1,0,0,1,1),      # 3
    '4': (0,1,1,0,0,1,1,1),      # 4
    '5': (1,0,1,1,0,1,1,1),      # 5
    '6': (1,0,1,1,1,1,1,1),      # 6
    '7': (1,1,1,0,0,0,0,0),      # 7
    '8': (1,1,1,1,1,1,1,1),      # 8
```

```
'9':(1,1,1,1,0,1,1)}                                # 9

count = 0                                              # Initialzie count

#
# Thread to refresh the 7-segment LED
#
def Refresh():                                         # Thread Refresh
    global count
    while True:                                       # Do forever
        cnt = str(count)                             # into string
        if len(cnt) == 3:                             # If 3 digits
            cnt = " " + cnt                           # Add " "
        elif len(cnt) == 2:                           # If 2 digits
            cnt = "  " + cnt                           # Add " "
        elif len(cnt) == 1:                           # If 1 digit
            cnt = "   " + cnt                           # Add " "
        for dig in range(4):                           # Do for 4 digits
            for loop in range(0,7):                     # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[cnt[dig]][loop])
            GPIO.output(LED_Digits[dig], 1)
            time.sleep(0.0005)
            GPIO.output(LED_Digits[dig], 0)

#
# Thread to count up every second
#
def UP_Count():                                       # Thread UP_Count
    global count
    while True:                                       # Do forever
        time.sleep(1)                                  # Wait a second
        count = count + 1                             # Increment count
        if count == 10000:                             # If count = 10000
            count = 0

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(UP_Count, ())

while True:
    pass
```

Figure 6.42 Program: SevenCount4.py

6.4.10 Project 10 – Four-digit 7-segment display conveyor belt object counter

Description: In this project, the objects (e.g bottles) moving on a conveyor belt are counted and displayed on a 7-segment 4-digit multiplexed LED display.

Block Diagram: As shown in Figure 6.43, the project is based on a light-dependent resistor (LDR). A light beam shines from one side of the conveyor belt to the objects moving on the conveyor belt. An LDR on the other side of the conveyor belt detects when an object interrupts the light beam. This generates a logic 1 signal which is used to increment a counter where the total count is displayed on the LED.

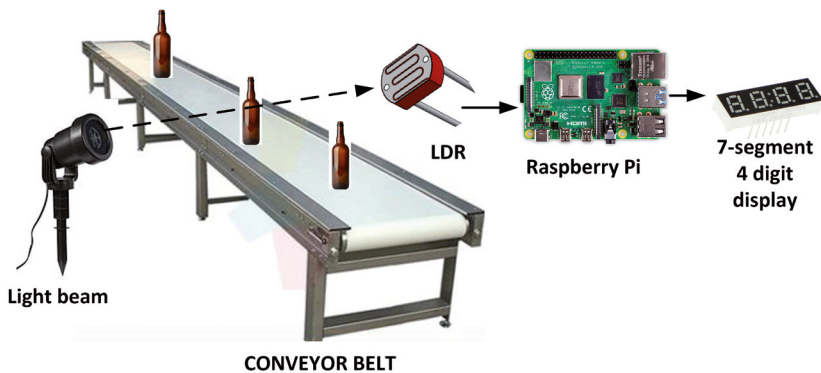


Figure 6.43 Block diagram of the project

The LDR sensor used in this project is known as the **KY-018**. As shown in Figure 6.44 it consists of a photoresistor in series with a 10K resistor. Photoresistors, also known as Light Dependent Resistors (LDR) are light-sensitive 2-pin resistors that can be used to indicate the presence or absence of light. Their dark resistances are very high (up to 1Mohm), but when they are exposed to light their resistances can drop down to several ohms depending on light intensity (Figure 6.45). Modern LDRs are made up of cadmium selenide (CdS), where a light-sensitive tract is created using CdS (Figure 6.46). The sensitivity of a photoresistor varies with light wavelength. In general, they have lower sensitivities than photodiodes or phototransistors. LDRs are also sensitive to temperature changes and because of this, they are not suitable for precise light intensity measurements. LDRs are slow devices. It can take up to 10ms for their resistances to change when light intensity changes. For this reason, LDRs cannot be used in applications where light intensity changes fast. LDRs are normally used with a series resistor where voltage is applied to the resistor. The voltage across the LDR is related to the light intensity striking on the LDR.

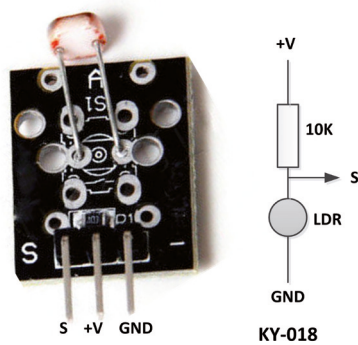


Figure 6.44 KY-018 photoresistor module

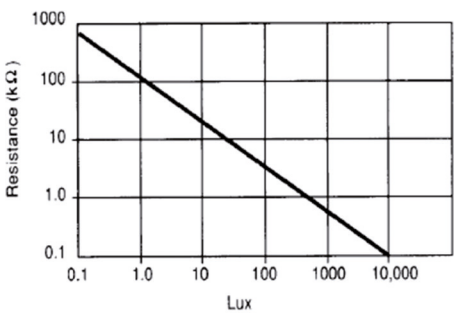


Figure 6.45 Light intensity-resistance characteristic of an LDR

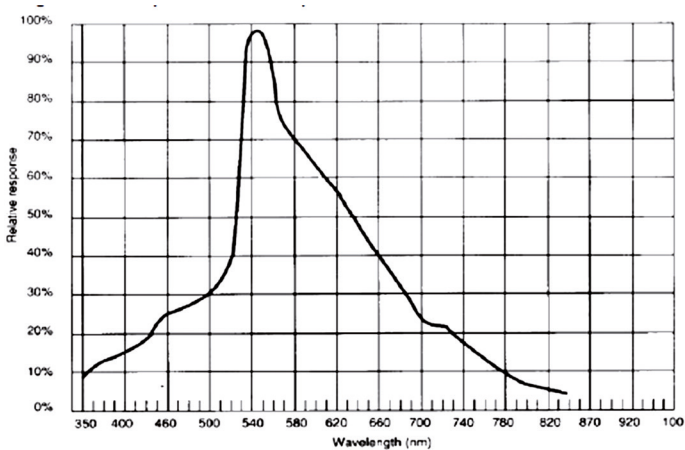


Figure 6.46 LDR sensitivity to light wavelength

Circuit Diagram: When there are no objects in front of the LDR, the light shines directly on the LDR. It is assumed that under such circumstances the resistance of the LDR is about 1K or less. It is assumed when there is an object in front of the light the resistance of the LDR will increase to about 100K. Supplying +3.3V to the LDR module, the voltage at the LDR output will be (see Figure 6.47):

With no object in front of the LDR: $V = 3.3V \times 1K / (10K + 1K) = 0.3V$

With object in front of the LDR: $V = 3.3V \times 100K / (10K + 100K) = 3V$

Therefore, when there is no object in front of the LDR the output logic state can be taken as 0, and when there is an object in front of the LDR we can assume a logic state of 1. The rising edge of the output voltage from the LDR can be used to detect the presence of an object and this voltage can be used as an input to our Raspberry Pi to count the number of objects passing in front of the LDR.

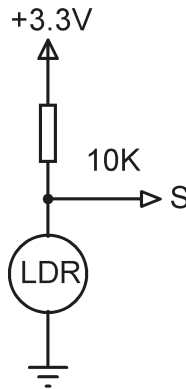


Figure 6.47 The LDR circuit

The display used in this project is the 4 digit 7-segment LED used in Project 9, connected to the Raspberry Pi as in Project 9. Figure 6.48 shows the circuit diagram of the project. The output pin of the LDR (pin S) is connected to GPIO 16.

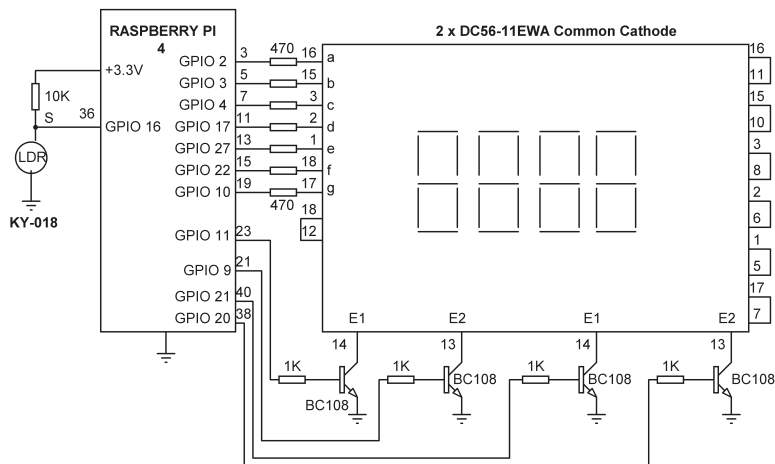


Figure 6.48 Circuit diagram of the project

Program Listing: Figure 6.49 shows the program listing (program: **ldr.py**). This project is similar to Project 9. The program consists of two threads. Thread **Refresh** is the same as in Project 9. It refreshes the display and shows the count at any time. Thread **Object_Count** waits until an object is detected. The output of the LDR (GPIO 16) is normally at logic 0 and goes to logic 1 when an object is detected (i.e. when an object interrupts the light beam). At this point, variable **count** is incremented by one. The thread then waits until the object is moved away. The above process continues until stopped by the user.

```
#-----
#           CONVEYOR BELT OBJECT COUNTER
#           =====
#
# This is a conveyor belt object counter project. Object move on a
# conveyor belt. A light shines from one side to the objects. On the
# other side an LDR is used. When an object interrupts the light
# source the LDR output goes to logic 1 which is detected by the
# Raspberry Pi and the count is incremented and displayed on a 4
# digit 7-segment LED..
# The connections between the 7-segment LED and the Raspberry Pi are:
#
# 7-Segment LED   GPIO
#   a             2
#   b             3
#   c             4
#   d            17
#   e            27
#   f            22
#   g            10
#   E1 module1    11
#   E2 module1    9
#   E1 module2    21
#   E2 module2    20
#
# Author: Dogan Ibrahim
# File  : ldr.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO
import _thread
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM

LED_Segments = (2, 3, 4, 17, 27, 22, 10)
LED_Digits = (11, 9, 21, 20)
```

```

#
# Configure the segments and digits as outputs and clear them
#
for seg in LED_Segments:
    GPIO.setup(seg, GPIO.OUT)          # Segments are outputs
    GPIO.output(seg, 0)                 # Clear all segments

for dig in LED_Digits:
    GPIO.setup(dig, GPIO.OUT)          # Digits are outputs
    GPIO.output(dig, 0)                 # Clear all digits

LED_Bits = {
    ' ': (0,0,0,0,0,0,0),              # Blank
    '0': (1,1,1,1,1,1,0),              # 0
    '1': (0,1,1,0,0,0,0),              # 1
    '2': (1,1,0,1,1,0,1),              # 2
    '3': (1,1,1,1,0,0,1),              # 3
    '4': (0,1,1,0,0,1,1),              # 4
    '5': (1,0,1,1,0,1,1),              # 5
    '6': (1,0,1,1,1,1,1),              # 6
    '7': (1,1,1,0,0,0,0),              # 7
    '8': (1,1,1,1,1,1,1),              # 8
    '9': (1,1,1,1,0,1,1)}              # 9

ldr = 16                               # LDR port
GPIO.setup(ldr, GPIO.IN)               # LDR is input
count = 0                              # Initialzie count

#
# Thread to refresh the 7-segment LED
#
def Refresh():                          # Thread Refresh
    global count
    while True:                        # Do forever
        cnt = str(count)               # into string
        if len(cnt) == 3:              # If 3 digits
            cnt = " " + cnt            # Add " "
        elif len(cnt) == 2:            # If 2 digits
            cnt = "  " + cnt           # Add "  "
        elif len(cnt) == 1:            # If 1 digit
            cnt = "   " + cnt          # Add "   "
        for dig in range(4):           # Do for 4 digits
            for loop in range(0,7):    # Do for all segments
                GPIO.output(LED_Segments[loop], LED_Bits[cnt[dig]][loop])
            GPIO.output(LED_Digits[dig], 1)
        time.sleep(0.0005)

```

```
GPIO.output(LED_Digits[dig], 0)

#
# Thread to count up every second. Objects are counted on the rising
# edge of the pulse
#
def Object_Count():                                # Object LDR_Count
    global count
    while True:                                    # Do forever
        while GPIO.input(ldr) == 0:                # Wait for an object
            time.sleep(0.2)
        count = count + 1                          # Increment count
        while GPIO.input(ldr) == 1:                # Wait until object moved
            time.sleep(0.2)

#
# Create the threads
#
_thread.start_new_thread(Refresh, ())
_thread.start_new_thread(Object_Count, ())

while True:
    pass
```

Figure 6.49 Program: ldr.py

6.4.11 Project 11 – ON/OFF temperature controller with LCD

Description: In this project, we design an ON-OFF temperature control project based on a multitasking approach. An ON-OFF temperature controller, also known as a bang-bang controller, will turn OFF a heater whenever the temperature is above a pre-defined setpoint (i.e. the required value). Similarly, if the temperature is below the setpoint then the heater is turned ON. ON-OFF controllers are used for simplicity since there is no need to know the mathematical model of the plant to be controlled. The only parameter that needs to be set is the setpoint value. ON-OFF control is suitable when the response delay of the plant to be controlled is short and the output rate of rise time is small. If precision temperature control is required, it may be more appropriate to use professional PID (Proportional+Integral+Derivative) type controller algorithms with negative feedback or intelligent fuzzy logic-based controller. The problem with most professional controllers is that an accurate model of the plant to be controlled is normally required before a suitable control algorithm can be derived. Although the ON-OFF type controller is very simple to implement, it has the disadvantage that the relay used to turn the heater ON and OFF has to operate many times and this may shorten the lifetime of the relay.

In this project, it is required to control the temperature of an oven using an ON-OFF type controller. The setpoint temperature is entered by a keyboard on a laptop. This setpoint val-

ue can be changed at any desired time while the oven is under control. i.e. there is no need to stop the controller to change the setpoint value (hence multitasking). A relay connected to the Raspberry Pi turns the heater ON or OFF under the control of software. An LCD shows the setpoint temperature as well as the actual measured oven temperature. The temperature of the oven is measured using an accurate digital temperature sensor chip. A buzzer is used which becomes active if the temperature of the oven goes above a dangerous preset value to indicate an alarm condition. An LED displays the state of the oven at any time such that when the heater is ON then the LED is ON, and vice-versa.

Block Diagram: Figure 6.50 shows the block diagram of the project.

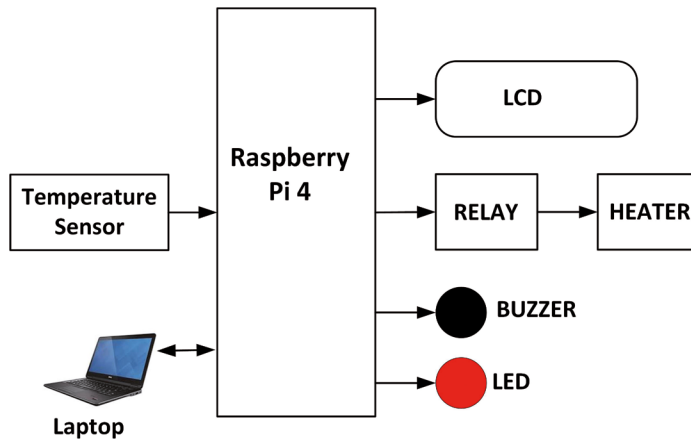


Figure 6.50 Block diagram of the project

Circuit Diagram: In this project, a DS18B20 type temperature sensor chip is used. This chip is available as a module with the name KY-001 (see Figure 6.51). The module has 3 pins: output (S pin), power supply (middle pin), and ground (- pin).

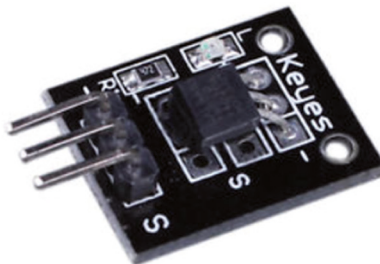


Figure 6.51 KY-001 temperature sensor module

DS18B20 is a digital temperature sensor chip that provides 9-bit to 12-bit temperature measurements and has an alarm function with nonvolatile user-programmable trigger points. The chip communicates over a 1-wire bus. Each DS18B20 has a unique 64-bit serial code, which allows multiple DS18B20s to be connected to the same bus. The basic features of the DS18B20 are:

- 1 wire communication
- Temperature measurement range: -55°C to +125°C
- Accuracy: $\pm 0.5^\circ\text{C}$
- Programmable resolution (9 or 12 bits)
- No external components are required
- Operating voltage: +3V to +5.5V
- Standby current: 750nA
- Active current: 1mA

In this project, an I²C type LCD is used to display the setpoint as well as the actual temperatures. I²C LCDs are standard LCDs but also have a small add-on circuit mounted on the back of the module which features a PCF8574 chip for I²C communication (see Figure 6.52), and a potentiometer to adjust the LED backlight. The I²C LCD has 4 pins: GND, +V, SDA, and SCL. SDA is connected to pin GPIO 2 and SCL is connected to pin GPIO 3. +V pin of the display should be connected to the +5V (pin 2) of the Raspberry Pi 4 (some versions of Raspberry Pi, such as the Raspberry Pi Zero and Raspberry Pi Zero W cannot supply enough power to drive the LCD. It is recommendable to use an external +5V supply for the LCD if you are using one of these models). Note there is no problem mixing the +3.3V GPIO pins of the Raspberry Pi with the +5V of the I²C LCD. This is because the Raspberry Pi is the I²C master device and the SDA and SCL lines are pulled up to +3.3V through resistors. The SCL line is the clock that is always output from the master device. The slave device (I²C LCD here) only pulls down the SDA line when it acknowledges the receipt of data and does not send any data to the master device. Therefore, there are no voltage level problems as long as the Raspberry Pi I²C output pins can drive the I²C LCD inputs, which is the case here.

I²C is a multi-slave, multi-master, single-ended serial bus used to attach low-speed peripheral devices to microcontrollers. The bus consists of only two wires called SDA and SCL where SDA is the data line and SCL the clock line. Up to 1008 slave devices can be supported on the bus. Both lines must be pulled up to the supply voltage by suitable resistors. The clock signal is always generated by the bus master. The devices on the I²C bus can communicate at 100 kHz or 400 kHz.

A small relay module (e.g. KY-019) is connected to the Raspberry Pi which controls the heater. Additionally, as shown in the circuit diagram in Figure 6.53, a buzzer is connected to the Raspberry Pi to indicate any over-temperature alarm conditions. An LED indicates whether the heater is ON or OFF.

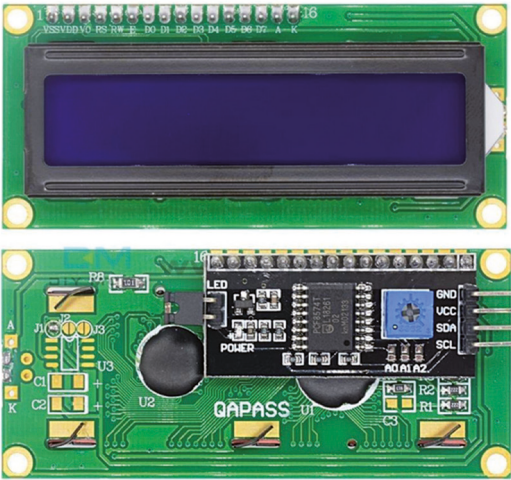
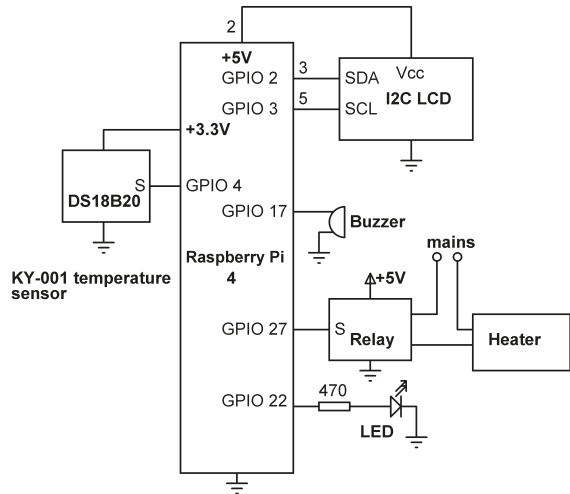


Figure 6.52 I²C LCD



Warning: Risk of electric shock. Do not connect the circuit to mains supply without seeking professional advice

Figure 6.53 Circuit diagram of the project

The connections between the Raspberry Pi and the peripheral devices are as follows:

Raspberry Pi	Peripheral device
GPIO 2	SDA (I ² C LCD)
GPIO 3	SCL (I ² C LCD)
GPIO 4	S (KY-001)
GPIO 17	Buzzer

GPIO 27	S (Relay)
GPIO 22	LED
+5V	Power (I ² C, Relay)
+3.3V	KY-001 power
GND	I ² C LCD, KY-001, Buzzer, Relay, LED

Figure 6.54 shows a simplified block diagram of our oven control system.

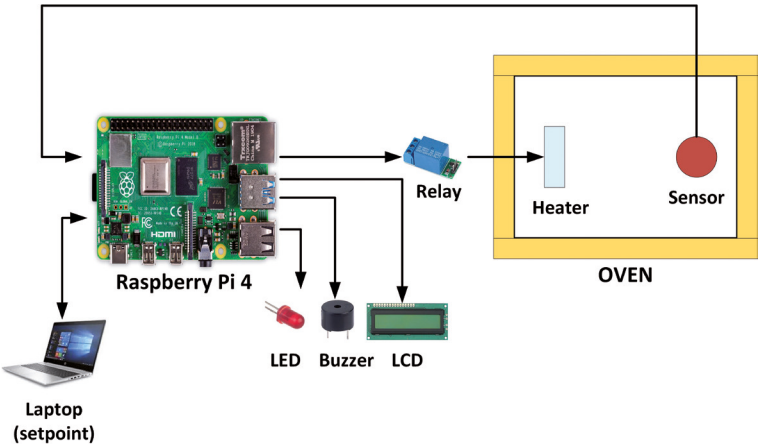


Figure 6.54 Oven control system

Program Listing: Before writing the program we have to install the required library modules to our Raspberry Pi for the I²C LCD and the DS18B20 temperature sensor chip. These are described below:

Installing the I²C LCD library

- Start the configuration menu from the command prompt:

```
pi@raspberrypi:~ $ sudo raspi-config
```

- Go down the menu to **Interface Options**
- Go down and select **I2C**
- Enable the I²C interface
- Select **Finish** to complete

Now we have to install the I²C library on our Raspberry Pi 4. The steps are as follows:

- Enter the following commands from the command menu:

```
pi@raspberrypi:~ $ sudo apt-get update
pi@raspberrypi:~ $ sudo apt-get install -y python-smbus i2c-tools
pi@raspberrypi:~ $ sudo reboot
```

- Enter the following command to test the installation. You should see **i2c_bcm2837** listed:

```
pi@raspberrypi:~ $ lsmod | grep i2c
```

- Modify the config file as follows to add **i2c_bcm2837**:

```
pi@raspberrypi:~ $ sudo nano /etc/modules
Add the following lines (if they are not already there):
i2c-bcm2837
i2c-dev
```

- Exit **nano** by typing Ctrl X and Y to save the file. You can check the contents of this file by entering the command:

```
pi@raspberrypi:~ $ cat /etc/modules
```

- Reboot the Raspberry Pi by entering:

```
pi@raspberrypi:~ $ sudo reboot
```

- Connect your I²C LCD to the Raspberry Pi device as shown in Figure 6.53, and enter the following command to check whether or not the LCD is recognized by the Raspberry Pi:

```
pi@raspberrypi:~ $ sudo i2cdetect -y 1
```

You should see a table similar to the one shown in Figure 6.55. A number in the table means the LCD has been recognised correctly and the I²C slave address of the LCD is shown. In this example, the LCD address is 27:

```
pi@raspberrypi:~ $ sudo i2cdetect -y 1
 0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- 27 -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
pi@raspberrypi:~ $
```

Figure 6.55 I²C address of the PCD is 27

We should now install an I²C LCD library so we can send commands and data to our LCD. There are many Python libraries available for I²C type LCDs. The one preferred here is called the **RPi_I2C_driver**. This library is installed as follows:

- Go to the following web link:

<https://gist.github.com/DenisFromHR/cc863375a6e19dce359d>

- Scroll down to section **RPi_I2C_driver.py**. Click **Raw** on the top right-hand side of the screen and save the file in a folder (e.g. Desktop) with the name **RPi_I2C_driver.py** (the easiest option might be to copy the file into the Notebook and then save it as **RPi_I2C_driver.py**).
- Start your Raspberry Pi in command mode.
- Start the WinSCP file copy utility (you should install it if you don't already have it) on your PC and copy file **RPi_I2C_driver.py** to folder **usr/lib/python3** on your Raspberry Pi.

```
pi@raspberrypi@~ $ sudo cp RPi_I2C_driver.py /usr/lib/python3
```

- Check to make sure that the file is copied successfully. You should see the file listed with the command:

```
pi@raspberrypi:~ $ ls /usr/lib/python3  
pi@raspberrypi:~ $ dist-packages RPi_I2C_driver.py
```

The I²C LCD library supports the following functions (see the I²C LCD library documentation for more details):

<code>lcd_clear()</code>	clear LCD and set to home position
<code>lcd_display_string(text, row)</code>	display text at LCD row
<code>lcd_write_char(c)</code>	display character
<code>lcd_write(cmd)</code>	write command cmd to LCD
<code>lcd.backlight(1/0)</code>	enable/disable LCD backlight
<code>lcd_display_string_pos(text,row,col)</code>	display text at given row,column

Installing the DS18B20 library

The steps are:

- Install the w1thermsensor library

```
pi@raspberrypi:~ $ sudo pip3 install w1thermsensor
```

- Enable the 1 wire interface:

```
pi@raspberrypi:~ $ sudo raspi-config
```

- select **Interfacing Options**
- select **1-Wire** and enable it
- Reboot your Raspberry Pi

We are now ready to write our program.

Program Listing: Figure 6.56 shows the program listing (program: **onoff.py**). The LCD library and the 1 wire library are imported at the beginning of the program in addition to the other libraries used in the program. **Buzzer**, **LED**, and **Relay** are set to 17, 22, and 27 respectively which correspond to the GPIO port pin numbers. These ports are then configured as outputs. The program consists of two threads: **DS18B20** and **Set_Temperature**. Thread **Set_Temperature** prompts the user to enter the required setpoint temperature and this is stored in a variable called **setpoint**. Thread **DS18B20** reads the ambient temperature from the DS18B20 chip every second and stores it in the **temperature** variable. If **temperature** is greater than the **setpoint** value, then the heater is turned OFF and at the same time, the LED is also turned OFF to indicate the heater is OFF. Otherwise (if the **temperature** is less than the **setpoint** value), both heater and LED are turned ON. The temperature alarm value is set to 30°C at the beginning of the program. If the temperature reaches this value, the buzzer is activated to indicate the alarm condition, otherwise, the buzzer is deactivated.

When the program is started, the message **ON-OFF CONTROLLER** is displayed on the LCD for 2 seconds. After this time the setpoint temperature value and measured temperature are displayed on the LCD every second.

```
#-----
#                               ON-OFF TEMPERATURE CONTROLLER
#                               =====
#
# This is an on-off temperatrue control project which controls
# the temperature of an oven (or room). A digital temperature
# sensor measures the temperature. If this temperatrue is lower
# then the required setpoint temperature then a heater is activated
# and at the same time an LED is turned ON to indicate that the
# heater is ON. If the temperature is above a preset alarm value
# then a buzzer is activated
#
# Author: Dogan Ibrahim
# File  : onoff.py
# Date  : May 2020
```

```
#-----
import RPi.GPIO as GPIO
import _thread
import time
from w1thermsensor import W1ThermSensor      # 1 wire library
import RPi_I2C_driver                        # I2C library
LCD = RPi_I2C_driver.lcd()
sensor = W1ThermSensor()
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                      # GPIO mode BCM

Buzzer = 17                                # Buzzer at GPIO 17
LED = 22                                    # LED at GPIO 22
Relay = 27                                  # Relay at GPIO 27

GPIO.setup(Buzzer, GPIO.OUT)                # Buzzer is output
GPIO.setup(LED, GPIO.OUT)                   # LED is output
GPIO.setup(Relay, GPIO.OUT)                 # Relay is output

setpoint = 20                              # Initialize setpoint
temperature = 20                            # initialize temperature
alarm = 30                                  # Alarm temperature

#
# Thread to read the temperature
#
def DS18B20():                              # Thread DS18B20
    global setpoint
    global temperature
    while True:                             # Do forever
        temperature = str(sensor.get_temperature())[0:4]
        if float(temperature) > float(setpoint):
            GPIO.output(Relay, 0)
            GPIO.output(LED, 0)
        else:
            GPIO.output(Relay, 1)             # Relay ON
            GPIO.output(LED, 1)              # LED ON

        if float(temperature) > alarm:        # If > alarm value
            GPIO.output(Buzzer, 1)          # Buzzer ON
        else:
            GPIO.output(Buzzer, 0)          # Buzzer OFF
        time.sleep(1)                        # 1 second delay

#
# Thread to set the required temperature
```

```

#
def Set_Temperature():                                # Thread Set_temperature
    global setpoint
    print("ON-OFF TEMPERATURE CONTROLLER")           # Heading
    print("=====")
    while True:                                       # Do forever
        setpoint = str(input("Enter the setpoint temperature: "))

#
# Create the threads
#
_thread.start_new_thread(DS18B20, ())
_thread.start_new_thread(Set_Temperature, ())

#
# Display the setpoint and the measured temperatures
#
LCD.lcd_clear()                                     # Clear LCD
LCD.lcd_display_string("ON-OFF CONTROL", 1)           # Heading
time.sleep(2)                                       # Wait 2 secs
LCD.lcd_clear()                                     # Clear LCD

while True:                                         # DO forever
    firstline = "Setpoint = " + str(setpoint)
    secondline = "Measured = " + str(temperature)
    LCD.lcd_display_string(firstline,1)
    LCD.lcd_display_string(secondline, 2)
    time.sleep(1)

```

Figure 6.56 Program: onoff.py

Figure 6.57 shows an example display on the LCD.



Figure 6.57 Example display on the LCD

6.5 Summary

In this chapter, we learned how to develop multitasking projects using threads. In the next chapter, we will look at how the threading library can be used to create multitasking projects.

Chapter 7 • Raspberry Pi multitasking projects - using threading

7.1 Overview

In the last chapter, we looked at how to create projects using threads in Python. In this chapter, we will be using **threading** for creating multitasking projects.

Python supports two types of thread modules: the **thread** as described in Chapter 6, and **threading** which will be described in this chapter. Threading is a higher-level interface and uses the standard thread module internally. Threading has been supported since Python 2.4 and provides higher-level support.

7.2 Threading

Threading uses all methods of the thread module and provides support for the following additional methods:

- threading.activeCount()**: Returns the number of thread objects that are active
- threading.currentThread()**: Returns the number of thread objects in the caller's thread
- threading.enumerate()**: Returns a list of all thread objects that are currently active

The threading module also provides the following options:

- run()**: entry point of a thread
- start()**: starts a thread by calling the run method
- join([time])**: waits for threads to terminate
- isAlive()**: checks whether a thread is still executing
- getName()**: returns the name of a thread
- setName()**: sets the name of a thread
- is_alive()**: returns whether the thread is alive

An example threading is given below.

Example – single thread

In this example program shown in Figure 7.1 (program: **hellothreading.py**), function **newprocess** is started as a new thread and displays the message **Hello from new process...** The thread is started with the function call **thread.start()**. Function **thread.join()** waits until the thread is complete and then displays the message **Hello from the creator...** The function **thread.join()** blocks indefinitely until all threads exit. We can specify a floating-point timeout value, for example, **thread.join(2)** so that the program exits after 2 seconds even if the threads do not complete within this timeout value.

```
#-----  
#                               THREADING EXAMPLE  
#                               =====  
#  
# This program creates a new thread called newprocess which  
# displays message Hello from new process... When the thread  
# terminates the message Hello from the creator... is displayed  
#  
# Author: Dogan Ibrahim  
# File  : hellothreading.py  
# Date  : May 2020  
#-----  
import threading  
  
def newprocess():  
    print("Hello from new process...")  
  
thread = threading.Thread(target = newprocess, args=())  
thread.start()  
thread.join()  
print("Hello from the creator...")
```

Figure 7.1 Program: hellothreading.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 hellothreading.py  
Hello from new process...  
Hello from the creator
```

We can give a name to a thread. For example, let us name the thread created by function **newprocess** in Figure 7.1 **TASK1** and then display the name of this thread as shown in the program in Figure 7.2 (program: **namethread.py**).

```
#-----  
#                               THREADING EXAMPLE  
#                               =====  
#  
# This program shows how a thread can be named  
#  
# Author: Dogan Ibrahim  
# File  : namethread.py  
# Date  : May 2020  
#-----  
import threading
```

```
def newprocess():
    thread.setName("Task1")
    name = thread.getName()
    print(name)
    print("Hello from new process...")

thread = threading.Thread(target = newprocess, args=())
thread.start()
thread.join()
print("Hello fro the creator...")
```

Figure 7.2 Program: namethread.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 namethread.py
Task1
Hello from new process...
Hello from the creator
```

Example – multiple threads

In the example program shown in Figure 7.3 (program: **multiple.py**) multiple threads are created and data is passed to each thread. In this example, 5 threads are created in a loop and the loop count is passed as an argument to the created threads

```
#-----
#                               MULTIPLE THREAD EXAMPLE
#                               =====
#
# This program shows how multiple threads can be created
#
# Author: Dogan Ibrahim
# File  : multiple.py
# Date  : May 2020
#-----

import threading

def newprocess(j):
    print("Hello from new process %d" %j)

for i in range(5):
    t = threading.Thread(target = newprocess, args=(i, ))
    t.start()
```

Figure 7.3 Program: multiple.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 multiple.py
Hello from new process 0
Hello from new process 1
Hello from new process 2
Hello from new process 3
Hello from new process 4
```

7.2.1 Thread lock objects

Lock objects are synchronization primitives used to synchronize threads. A lock can be in one of two states: **locked** or **unlocked**. It is created in the unlocked state and has two methods: **acquire()** and **release()**. When the state is **locked()**, **acquire()** blocks until a call to **release()** in another thread changes it to unlocked. When the state is unlocked, **acquire()** changes the state to locked and immediately returns. The **release()** method should only be called in the locked state; it changes the state to unlocked and returns immediately. If an attempt is made to release an unlocked lock, an error will be raised. **acquire()** has blocking or non-blocking as the first argument where by default it is blocking. An optional timeout can be specified as its second argument to specify the number of seconds to block.

An example use of the lock objects is given below.

Example

Figure 7.4 shows an example program (program: **locks.py**). In this example, a shared variable called **count** is used. Thread Task1 locks the shared variable **count** and releases it after incrementing by 1. Task2 locks the shared variable **count** and releases it after incrementing by 2.

```
#-----
#                               EXAMPLE THREAD LOCK OBJECTS
#                               =====
#
# This program shows how thread objects can be used
#
# Author: Dogan Ibrahim
# File  : locks.py
# Date  : May 2020
#-----
import threading
lock = threading.Lock()
count = 0

def Task1():
    global count
    lock.acquire()                # Lock count
```

```
count = count + 1          # Increment count
lock.release()             # Release count

def Task2():
    global count
    lock.acquire()          # Lock count
    count = count + 2       # Increment count
    lock.release()          # Release count

t1 = threading.Thread(target = Task1, args=())
t2 = threading.Thread(target = Task2, args=())
t1.start()
t2.start()
t1.join()
t2.join()
print(count)
```

Figure 7.4 program: locks.py

7.2.2 Semaphores

A semaphore is an advanced lock mechanism. It is used to limit access to a resource with limited capacity. A semaphore has an internal counter which is decremented when the semaphore is acquired and incremented when the semaphore is released. If the counter reaches zero when acquired, the acquiring thread will block. When the semaphore is incremented again, one of the blocking threads will run.

The semaphore counter must be initialized before used. If not initialized, a count of 1 is assumed by default. For example, to set the counter to 5, use the statements:

```
max = 5
semaphore = threading.BoundedSemaphore(max)
```

An example use of a semaphore is given below:

Example

Figure 7.5 shows an example program (program: **sema.py**). In this example, the semaphore count is set to 1 and after acquiring the semaphore numbers 0 to 4 are displayed by Task1. After the semaphore is released, numbers 0 to 4 are displayed by Task2.

```
#-----
#                               EXAMPLE SEMAPHORE
#                               =====
#
# This program shows how a semaphore can be used
#
```

```
# Author: Dogan Ibrahim
# File : sema.py
# Date : May 2020
#-----
import threading
sema = threading.BoundedSemaphore(1)
count = 0

def Task1():
    sema.acquire()                # acquire semaphore
    for i in range(4):
        print(i)
    sema.release()                # release semaphore

def Task2():
    sema.acquire()                # acquire semaphore
    for i in range(4):
        print(i)
    sema.release()                # release semaphore

t1 = threading.Thread(target = Task1, args=())
t2 = threading.Thread(target = Task2, args=())
t1.start()
t2.start()
t1.join()
t2.join()
```

Figure 7.5 Program: sema.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 sema.py
0
1
2
3
0
1
2
3
```

7.2.3 Events

Events are simple synchronization objects using internal flags. Threads can wait for the flag to be set, or set or clear the flags.

The following event operations are available:

set(): set the internal flag to true
clear(): reset (clear) the internal flag to false
wait(): block until the internal flag is set to true
is_set(): returns True if the internal flag is set true

An example program is given below

Example

Figure 7.6 shows an example program (program: **event.py**). In this example, thread Task1 waits until the event flag is set. Thread Task2 prompts the user to enter a key. After entering a key, the event flag is set and Task1 displays a message **Event occurred...**

```
#-----
#                               EXAMPLE EVENT
#                               =====
#
# This program shows how an event can be used
#
# Author: Dogan Ibrahim
# File   : event.py
# Date   : May 2020
#-----

import threading
event = threading.Event()

def Task1():
    event.wait()                # Wait for the event
    print("Event occurred...")

def Task2():
    a = input("Enter a key to set the event:")
    event.set()

t1 = threading.Thread(target = Task1, args=())
t2 = threading.Thread(target = Task2, args=())
t1.start()
t2.start()
t1.join()
t2.join()
```

Figure 7.6 Program: event.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 event.py
Enter a key to set the event: a
Event occurred...
```

7.2.4 Timer threading

Python also supports timer threads. A timer thread is a delayed thread that starts after the specified time delay. In the program example shown in Figure 7.7 (program: **tmrthread.py**), **TimerThread** starts working 5 seconds after it is started by the **T.start()** function.

```
#-----
#                               TIMER THREAD EXAMPLE
#                               =====
#
# This program shows how a timer thread can be used
#
# Author: Dogan Ibrahim
# File  : tmrthread.py
# Date  : May 2020
#-----

import threading
import time

def TimerThread():
    print("TimerThread started.")

print("TimerThread will start after 5 seconds.")
T = threading.Timer(5, TimerThread)
T.start()
```

Figure 7.7 Program: tmrthread.py

The following output will be displayed when the program is run:

```
pi@raspberrypi:~ $ python3 tmrthread.py
TimerThread will start after 5 seconds.
TimerThread started.
```

7.3 Threading based projects

Threading is very similar to threads discussed in Chapter 6. **All the projects developed in Chapter 6 can be modified to run with threading.** An example project is given in the next section to show how threading can be used in hardware-based Raspberry Pi projects.

7.3.1 Project 1 – LED flashing counter

Description: In this project, an LED is connected to GPIO 22 of the Raspberry Pi through a 470 Ohm current limiting resistor. The project counts the number of flashes and displays the count on the display. An event flag is used to synchronize the flashing thread with the counting and displaying thread.

Program Listing: Figure 7.8 shows the program listing (program: **flashcount.py**). At the beginning of the program, an event flag is created with the name **event**. The LED is assigned to 22 which corresponds to the GPIO port number where it is connected to. This port is then configured as an output.

There are two threads in the program. Thread **Flash** flashes the LED every second. Just before turning the LED ON, the event flag is set, Thread **Count** waits for the event flag to be set, then clears the event flag, and increments the **count** variable by 1. The total **count** is then displayed on the screen.

```
#-----
#                               EXAMPLE SEMAPHORE
#                               =====
#
# This program shows how a semaphore can be used
#
# Author: Dogan Ibrahim
# File  : flashcount.py
# Date  : May 2020
#-----

import threading
import time
import RPi.GPIO as GPIO
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
event = threading.Event()

LED = 22                                # LED at port GPIO 22
GPIO.setup(LED, GPIO.OUT)               # LED is output

def Flash():                             # Thread Flash
    while True:                          # Do forever
        GPIO.output(LED, 0)              # LED OFF
        time.sleep(1)                    # Wait 1 second
        event.set()                      # Set event flag
        GPIO.output(LED, 1)              # LED ON
        time.sleep(1)                    # Wait 1 second
```

```
def Count():                                # Thread Count
    count = 0                               # Count = 0
    while True:                             # Do forever
        event.wait()                        # Wait for event flag
        event.clear()                      # Clear event flag
        count = count + 1                  # Increment count
        print("Count = %d" % count)        # Display count

t1 = threading.Thread(target = Flash, args=())
t2 = threading.Thread(target = Count, args=())
t1.start()
t2.start()
t1.join()
t2.join()
```

Figure 7.8 Program: flashcount.py

The following output will be displayed when the program is run (only part of the output is shown):

```
pi@raspberrypi:~ $ python3 flashcount.py
Count = 1
Count = 1
Count = 1
Count = 1
Count = 1
```

7.4 Summary

In this chapter, we learned how to develop multitasking projects using the threading module of Python on a Raspberry Pi. In the next chapter, we will use subprocesses to create multitasking applications.

Chapter 8 • Using subprocesses

8.1 Overview

Subprocesses were introduced to replace the following various old modules (functions) present in Python:

- `os.system`
- `os.spawn`
- `os.popen`
- `commands`

Subprocesses are used to run independent programs (or commands). Details of how to use subprocesses are given in the following sections.

8.2 Subprocesses call

This function is used to run a command and get its return code. An example of its use is shown below:

```
>>> import subprocess
>>> print(subprocess.call(["date"]))
Sun 24 May 18:12:00 BST 2020
>>>
```

Parameters can be passed to the command as shown in the following example:

```
>>> import subprocess
>>> print(subprocess.call(["pwd", "-P"]))
>>> /home/pi
>>>
```

8.3 Subprocess run

This function is like the `call` method and it runs a command. Additionally, the return code of the command is also displayed. An example of its use is shown below:

```
>>> import subprocess
>>> print(subprocess.run(["pwd", "-P"]))
>>> /home/pi
CompletedProcess(args='pwd', '-P', returncode=0)
>>>
```

8.4 Subprocess check_call

This function is like the `call` method and it runs a command. Additionally, if there was an error in running the specified command, it raises an exception (`CalledProcessError`). An example of its use is shown below:

```
>>> import subprocess
>>> print(subprocess.check_call("false"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.7/subprocess.py", line 347, in check_call
    Subprocess.CalledProcessError: Command 'false' returned non-zero exit status 1.
>>>
```

8.5 Subprocess check_output

The output is bound to the parent process and is not accessible when we use the call function to run a command. The check_output function can be used to capture the output as shown in the following example:

```
>>> import subprocess
>>> print(subprocess.check_output(["pwd", "-P"]))
b'/home/pi\n'
>>>
```

8.6 Subprocess Popen and communicate

Popen is used to execute a child program in a new process. The communicate function is used to read input and output from the process itself, where stdout and stderr are the process output and input respectively. An example is shown below:

```
>>> import subprocess
>>> pr = subprocess.Popen(["pwd", "-P"], stdout = subprocess.PIPE)
>>> stdout, stderr = pr.communicate()
>>> print(stdout)
b'/home/pi\n'
>>>
```

8.7 Running a Python program

An example program is given in this section to show how a Python program can be run using the subprocess module.

In this example, program **hello.py** contains the following line:

```
print("Hello from program")
```

In addition, program **disp.py** has the following lines:

```
import subprocess
subprocess.call(["python3", "hello.py"])
```

If we now run the program **disp.py** we will get:

```
pi@raspberrypi:~ $ python3 disp.py
```

```
Hello from program
pi@raspberrypi:~ $
```

Notice that subprocess calls are blocking, but we can use **Popen** to execute a child process so that the parent process is not blocked. An example project is given below.

8.8 Project 1 – Two LEDs flashing at different rates

Description: This project is the same as Project 1 in Chapter 5.3.1 where two LEDs are connected to the Raspberry Pi. One of the LEDs flashes every second while the other one flashes every 250 milliseconds.

Block Diagram: The block diagram of the project is the same as in Figure 5.1.

Circuit Diagram: The circuit diagram of the project is the same as in Figure 5.2. The LEDs are connected to Raspberry Pi GPIO 2 and GPIO 3 through 470 Ohm current limiting resistors.

Program Listing: Figure 8.1 shows the parent program listing (program: **ledsubprocess.py**). This program activates two child programs called **flash1000.py** and **flash250.py**. Program **flash1000.py** (Figure 8.2) flashes the LED connected to GPIO 2 every second. Similarly, program **flash250.py** (Figure 8.3) flashes the LED connected to GPIO 3 every 250 milliseconds.

```
#-----
#           START PROGRAMS TO FLASH 2 LEDs
#           =====
#
# This program starts programs flash1000.py and flash250.py
# to flash two LEDs at the rates 1 second and 250ms
#
# Author: Dogan Ibrahim
# File  : ledsubprocess.py
# Date  : May 2020
#-----

import subprocess
print("Start flash1000.py")
subprocess.Popen(["python3", "flash1000.py"])
print("Start flash250.py")
subprocess.Popen(["python3", "flash250.py"])
print("Parent exited")
```

Figure 8.1 Program: ledsubprocess.py

```

#-----#
#          FLASH LED EVERY SECOND
#          =====
#
# This program flashes the LED at GPIO 2 every second
#
# Author : Dogan Ibrahim
# File   : flash1000.py
# Date   : May 2020
#-----#
import RPi.GPIO as GPIO          # Import RPi
import time
GPIO.setwarnings(False)         # Disable warnings

LED = 2                          # LED at GPIO 2
GPIO.setmode(GPIO.BCM)          # GPIO mode
GPIO.setup(LED, GPIO.OUT)        # LED is output

while True:                     # Flash the LED
    GPIO.output(LED, 1)          # LED ON
    time.sleep(1)                # Wait 1 second
    GPIO.output(LED, 0)          # LED OFF
    time.sleep(1)                # Wait 1 second

```

Figure 8.2 Program: flash1000.py

```

#-----#
#          FLASH LED EVERY 250ms
#          =====
#
# This program flashes the LED every 250ms
#
# Author : Dogan Ibrahim
# File   : flash250.py
# Date   : May 2020
#-----#
import RPi.GPIO as GPIO          # Import RPi
import time
GPIO.setwarnings(False)         # Disable warnings

LED = 3                          # LED at GPIO 3
GPIO.setmode(GPIO.BCM)          # GPIO mode
GPIO.setup(LED, GPIO.OUT)        # LED is output

while True:                     # Flash the LED
    GPIO.output(LED, 1)          # LED ON

```

```
time.sleep(0.25)                # Wait 250ms
GPIO.output(LED, 0)             # LED OFF
time.sleep(0.25)                # Wait 250ms
```

Figure 8.3 Program: flash250.py

The program is started with the command:

```
pi@raspberrypi:~ $ python3 ledsubprocess.py
```

Notice that the two programs `flash1000.py` and `flash250.py` will run independently of the parent program.

You can terminate the programs with the command:

```
pi@raspberrypi:~ $ killall python3
```

8.9 Summary

In this chapter, we have looked briefly at the subprocesses and learned how they can be used in projects. In the next chapter, we will look in detail at how to use multi programs in Raspberry Pi Python projects.

Chapter 9 • Raspberry Pi multitasking projects - using multiprocessing

9.1 Overview

Multiprocessing (also called parallel processing) is the method of using more than one processor (e.g. CPU) by an application. Multiprocessing is highly suitable for heavyweight tasks such as CPU bound tasks. Python multiprocessing module provides a powerful API to develop multiprogramming applications. Processors such as the Raspberry Pi include several cores in their CPUs and as a result creating multiprocessing applications on such systems are highly efficient.

In a multiprocessing application, all processes are independent of each other and have their share of the overall system resources, such as memory, processing power, etc. Processes in a multiprogramming application can share memory and communicate with each other using functions provided in the multiprocessing API.

9.2 Multiprocessing or threading?

The readers may be confused about whether to use multiprocessing or threading (or threads) in a multitasking application. Some key differences between the two are summarized below to clarify this concept:

- Multiprocessing starts with different processes that are completely independent of each other. Threading, on the other hand, launches threads which are dependent on the parent process
- The processes in a multiprocessing system have their own CPU and memory spaces which are unique to each process. Threading applications, on the other hand, utilise the same CPU and memory present in the parent process
- In a multiprocessing system, if a process fails due to an error or exception, the other processes continue to run. On a thread-based system, however, if a thread fails then all other threads terminate
- Sharing objects (e.g. data) in a thread-based system is very easy because such objects are global to all the threads in the system. Sharing data in a multiprocessing system however is more difficult because special synchronisation and software functions are required to share objects between different processes.
- Threading is best suited to input-output based applications. Multiprocessing, on the other hand, is more suited to CPU intensive applications

9.3 How many CPU cores?

Raspberry Pi is a multi-core CPU based computer. The following Python statements can be used to find out how many cores our CPU has:

```
>>> import multiprocessing
>>> CPUS = multiprocessing.cpu_count()
>>> print("CPU count = %d" % CPUS)
CPU count=4
>>>
```

9.4 Multiprocessing process calls

Python offers a multiprocessing module that can be used to start parallel processes. The function **Process** is used to start functions as processes. The Process call is similar to threads but in a Process call, the function runs in a process and not in a thread. An example program using the Process call is given in Figure 9.1 (program: **multiproc.py**).

```
#-----
#                               SIMPLE MULTIPROCESSING EXAMPLE
#                               =====
#
# This program creates a process which displays a message
#
# Author : Dogan Ibrahim
# File   : multiproc.py
# Date   : May 2020
#-----
from multiprocessing import Process

def NewProcess():
    print("Hello from the new process...")

pr = Process(target = NewProcess, args=())
pr.start()
print("Hello from the creator...")
```

Figure 9.1 Program: multiproc.py

An example run of the program is shown in Figure 9.2.

```
pi@raspberrypi:~ $ python3 multiproc.py
Hello from the creator...
Hello from the new process...
pi@raspberrypi:~ $ █
```

Figure 9.2 Example run of the program

Notice the program in Figure 9.1 could have been written by importing the whole multiprocessing module as shown in the program in Figure 9.3 (program: **multiproc2.py**).

```
#-----  
#                               SIMPLE MULTIPROCESSING EXAMPLE  
#                               =====  
#  
# This program creates a process which displays a message.  
# In this version of the program whole multiprocessing  
# module is imported to the program  
#  
# Author : Dogan Ibrahim  
# File   : multiproc2.py  
# Date   : May 2020  
#-----  
import multiprocessing  
  
def NewProcess():  
    print("Hello from the new process...")  
  
p = multiprocessing.Process(target = NewProcess, args=())  
p.start()  
print("Hello from the creator...")
```

Figure 9.3 Program: multiproc2.py

9.5 Using Events in multiprocessing

As discussed in earlier chapters, events are useful tools to synchronise processes in multiprocessing applications. Events can also be used in forks and thread-based applications as long as they are created at the beginning of a program. A process can be programmed to wait for an event flag. If the event flag is cleared, then the process will block. If on the other hand the event flag is set, then the thread will continue. The basic event calls are **set()**, **wait()**, and **clear()**.

9.6 Conditions in multiprocessing

A condition is similar to an event flag. A condition allows a process to wait and then be signaled by another process based on some condition becoming true. Conditions are usually used in producer-consumer type applications where the consumer waits until an item becomes available by the producer, and then consumes it. Conditions support the following methods:

condition.acquire(): obtain an internal lock. The process is blocked until the lock is available

condition.notify(): wake up one of the processes waiting (if there is one waiting)

condition.notifyAll(): wake up all waiting processes

condition.Release(): release the internal lock

condition.wait(): release the lock and then block until awakened by notify()

9.7 Multiprocessing Queues

The multiprocessing Queue module provides a first-in-first-out (FIFO) type queue structure so that different processes can exchange data. In a queue, data is put from one end and is removed from the other end. For example, a process can put data into a queue and another process can extract and use this data. Queues can contain any type of data, including strings, integer, floating-point numbers, lists, dictionaries, and so on.

Queues must be created before they are used. Function **put(data)** puts data into the queue. Function **get()** gets data from the queue.

9.8 Sharing data in multiprocessing using Value and Array

The multiprocessing module offers shared memory variables called **Value** and **Array**. Data stored in these variables can be made to be common to all processes running in the system.

9.9 Anonymous Pipes in multiprocessing

Anonymous pipes are used to establish inter-process communication and data exchange between processes (also between threads and parent-child tasks created using the fork function). A pipe is like a shared memory buffer where one process puts data from one end and another process gets this data. Pipes are blocking. For example, a call to a pipe to read data will block the calling process until data is available.

9.10 Named Pipes

Named pipes are like files where they are opened by their names and data is written to or read from them as if they are files. A named file is created using the **os.mkfifo()** system call.

9.11 Signals in multiprocessing

Signals are used to trigger handlers in user programs. When a signal occurs the handler can be programmed to be activated automatically. The function **signal(number, handler)** is used to create a handler object. Here, number is the handler number assigned by the programmer. Handler is the function to be activated when the signal occurs. The program can be forced to sleep and wait for the signal to occur by calling to function **signal.pause()**.

9.12 Multiprocessing based projects

In this section, we will develop multitasking projects using the Python multiprocessing module. Some of the projects in this section are similar to the ones developed using forks and threads. The aim is to demonstrate how to use the multiprocessing module.

9.12.1 Project 1 - Two LEDs flashing at different rates

Description: This project is the same as Project 1 in Chapter 5.3.1 where two LEDs are connected to the Raspberry Pi. One of the LEDs flashes every second while the other one flashes every 250 milliseconds.

Block Diagram: The block diagram of the project is the same as in Figure 5.1.

Circuit Diagram: The circuit diagram of the project is the same as in Figure 5.2. The LEDs are connected to Raspberry Pi GPIO 2 and GPIO 3 through 470 Ohm current limiting resistors.

Program Listing: Figure 9.4 shows the program listing (program: **ledmulti.py**). After importing the modules used in the program, LED1 and LED2 are assigned to 2 and 3 which correspond to GPIO 2 and GPIO 3. Process Flash1000 flashes LED1 every second, while process Flash250 flashes LED2 every 250 milliseconds. Notice that processes Flash1000 and Flash250 are given the names Flash1000 and Flash250 respectively when they are created. Process Flash250 displays its name and process ID as soon as it runs. The two processes are started with function calls `start()`.

```
#-----#
#           FLASH 2 LEDs AT DIFEFRENT RATES
#           =====
#
# Two LEDs are connected to the Raspberry Pi. One of the LEDs
# is flashed every second, while the other one every 250m
#
# Author : Dogan Ibrahim
# File   : ledmulti.py
# Date   : May 2020
#-----#

import multiprocessing
import os
import RPi.GPIO as GPIO          # Import RPi
import time
GPIO.setwarnings(False)         # Disable warnings

LED1 = 2                         # LED1 at GPIO 2
LED2 = 3                         # LED2 at GPIO 3
GPIO.setmode(GPIO.BCM)          # GPIO mode
GPIO.setup(LED1, GPIO.OUT)       # LED is output
GPIO.setup(LED2, GPIO.OUT)

def Flash1000():                 # Process Flash100
    while True:                 # Flash the LED
        GPIO.output(LED1, 1)    # LED ON
        time.sleep(1)           # Wait 1 second
        GPIO.output(LED1, 0)    # LED OFF
        time.sleep(1)           # Wait 1 second

def Flash250():                  # Process Flash250
    myname = multiprocessing.current_process().name
    print("Process name=%s" % myname)
    myid = os.getpid()
```

```
print("Process id=%d" %myid)
while True:
    GPIO.output(LED2, 1)          # LED ON
    time.sleep(0.25)              # Wait 250ms
    GPIO.output(LED2, 0)          # LED OFF
    time.sleep(0.25)              # Wait 250ms

p = multiprocessing.Process(name="Flash1000", target = Flash1000, args = ())
q = multiprocessing.Process(name="Flash250", target = Flash250, args = ())
p.start()
q.start()
```

Figure 9.4 Program: ledmulti.py

Figure 9.5 shows an example run of the program.

```
pi@raspberrypi:~ $ python3 ledmulti.py
Process name=Flash250
Process id=11590
_
```

Figure 9.5 Example run of the program

9.12.2 Project 2 – Setting the LED flashing rate from the keyboard

Description: This project is the same as Project 3 in section 5.3.3 where an LED is connected to the Raspberry Pi and its flashing rate is set from the keyboard while the LED is flashing.

Circuit Diagram: The circuit diagram of the project is similar to Figure 5.2, but here only one LED is used and it is connected to Raspberry Pi port GPIO 2.

Program Listing: Figure 9.6 shows the program listing (program: **ledkey.py**). At the beginning of the program, a Queue with the name **q** is created. The LED is assigned to GPIO 2 and this port is configured as an output. The flashing rate is set to 1 second by sending 1 to the queue using function **put(1)**. Process **Flash** checks whether the queue is empty and if not the new flashing rate is read from the queue using function **get()**. The LED then flashes at this rate. The main program creates process **Flash** and starts it. The **input** function is then used to read the required flashing rate from the keyboard which is then sent to the queue.

```
#-----
#           CHANGE LED FLASHING RATE FROM THE KEYBOARD
#           =====
#
# In this program an LED is connected to the Raspberry Pi and
# its flashing rate is changed from the keyboard
#
# Author : Dogan Ibrahim
# File   : ledkey.py
```

```
# Date   : May 2020
#-----#

import multiprocessing
q = multiprocessing.Queue()          # Create queue
import RPi.GPIO as GPIO              # Import RPi
import time
GPIO.setwarnings(False)              # Disable warnings

LED = 2                               # LED1 at GPIO 2
GPIO.setmode(GPIO.BCM)               # GPIO mode
GPIO.setup(LED, GPIO.OUT)             # LED is output
q.put(1)                             # Default rate

def Flash():                          # Process Flash
    while True:                      # Flash the LED
        if not q.empty():            # If queue not empty
            rate = q.get()           # Get flashing rate
            GPIO.output(LED, 1)       # LED ON
            time.sleep(rate)          # Wait 1 second
            GPIO.output(LED, 0)       # LED OFF
            time.sleep(rate)          # Wait 1 second

#
# Start process Flash
#
p = multiprocessing.Process(target = Flash, args = ())
p.start()

#
# Input the flashing rate
#
while True:
    Flash_Rate = float(input("Enter flashing rate: "))
    q.put(Flash_Rate)
```

Figure 9.6 Program: ledkey.py

9.12.3 Project 3 - ON/OFF temperature controller

Description: This is the same as Project 11 given in section 6.4.11, but designed using processes instead of threads.

As in Project 11, in this project, it is required to control the temperature of an oven using ON-OFF type controller. The setpoint temperature is entered through the keyboard of a laptop. This setpoint value can be changed at any desired time while the oven is under control.

i.e. there is no need to stop the controller to change the setpoint value (hence multitasking). A relay connected to the Raspberry Pi turns the heater ON or OFF under the control of software. An LCD displays the setpoint temperature as well as the actual measured oven temperature. The temperature of the oven is measured using an accurate digital temperature sensor chip. A buzzer is used which becomes active if the temperature of the oven goes above a dangerous preset value to indicate an alarm condition. An LED displays the state of the oven at any time such that when the heater is ON then the LED is ON, and vice-versa.

Block Diagram: The block diagram of the project is as in Figure 6.50.

Circuit Diagram: The circuit diagram of the project is as in Figure 6.53, where the temperature is read using a DS18B20 type temperature sensor chip.

Program Listing: Figure 9.7 shows the program listing (program: **onoffproc.py**). The RPi, multiprocessing, time, one wire, and the I²C LCD libraries are imported at the beginning of the program. Two queues are then created: one for the temperature (called **qtemperature**) and one for the setpoint (called **qsetpoint**). The Buzzer, LED and Relay are assigned to 17,22, and 27 respectively to correspond to the GPIO numbers. These ports are then configured as outputs. The setpoint, temperature, and alarm value are set to 20, 20, and 30 respectively at the beginning of the program.

The program consists of two processes called: **DS18B20** and **Display_Temperature**. Process **DS18B20** reads the measured temperature from the DS18B20 chip and stores in a local variable called **temperature**. This value is also put into the temperature queue **qtemperature**.

Process **Display_Temperature** clears the display and displays heading **ON-OFF CONTROL** in the first row of the LCD. After 2 seconds the display is cleared. The setpoint and the measured temperature values are read from the corresponding queues and stored in local variables called **temperature** and **setpoint** respectively. The process then compares the measured value with the setpoint value and if the measured value is greater than the setpoint value, the Relay and the LED are both turned OFF. If on the other hand, the measured value is less than the setpoint value then both the Relay and the LED are turned ON. The process also compares the measured temperature with the alarm value and if the measured value is greater than the preset alarm value then the Buzzer is activated.

The main program displays the message **Enter the setpoint temperature:** and prompts the user to enter the required setpoint value. The value entered is read and put into queue **qsetpoint**.

```
#-----
#           ON-OFF TEMPERATURE CONTROLLER
#           =====
#
# This is an on-off temperature control project which controls
# the temperature of an oven (or room). A digital temperature
# sensor measures the temperature. If this temperature is lower
# then the required setpoint temperature then a heater is activated
# and at the same time an LED is turned ON to indicate that the
# heater is ON. If the temperature is above a preset alarm value
# then a buzzer is activated
#
# Author: Dogan Ibrahim
# File  : onoffproc.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO          # RPi library
import multiprocessing          # multiprocessing
import time                     # time
from w1thermsensor import W1ThermSensor # 1 wire library
import RPi_I2C_driver          # I2C library
LCD = RPi_I2C_driver.lcd()
sensor = W1ThermSensor()
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)          # GPIO mode BCM

qsetpoint = multiprocessing.Queue() # Create queue
qtemperature = multiprocessing.Queue() # Create queue

Buzzer = 17                     # Buzzer at GPIO 17
LED = 22                        # LED at GPIO 22
Relay = 27                      # Relay at GPIO 27

GPIO.setup(Buzzer, GPIO.OUT)     # Buzzer is output
GPIO.setup(LED, GPIO.OUT)        # LED is output
GPIO.setup(Relay, GPIO.OUT)      # Relay is output

qsetpoint.put(20)                # Initialize setpoint
qtemperature.put(20)             # initialize temperature
alarm = 30                       # Alarm temperature

#
# Process to read the temperature and send to temperature queue
#
def DS18B20():                  # Process DS18B20
    while True:                 # Do forever
```

```
    temperature = str(sensor.get_temperature())[0:4]
    if qtemperature.empty():
        qtemperature.put(temperature)           # Send to queue
    time.sleep(1)                                # 1 second delay

#
# Process to display the setpoint and measured temperatures and to
# control the Relay, LED and Buzzer
#
def Display_Temperature():                      # Process Display_Temperature
    LCD.lcd_clear()                            # Clear LCD
    LCD.lcd_display_string("ON-OFF CONTROL", 1)
    time.sleep(2)                              # Wait 2 seconds
    LCD.lcd_clear()                            # Clear LCD

while True:                                    # Do forever
    if not qtemperature.empty():
        temperature = qtemperature.get()        # Get temperature
    if not qsetpoint.empty():
        setpoint = qsetpoint.get()              # Get setpoint
    firstline = "Setpoint = " + str(setpoint)
    secondline = "Measured = " + str(temperature)
    LCD.lcd_display_string(firstline, 1)
    LCD.lcd_display_string(secondline, 2)

#
# Compare the setpoint value with the measured value and control the
# Relay, LED and the Buzzer accordingly
#
    if float(temperature) > float(setpoint):
        GPIO.output(Relay, 0)                  # Relay OFF
        GPIO.output(LED, 0)                    # LED OFF
    else:
        GPIO.output(Relay, 1)                  # Relay ON
        GPIO.output(LED, 1)                    # LED ON

    if float(temperature) > alarm:
        GPIO.output(Buzzer, 1)                 # Alarm ON
    else:
        GPIO.output(Buzzer, 0)                 # Alarm OFF
    time.sleep(1)                              # Wait 1 second

#
# Create the processes
#
p = multiprocessing.Process(target = DS18B20, args = ())
q = multiprocessing.Process(target = Display_Temperature, args = ())
```

```

p.start()
q.start()

#
# Read the setpoint value from the keyboard
#
print("ON-OFF TEMPERATURE CONTROLLER")
print("=====")
while True:
    setpoint = str(input("Enter the setpoint temperature:" ))
    qsetpoint.put(setpoint)

```

Figure 9.7 Program: *onoffproc.py*

The text displayed on the screen is the same as in Figure 6.57.

9.12.4 Project 4 - Metronome

Description: A metronome is a device that produces an audible sound at regular intervals that can be set by the user. Metronomes are used by musicians to improve their timings when they play instruments. Early metronomes used to be mechanical devices but nowadays metronomes are usually electronic devices. The timings of metronomes are set in beats per minute (bpm).

This project is about the design of a metronome using the Raspberry Pi in a multitasking environment. The default timing of the metronome is set to 120 bpm. Two buttons, named UP and DOWN are used to change the bpm. Pressing UP increments the bpm by 10. Similarly, pressing DOWN decrements the bpm by 10. The bpm setting is displayed on an I²C LCD. An active buzzer is used to generate pulsing sounds to indicate the timings (you may like to use an amplifier to increase the sound level of the buzzer or use a small speaker with an audio amplifier).

Block Diagram: The block diagram of the project is shown in Figure 9.8.

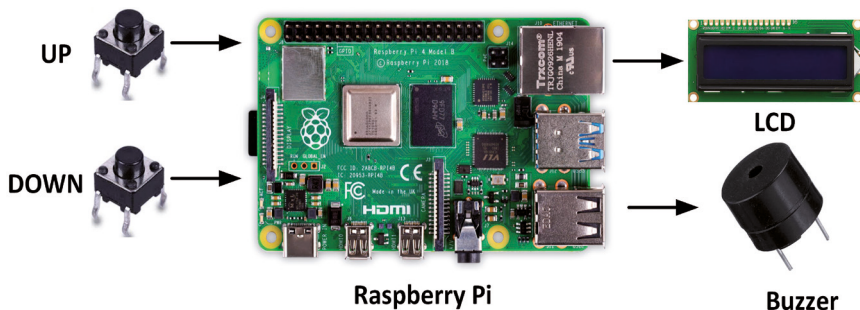


Figure 9.8 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 9.9. Buttons UP and DOWN are connected to GPIO 22 and GPIO 27 respectively. The active buzzer is connected to GPIO 17. The SDA and SCL pins of the I²C LCD are connected to GPIO 2 and GPIO 3 respectively.

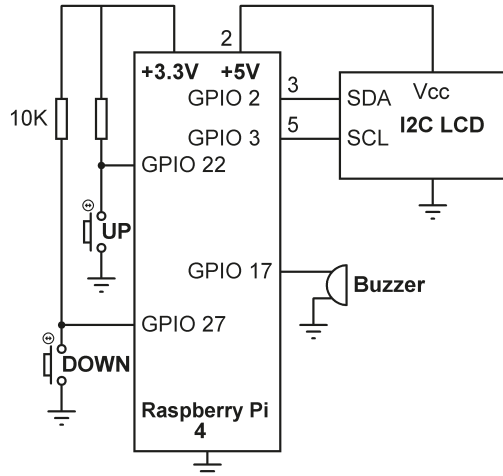


Figure 9.9 Circuit diagram of the project

Program Listing: Figure 9.10 shows the program listing (program: **metronome.py**). At the beginning of the program modules RPi, multiprocessing, time, and I²C LCD library modules are included in the program. Two queues with the names **q1** and **q2** are created. **Buzzer**, **UP** and **DOWN** are assigned to 17, 22 and 27 to correspond to the GPIO port names. **Buzzer** is configured as an output, and **UP** and **DOWN** buttons are configured as inputs. There are 3 processes in the program: **UP_Button**, **DOWN_Button**, and **Activate_Buzzer**.

Process **UP_Button** increments the bpm by 10 every time the **UP** button is pressed. The default starting value of the bpm is set to 120. After changing the bpm it is sent to both queues **q1** and **q2**. Queue **q1** is used by processes **UP_Button** and **DOWN_Button** and also by the main program which displays the current value of the selected bpm. Queue **q2** on the other hand is used by process **Activate_Buzzer**. It was necessary to use two queues in this program because processes **UP_Button** and **DOWN_Button** change the bpm and send it to queue **q1**. But this value is taken from the queue by the display program and therefore nothing is left in the queue for the process **Activate_Buzzer** to get. By using another queue for **Activate_Buzzer** the problem has been solved.

Process **DOWN_Button** decrements the bpm by 10 every time the **DOWN** button is pressed. If the bpm becomes less than 10 then it is set to 10. As in process **UP_Button**, both queues **q1** and **q2** are used in this process.

Process **Activate_Buzzer** activates the buzzer so that the metronome pulses can be heard. The pulse width of the metronome pulses is set to a low value. i.e. to 0.0625 second (62.5ms). Therefore, the buzzer ON time is 0.0625 seconds. The buzzer OFF time is chosen such that the required bpm is achieved. The required overall time delay is stored in variable **dly** which is given by:

$$\text{dly} = 60 / \text{bpm}$$

The buzzer OFF time is then set to **dly – 0.0625** seconds. For example, if the bpm is selected as 120, then **dly = 60 / 120 = 0.5** second. The buzzer ON time is then 0.0625 second, and the OFF time is $0.5 - 0.0625 = 0.4375$ second, in total making 0.5 seconds which corresponds to 120 bpm.

The LCD displays the selected bpm and is updated every second in the main part of the program.

```
#-----
#                               METRONOME
#                               =====
#
# This is a metronome project. A buzzer is connected to the Raspberry Pi
# to sound the timing pulses. The timings are set by two buttons and are
# displayed on an I2C LCD. The default bpm is set to 120.
#
# Author: Dogan Ibrahim
# File  : metronome.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO                # RPi library
import multiprocessing                 # multiprocessing
import time                           # time
import RPi_I2C_driver                 # I2C library
LCD = RPi_I2C_driver.lcd()
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM

q1 = multiprocessing.Queue()          # Create queue q1
q2 = multiprocessing.Queue()          # Create queue q2

Buzzer = 17                           # Buzzer at GPIO 17
UP = 22                               # UP at GPIO 22
DOWN = 27                             # DOWN at GPIO 27

GPIO.setup(Buzzer, GPIO.OUT)           # Buzzer is output
GPIO.setup(UP, GPIO.IN)                # UP is input
GPIO.setup(DOWN, GPIO.IN)              # DOWN is input
```

```

bpm = 120                                # Default bpm=120
q1.put(bpm)

#
# Process to increment bpm
#
def UP_Button():                          # Process UP_Button
    bpm = 120
    while True:                          # Do forever
        while GPIO.input(UP) == 1:      # UP not pressed
            pass
        while not q1.empty():            # If not empty
            bpm = q1.get()
            bpm = bpm + 10                # Increment bpm
            q1.put(bpm)                  # Send bpm to queue
            q2.put(bpm)                  #
        while GPIO.input(UP) == 0:      # UP not released
            pass
#
# Process to decrement bpm
#
def DOWN_Button():                       # Process DOWN_Button
    bpm = 120
    while True:                          # Do forever
        while GPIO.input(DOWN) == 1:    # DOWN not pressed
            pass
        while not q1.empty():            # If not empty
            bpm = q1.get()
            bpm = bpm - 10                # Decrement bpm
            if bpm < 10:                  # If < 10
                bpm = 10
            q1.put(bpm)                  # Send bpm to queue
            q2.put(bpm)                  #
        while GPIO.input(DOWN) == 0:    # DOWN not released
            pass

def Activate_Buzzer():                  # Process Activate_Buzzer
    bpm = 120
    while True:                          # Do forever
        while not q2.empty():            #
            bpm = q2.get()
            dly = 60 / bpm                # Delay time
            GPIO.output(Buzzer, 1)        # Buzzer ON
            time.sleep(0.0625)            # Wait 0.625s
            GPIO.output(Buzzer, 0)        # Buzzer OFF
            time.sleep(dly-0.0625)        # Wait

```

```
#
# Create the processes
#
p = multiprocessing.Process(target = UP_Button, args = ())
r = multiprocessing.Process(target = DOWN_Button, args = ())
s = multiprocessing.Process(target = Activate_Buzzer, args = ())
p.start()
r.start()
s.start()

#
# Clear the LCD and display the bpm
#
LCD.lcd_clear()
while True:
    while not q1.empty():
        bpm = q1.get()
        q1.put(bpm)
        firstline = "bpm = " + str(bpm) + " "
    LCD.lcd_display_string(firstline, 1)
    time.sleep(1)
```

Figure 9.10 Program: metronome.py

Figure 9.11 shows an example display on the LCD.



Figure 9.11 Example display on the LCD

9.12.5 Project 5 – Traffic lights controller

Description: In this project, a simple traffic light controller is designed for a junction. The junction is located at the intersection of two roads: East Street and North Street. There are traffic lights at each end of the junction. There are pedestrian buttons located near the traffic lights on North Street. Pressing a pedestrian button turns all lights to red at the end of their cycles. A buzzer is then sounded to indicate that the pedestrians can cross the road safely. Also, an LCD is connected to the system to display whether the pedestrian cycle is running or the traffic cycle is running. Figure 9.12 shows the layout of the equipment at the junction.

In this project, the following fixed times are given to each traffic light duration, and also to the duration of the pedestrian buzzer. For simplicity, both roads of the junction are assumed to have the same timings:

Red time:	19 seconds
Amber time:	2 seconds
Green time:	15 seconds
Amber+Red time:	2 seconds
Pedestrian time:	10 seconds

The total cycle time of the lights in this example project is set to be 38 seconds.

The sequence of traffic lights is assumed to be as follows (different countries may have different sequences):

Red	Green
Amber+Red	Amber
Green	Red
Amber	Amber+Red

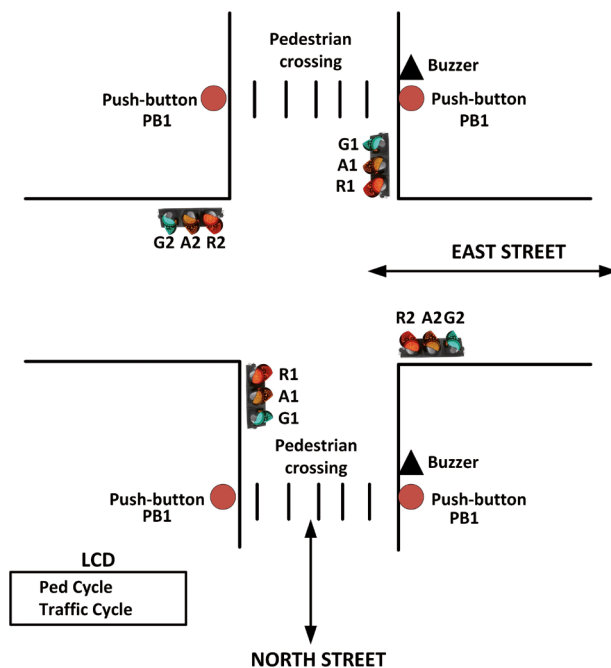


Figure 9.12 Layout of the equipment at the junction

Block Diagram: Figure 9.13 shows the block diagram of the project.

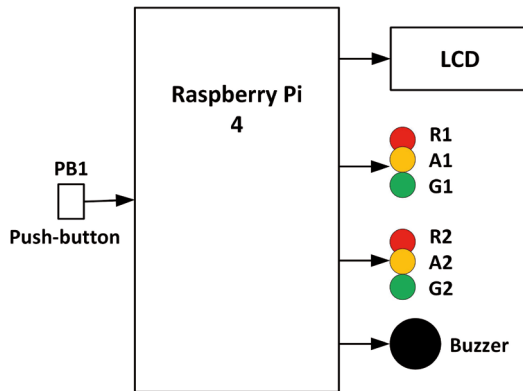


Figure 9.13 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 9.14. Red (R), Amber (A), and Green (G) LEDs are used in this project to represent the real traffic lights. The following connections are made between the Raspberry Pi and road traffic equipment:

Raspberry Pi	Equipment
GPIO 21	LED R1
GPIO 20	LED A1
GPIO 16	LED G1
GPIO 12	LED R2
GPIO 7	LED A2
GPIO 8	LED G2
GPIO 25	Buzzer
GPIO 2	LCD SDA
GPIO 3	LCD SCL
GPIO 24	PB1 (push-button switch)

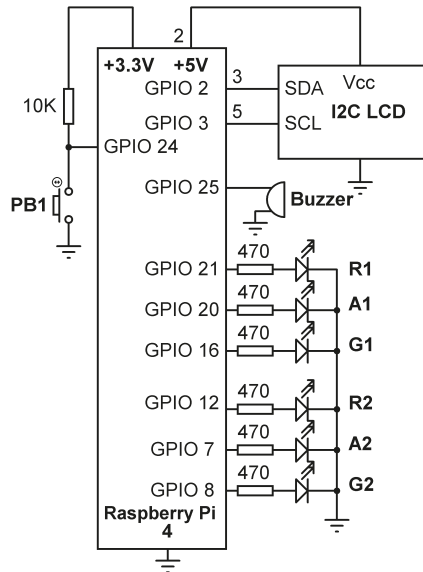


Figure 9.14 Circuit diagram of the project

Program Listing: Figure 9.15 shows the program listing (program: **Traffic.py**). At the beginning of the program modules RPi, time, I²C LCD driver, and multiprocessing are imported to the program and two queues named **pedq** and **lcdq** are created.

Two functions are defined in the program with the names **ONOF** and **CONF_OUT**. Function **ONOF** has two arguments: **port**, and **state**. This function sends the state (0 or 1) to the specified GPIO port. Function **CONF_OUT** has one parameter called **port**. This function configures the specified GPIO port to output state.

There are two processes in the program: **Lights**, and **Pedestrian**. At the beginning of **Lights** process, the connections between the Raspberry Pi and the LEDs (traffic lights) and the Buzzer are defined and these ports are configured as outputs. Additionally, all these port outputs are cleared to 0 so that all LEDs and Buzzer are set OFF. Additionally, the LEDs are sequenced in the correct order with the correct timings. Towards the end of the function, it is checked as to whether the pedestrian button has been pressed. The pedestrian button is pressed if queue **pedq** is not empty. During the pedestrian cycle, the red lights are turned ON on both streets to stop the traffic flowing and give way to the pedestrians. Also, the Buzzer is activated for 10 seconds during the pedestrian cycle to inform the pedestrians that it is safe to cross the road.

The **Pedestrian** process continuously monitors button PB1. If the button is pressed, then a 1 is sent to queue **pedq** so that process **Lights** can easily detect this action and start the pedestrian cycle.

The main program controls the LCD. When the program is started, the message **TRAFFIC CONTROL** is displayed on the first row of the LCD. The second row of the LCD continuously checks queue **lcdq** and displays either **Ped Cycle** or **Traffic Cycle**.

```
#-----
#                                TRAFFIC LIGHTS CONTROLLER
#                                =====
#
# This is a traffic lights controller project controlling lights
# at a junction. 6 LEDs are used to represent the traffic lights.
# Additionally a button is used for pedestrian crossing, and an
# LCD shows the state of the traffic lights at any time
#
# Author: Dogan Ibrahim
# File  : traffic.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO                # Import RPi
import multiprocessing                 # Import multiprocessing
import time                            # Import time
import RPi_I2C_driver                 # I2C library
LCD = RPi_I2C_driver.lcd()            # Import LCD
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                # GPIO mode BCM
pedq = multiprocessing.Queue()        # Create queue
lcdq = multiprocessing.Queue()        # Creaet queue

#
# This function sends data 'state (0 or 1)' to specified port
#
def ONOF(port, state):
    GPIO.output(port, state)

#
# This function configures the specified port as output
#
def CONF_OUT(port):
    GPIO.setup(port, GPIO.OUT)

#
# Process to control the lights
#
def Lights():
    R1=21; A1=20; G1=16                # Process Lights
    R2=12; A2=7;  G2=8                # LED connections
    Buzzer=25                          # LED conenctions
    Buzzer=25                          # Buzzer connection
```

```

CONF_OUT(R1); CONF_OUT(A1); CONF_OUT(G1)      # Configure
CONF_OUT(R2); CONF_OUT(A2); CONF_OUT(G2)      # Configure
CONF_OUT(Buzzer)                              # Configure
ONOF(R1,0); ONOF(A1,0); ONOF(G1,0); ONOF(R2,0); ONOF(A2,0); ONOF(G2,0)
ONOF(Buzzer, 0)

RedDuration = 15
GreenDuration = 15
AmberDuration = 2

#
# Control the traffic light sequence
#
while True:                                     # Do forever
    ONOF(R1,0); ONOF(A1,0); ONOF(G1,1); ONOF(R2,1); ONOF(A2,0); ONOF(G2,0)
    time.sleep(RedDuration)
    ONOF(G1,0); ONOF(A1,1)
    time.sleep(AmberDuration)
    ONOF(A1,0); ONOF(R1,1); ONOF(A2,1)
    time.sleep(AmberDuration)
    ONOF(A2,0); ONOF(R2,0); ONOF(G2,1)
    time.sleep(GreenDuration)
    ONOF(G2,0); ONOF(A2,1)
    time.sleep(AmberDuration)
    ONOF(A2,0); ONOF(A1,1); ONOF(R2,1)
    time.sleep(AmberDuration)

    while not pedq.empty():                     # If ped request
        lcdq.put(1)
        ONOF(G1,0); ONOF(R1,1); ONOF(A1,0)      # Only RED ON
        ONOF(G2,0); ONOF(R2,1); ONOF(A2,0)      # Only RED ON
        d = pedq.get()                          # Clear ledq
        ONOF(Buzzer, 1)                        # Buzzer ON
        time.sleep(10)                         # Wait 10 secs
        ONOF(Buzzer, 0)                        # Buzzer OFF
        d = lcdq.get()                          # Clear lcdq

def Pedestrian():                              # Process Pedestrian
    PB1 = 24
    GPIO.setup(PB1, GPIO.IN)                  # PB1 is input

    while True:                                # Do forever
        while GPIO.input(PB1) == 1:            # PB1 not pressed
            pass
        pedq.put(1)                            # Send to Ped queue

```

```
while GPIO.input(PB1) == 0:                # PB1 not released
    pass

#
# Create the processes
#
p = multiprocessing.Process(target = Lights, args = ())
q = multiprocessing.Process(target = Pedestrian, args = ())
p.start()
q.start()

#
# LCD Display control. Display 'Ped Cycle' or 'Traffic Cycle'
#
LCD.lcd_clear()                            # Clear LCD
LCD.lcd_display_string("TRAFFIC CONTROL", 1) # Heading

while True:                                # DO forever
    if not lcdq.empty():
        LCD.lcd_display_string("Ped Cycle   ", 2)
    else:
        LCD.lcd_display_string("Traffic Cycle", 2)
    time.sleep(1)
```

Figure 9.15 Program: traffic.py

An example display on the LCD is shown in Figure 9.16.



Figure 9.16 Example display on the LCD

9.12.6 Project 6 – Ultrasonic car parking aid with buzzer

Description: In this project, an ultrasonic sensor module is used to measure the distance to objects while parking a vehicle. The sensor is assumed to be mounted on the rear of the vehicle. This is a multitasking project where the buzzer sounds to indicate the distance to any object behind the vehicle such that the activation rate of the buzzer increases as the vehicle gets closer to an object. i.e. the detection of the distance to the object and the buzzer activation are different processes.

Block Diagram: Figure 9.17 shows the block diagram of the project.

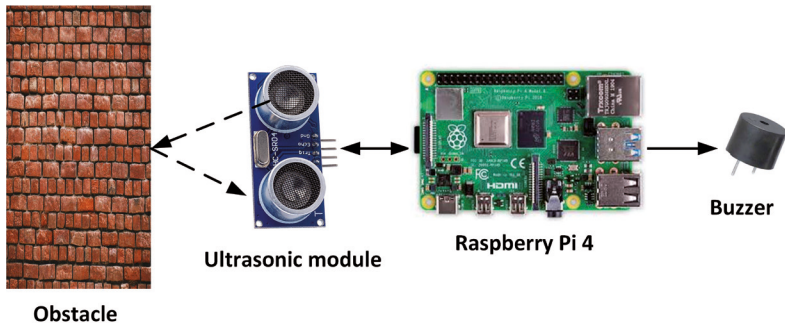


Figure 9.17 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 9.18. In this project, the popular HC-SR04 ultrasonic module is used (see Figure 9.19). This module has the following specifications:

- Operating voltage (current): 5V (2mA) operation
- Detection distance: 2cm – 450cm
- Input trigger signal: 10us TTL
- Sensor angle: not more than 15 degrees

The sensor module has the following pins:

Vcc: +V power
 Trig: Trigger input
 Echo: Echo output
 Gnd: Power ground

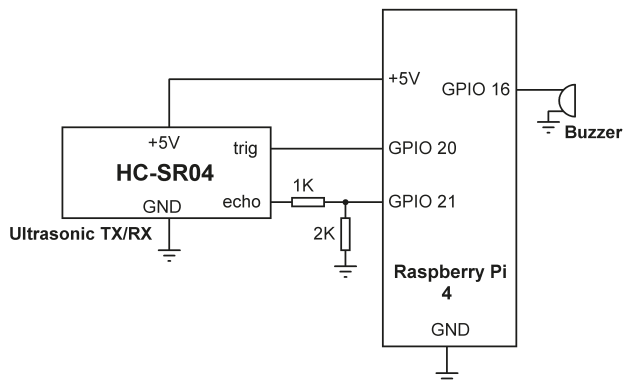


Figure 9.18 Circuit diagram of the project

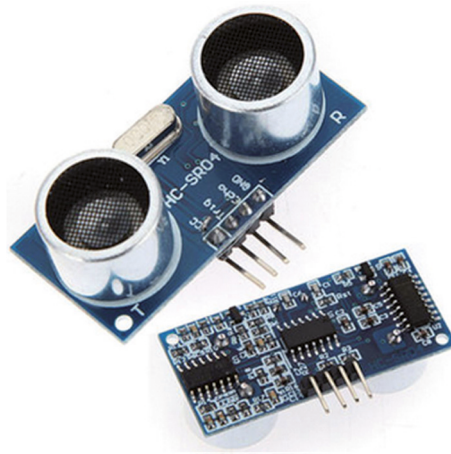


Figure 9.19 Ultrasonic transmitter/receiver module

The principle of operation of the ultrasonic sensor module is as follows:

- A 10us trigger pulse is sent to the module
- The module then sends eight 40kHz square wave signals and automatically detects the returned (echoed) pulse signal
- If an echo signal is returned, the time to receive this signal is recorded

The distance to the object is calculated as:

$$\text{Distance to object (in metres)} = (\text{time to received echo in seconds} * \text{speed of sound}) / 2$$

The speed of sound is 340 m/s, or 34000 cm/s

Therefore,

$$\text{Distance to object (in cm)} = (\text{time to received echo in s}) * 34000 / 2$$

Figure 9.20 shows the principle of operation of the ultrasonic sensor module. For example, if the time to receive the echo is 0.3 milliseconds, the distance to the object is calculated as:

$$\text{Distance to object (cm)} = 0.0003 * 34000 / 2 = 5 \text{ cm}$$

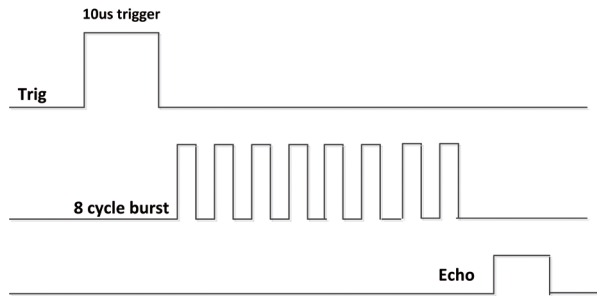


Figure 9.20 Operation of the ultrasonic sensor module

The output of the ultrasonic sensor is +5V and is therefore not compatible with the inputs of the Raspberry Pi. A resistive potential divider circuit is used to lower the voltage to +3.3V. The voltage at the output of the potential divider resistor is:

$$V_o = 5V \times 2K / (2K + 1K) = 3.3V$$

Program Listing: Figure 9.21 shows the program listing (program: **park.py**). At the beginning of the program, various modules are imported to the program. Trigger and echo pins of the ultrasonic module are set to 20 and 21 corresponding to pin numbers GPIO 20 and GPIO 21 respectively. Trigger and echo pins are configured as output and input respectively. Two processes are created in the program with the names **Measure_Distance** and **Activate_Buzzer**.

Process **Measure_Distance** implements the distance measurement algorithm as described earlier in this project. The measured distance is stored in local variable **Distance** and is sent to the queue called **distq**.

Process **Activate_Buzzer** reads the distance to the obstacle from queue **distq**. Then, sounds with different pulsing rates are generated depending on the distance to the obstacle as in the following code, where **d** is the distance to the obstacle:

```
If d < 10:
    BuzzerSound(50)
elif d < 20:
    BuzzerSound(150)
elif d < 30:
    BuzzerSound(250)
elif d < 40:
    BuzzerSound(350)
elif d < 50:
    BuzzerSound(450)
elif d < 70:
    BuzzerSound(700)
elif d < 90:
    BuzzerSound(900)
```

Function **BuzzerSound** has one argument: the ON and OFF times of the Buzzer in milliseconds. For example, if the distance to the obstacle is less than 10cm, the Buzzer ON and OFF times are set 50ms. Similarly, if the distance to the obstacle is less than 20cm, the Buzzer ON and OFF times are set to 150ms and so on.

```
#-----
#           ULTRASONIC CAR PARKING AID WITH BUZZER
#           =====
#
# This is an ultrasonic car parking aid project. It is assumed that an
# ultrasonic module is mounted at the rear (or in-front) of a vehicle.
# The program measures the distance to obstacles and sound a buzzer.
# The pulse rate of the buzzer increases as the vehicle gets closer to
# the object.
#
# Author: Dogan Ibrahim
# File  : park.py
# Date  : May 2020
#-----

import RPi.GPIO as GPIO          # Import RPi
import multiprocessing          # Import multiprocessing
import time                     # Import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)          # GPIO mode BCM
distq = multiprocessing.Queue() # Create queue

trig = 20                        # trig at GPIO 20
echo = 21                       # echo at GPIO 21

GPIO.setup(trig, GPIO.OUT)       # trig is output
GPIO.setup(echo, GPIO.IN)       # echo is input
GPIO.output(trig, 0)
time.sleep(1)                   # Wait to settle

#
# Process Measure_Distance
#
def Measure_Distance():          # Process Measure_Distance
    div = 34300 / 2

    while True:                 # Do forever
        GPIO.output(trig, 1)    # Send trigger
        time.sleep(0.00001)     # 10us delay
        GPIO.output(trig, 0)    # Stop trigger

        while GPIO.input(echo) == 0: # Wait if 0
```

```
        TimeStart = time.time()                # Get start time

        while GPIO.input(echo) == 1:            # Wait if 1
            TimeEnd = time.time()                # Get end time

            TimeDiff = TimeEnd - TimeStart        # Elapsed time in ms
            Distance = TimeDiff * div            # Calculate distance

            if distq.empty():                    # If queue is empty
                distq.put(Distance)              # Send to queue
            time.sleep(0.1)                      # Time between readings

#
# Generate sound at different rates with the buzzer
#
def BuzzerSound(F):
    Buzzer = 16                                # Buzzer at GPIO 16
    ms = F / 1000                              # In milliseconds
    GPIO.setup(Buzzer, GPIO.OUT)               # Buzzer is output
    GPIO.output(Buzzer, 1)                     # Buzzer ON
    time.sleep(ms)                             # Wait ms
    GPIO.output(Buzzer, 0)                     # Buzzer OFF
    time.sleep(ms)                             # Wait ms

def Activate_Buzzer():
    # Process Activate_Buzzer
    d=100
    while True:
        while not distq.empty():                # If distance measured
            d = distq.get()                    # Get distance
            if d < 10:                          # If less than 10cm
                BuzzerSound(50)                # ON/OFF = 50ms
            elif d < 20:                        # If less than 20cm
                BuzzerSound(150)               # ON/OFF = 150ms
            elif d < 30:                        # If less than 30cm
                BuzzerSound(250)               # ON/OFF = 250ms
            elif d < 40:                        # If < 40cm
                BuzzerSound(350)               # ON/OFF = 350ms
            elif d < 50:                        # If < 50cm
                BuzzerSound(450)               # ON/OFF = 450ms
            elif d < 70:                        # If < 70cm
                BuzzerSound(700)               # ON/OFF = 700ms
            elif d < 90:                        # If < 90cm
                BuzzerSound(900)               # ON/OFF = 900ms

#
```

```
# Create the processes
#
p = multiprocessing.Process(target = Measure_Distance, args = ())
q = multiprocessing.Process(target = Activate_Buzzer, args=())
p.start()
q.start()

while True:                                # We never reach here
    pass
```

Figure 9.21 Program: park.py

Notice the size of a Queue can be specified as an argument when the queue is created. Queues have the following methods:

qsize(): returns the size of the queue

empty(): returns True if the queue is empty

full(): returns True if the queue is full

put(obj[,block[,timeout]]): put *obj* into the queue. If option *block* is True (default) and *timeout* is None (default), the queue will block until a free slot is available. If *timeout* is a positive number, the queue will block at most *timeout* seconds and raise the *queue.Full* exception if no free slot was available within that time

put_nowait(obj): put *obj* into queue (same as above when *block* is False)

get([block[,timeout]]): remove and return an item from the queue. If *block* is True (default) and *timeout* is None (default), the queue will block until an item is available. If *timeout* is a positive number, the queue blocks at most *timeout* seconds and raises the *queue.Empty* exception if no item was available within that time.

get_no_wait(): remove and return an item from the queue (same as above when *block* is False)

Additionally, **SimpleQueue** is supported by the following basic methods:

put(item): put item into queue

get(): remove and return item from queue

empty(): return True if the queue is empty

Sometimes we may want to check whether the queue is full or empty, or to detect if an error occurs when we want to put items to a full queue, or to read items from an empty queue. The following exception can be used to check when the queue is full and take the required actions:

```
try:
    q.put(item)                                # Put item into queue
except q.Full:
    # code to take action if the queue is full  # Queue is full
```

or,

```
try:
    d = q.get()                                # Get item from queue
except q.Empty:
    # code to take action if the queue is empty # Queue is empty
```

9.12.7 Project 7 – Reaction timer

Description: This is a reaction timer project which makes use of an LED and a push-button switch. The user is expected to press the push-button switch as soon as the LED is turned ON. The time between the LED being turned ON and the user pressing the button is measured and displayed on an LCD in seconds. The LED is turned ON again after a *random* delay, ready for the next measurement.

Block Diagram: Figure 9.22 shows the block diagram of the project.

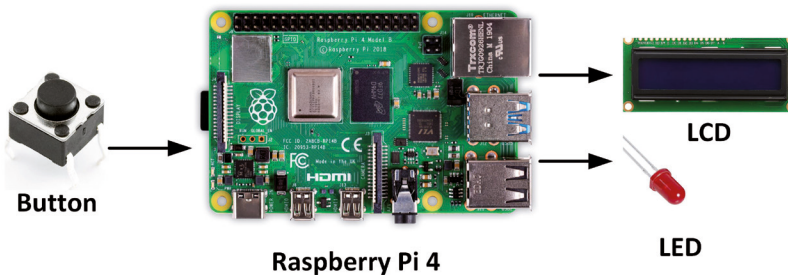


Figure 9.22 Block diagram of the project

Circuit Diagram: The circuit diagram of the project is shown in Figure 9.23. The LED and the button are connected to GPIO 21 and GPIO 20 respectively. The I²C LCD is connected to GPIO 2 and GPIO 3 SA and SCL pins of the Raspberry Pi as in the previous LCD projects.

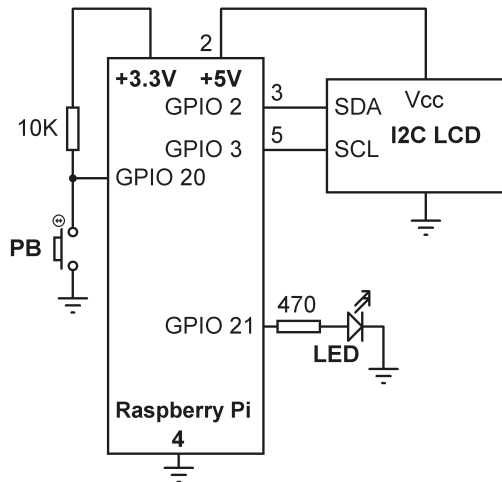


Figure 9.23 Circuit diagram of the project

Program Listing: The program listing (program: **reaction.py**) is shown in Figure 9.24. At the beginning of the program, the modules used in the program are imported. Notice module `random` is used to generate random numbers which are then used to generate random delays. Two events are created in the program with names **e** and **t**. Button (PB) is assigned number 20 and this port is configured as input.

The program consists of a process called **LED_ON**. This process configures the LED port as output and turns OFF the LED. The remainder of this process is executed in an endless loop. The LCD is controlled in the main program. Here the program waits until the LED is turned ON and then starts a timer. When the button is pressed, the timer reading is read and the elapsed time is calculated and displayed as the reaction time of the user.

```
#-----
#                               REACTION TIMER
#                               =====
#
# This is a reaction timer project. An LED is lit at random times
# and the user is requested to press a button as soon as it is lit.
# The time between seeing the LED lit and pressing the button is
# measured and displayed in seconds on the LCD.
#
# Author: Dogan Ibrahim
# File  : reaction.py
# Date  : June 2020
#-----
import RPi.GPIO as GPIO          # Import RPi
import multiprocessing          # Import multiprocessing
import random                   # Import random
import time                     # Import time
```

```

import RPi_I2C_driver                # I2C library
LCD = RPi_I2C_driver.lcd()           # Import LCD
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)               # GPIO mode BCM
e = multiprocessing.Event()          # Create event
t = multiprocessing.Event()          # Create event

PB = 20                               # Button at GPIO 20
GPIO.setup(PB, GPIO.IN)              # Button is input

#
# Process to turn ON the LED
#
def LED_ON():                         # Process LED_ON
    LED = 21                          # LED at GPIO 21
    GPIO.setup(LED, GPIO.OUT)         # LED is output
    GPIO.output(LED, 0)

    while True:                      # Do forever
        GPIO.output(LED, 0)          # LED OFF
        r = random.randint(1,10)    # Generate random no
        time.sleep(r)               # Wait random seconds
        GPIO.output(LED, 1)          # LED ON
        e.set()                     # Set event
        t.wait()                    # Wait for event
        t.clear()                   # Clear event
        GPIO.output(LED, 0)          # LED OFF
        time.sleep(5)               # Wait and repeat

#
# Create the process
#
p = multiprocessing.Process(target = LED_ON, args = ())
p.start()

#
# LCD Display control. Display the reaction time in seconds
#
LCD.lcd_clear()                      # Clear LCD
LCD.lcd_display_string("REACTION TIMER", 1) # Heading

while True:                          # DO forever
    e.wait()                         # Wait for event flag
    e.clear()                        # Clear event flag
    TimeStart = time.time()          # Start time
    while GPIO.input(PB) == 1:       # Button not pressed
        pass

```

```

TimeEnd = time.time()                # End time
t.set()
ReactionTime = str(TimeEnd - TimeStart)[:6] + " secs"
LCD lcd_display_string(ReactionTime, 2)    # Display reaction time

```

Figure 9.24 Program: reaction.py

Figure 9.25 shows the operation of the program.

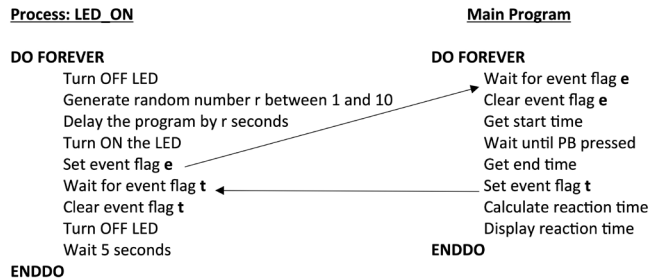


Figure 9.25 Operation of the program

An example display is shown in Figure 9.26 where the reaction time was 0.7370 seconds.



Figure 9.26 Example display

The multiprocessing Event has the following methods (assuming **e** is the created event flag):

e.wait(t): wait **t** seconds for the event flag to be set. If not set within the specified timeout, continue. Here, **t** is optional

e.set(): set the event flag

e.clear(): clear the event flag

e.is_set(): check if the event flag is set

9.12.8 Project 8 – Stepper motor controller with keyboard

Description: This project shows how to use a stepper motor in a multitasking environment. A small stepper motor is connected to the Raspberry Pi through a driver module. The operation of the motor is controlled from the keyboard using the following commands:

CWn: turn clockwise by n revolutions
 AWn: turn anticlockwise by n revolutions
 S: stop the motor
 X: terminate the program

Block Diagram: Figure 9.27 shows the block diagram of the project.

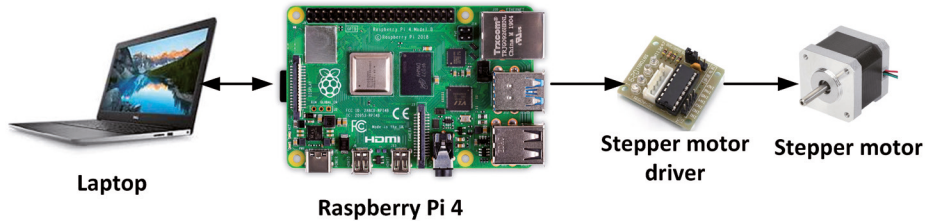


Figure 9.27 Block diagram of the project

Stepper motors

Stepper motors are DC motors that rotate in small steps. These motors have several coils that are energized in sequence, causing the motor to rotate one step at a time. Stepper motors have the advantages that very precise positioning or speed control of the motor shaft can be achieved. These motors are used in many precision motion control applications, in robotic arms, and in mobile robots to drive the wheels.

There are two types of stepper motors: unipolar and bipolar.

Unipolar stepper motors

Unipolar stepper motors have four windings with a common center tap on each pair of windings (see Figure 9.28). Therefore, there are normally 5 or 6 leads depending on whether the common leads are joined or not.

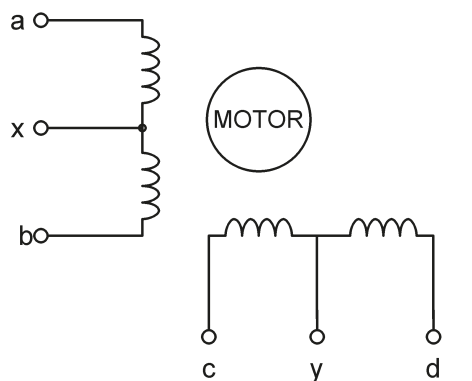


Figure 9.28 Unipolar stepper motor windings

Unipolar motors can be rotated in reverse by reversing the sequence of applied pulses. Unipolar stepper motors can be driven in full stepping mode or half-stepping mode. The most popular drive modes are 1 phase full-step, 2 phase full-step, and 2 phase half-step.

In 1 phase full-step mode, as shown in Table 9.1, each motor winding receives one pulse per step. This mode has the disadvantage that the available torque is low.

Step	a	c	b	d
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1

Table 9.1 1 Phase full-step mode

In 2-phase full-step mode, as shown in Table 9.2, two motor windings receive pulses per step. The advantage of this mode is that a higher torque is available from the motor.

Step	a	c	b	d
1	1	0	0	1
2	1	1	0	0
3	0	1	1	0
4	0	0	1	1

Table 9.2 2 Phase full-step mode

In 2-phase half-step mode, as shown in Table 9.3, two motor windings sometimes receive pulses, and sometimes only one winding receives a pulse. Because the motor is driven at half-step mode, 8 steps are required to complete a cycle instead of 4. This mode gives higher precision, but at the expense of lower torque.

Step	a	c	b	d
1	1	0	0	0
2	1	1	0	0
3	0	1	0	0
4	0	1	1	0
5	0	0	1	0
6	0	0	1	1
7	0	0	0	1
8	1	0	0	1

Table 9.3 2 Phase half-step mode

Bipolar stepper motors

Bipolar stepper motors have one winding per phase as shown in Figure 9.29. Bipolar motor driver circuits are more complicated since the current in the windings needs to be reversed to rotate them in the reverse direction.

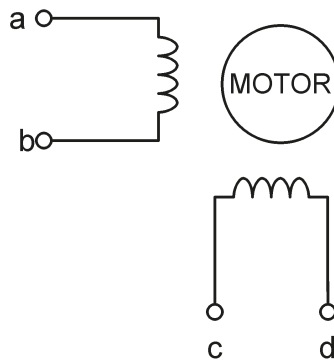


Figure 9.29 Bipolar stepper motor windings

Table 9.4 shows the steps required to drive a bipolar stepper motor. Here, + and – signs refer to the polarity of the voltages applied to the motor leads.

Step	a	c	b	d
1	+	-	-	-
2	-	+	-	-
3	-	-	+	-
4	-	-	-	+

Table 9.4 2 Bipolar stepper motor driving sequence

Speed of a stepper motor

The speed of a stepper motor depends on the time between the pulses given to its windings. For faster speeds, the pulses must be given with shorter delays between them. If **T** is the time between the pulses and β is the step constant of the motor, the motor rotates by β/T steps in one second. Since a complete revolution is 360° , the number of revolutions in a second is $\beta/360T$. The speed of a motor is normally quoted in RPM and therefore:

$$\text{RPM} = 60\beta/360T$$

or,

$$\text{RPM} = \beta/6T$$

where **RPM** is the number of revolutions per minute, β is the step constant of the motor in degrees, and **T** is the time between the steps in seconds.

As an example, assume that the step constant of a stepper motor is 10 degrees ($\beta = 10^\circ$). If we want to rotate this motor at a speed of 1000 RPM (assuming that the motor is capable of rotating this fast), the time between the pulses is calculated as:

$$T = \beta/6\text{RPM} = 10 / (6 \times 1000) = 1.66\text{ms}$$

Therefore, the pulses between each step must be 1.66ms.

Movement of the motor shaft

In some applications, we may want the motor shaft to rotate a specified amount and we need to know how many pulses to send to the motor. If β is the step constant of the motor and we want the shaft to rotate by v degrees, the required number of pulses is given by:

$$n = v/\beta$$

For example, assuming that the step constant is 5° ($\beta = 5$) and that we want the motor to rotate by 200 degrees, the required number of steps is:

$$n = 200/5 = 40$$

Motor rotation time

Sometimes we may want the motor to rotate for a given time and we want to know how many pulses to apply to the motor. If T is the time between the pulses and we send n pulses to the motor, the rotation time T_0 can be calculated as follows:

$$T_0 = nT$$

For example, assuming the time between the pulses is 1ms, if we want the motor to rotate for 5 seconds, the required number of pulses is given by:

$$N = T_0/T = 5/0.001 = 5000$$

Stepper motors can be driven by several ways, such as using bipolar transistors, using MOSFET transistors, or using integrated circuits such as L293, ULN2003, and so on.

Circuit Diagram: In this project, a small 28BYJ-48 type unipolar stepper motor (see Figure 9.30) is used. This motor has the following specifications:

Rated voltage:	5V
Number of phases:	4
Gear ratio:	64
Frequency:	100Hz
Step angle:	11.25° / step
Maximum speed:	18 RPM

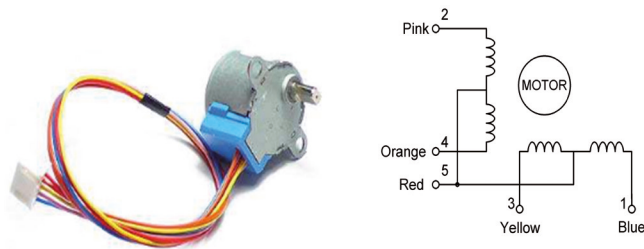


Figure 9.30 28BYJ-48 unipolar stepper motor

In this project, the motor is driven using a ULN2003 IC-based motor driver module shown in Figure 9.31 together with its circuit diagram. This module has four input connections labelled IN1, IN2, IN3, and IN4. The motor is plugged into the socket in the middle of the module. Four LEDs, labelled A, B, C, D are provided to see the status of the motor windings. Power to the module is applied through the bottom two header pins on the right-hand side of the module. The LEDs can be enabled by shorting the two top header pins on the right-hand side of the module. In this project, the module is powered from an external +5V DC power supply for the motor (it is recommended to use an external +5V power supply, and not use the Raspberry Pi +5V power supply as it may not provide enough current to drive the motor).

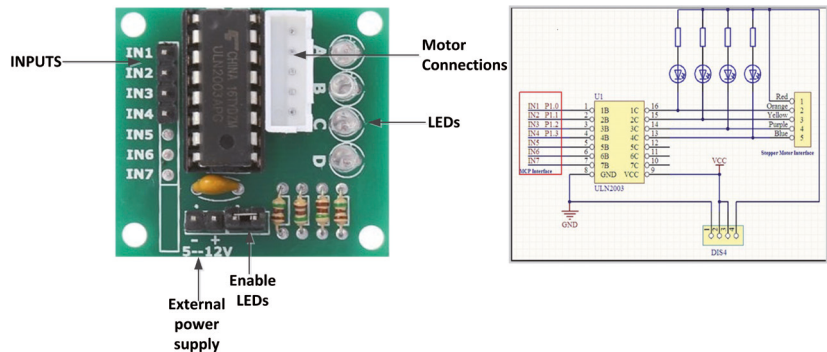


Figure 9.31 ULN2003 motor driver module

Figure 9.32 shows the circuit diagram of the project. The connections between the stepper motor controller and Raspberry Pi are as follows:

Stepper motor controller	Raspberry Pi
IN1	GPIO 12
IN2	GPIO 16
IN3	GPIO 20
IN4	GPIO 21

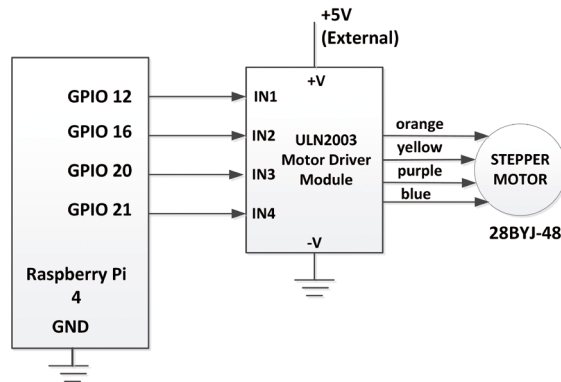


Figure 9.32 Circuit diagram of the project

Program Listing: The 28BYJ-48 stepper motor can either be operated in full-step or in half-step modes.

Full-step mode

In full-step mode, there are 4 steps per cycle and 11.25 degrees/step, corresponding to 32 steps per one revolution of the internal motor shaft. Because the motor is geared with a gear ratio of 64 (in fact the gear ratio is 63.68395), the number steps for one external complete revolution are 2048 steps/revolution (512 cycles with 4 steps per cycle).

Table 9.5 shows the motor winding sequence for the full-step mode (this sequence is repeated. Reversing the sequence reverses the direction of rotation).

Step	4 (orange) IN1	3 (yellow) IN2	2 (pink) IN3	1 (blue) IN4
1	1	1	0	0
2	0	1	1	0
3	0	0	1	1
4	1	0	0	1

Table 9.5 Full-step mode

Half-step mode

The half-step mode with 8 steps per cycle is recommended by the manufacturer. In half-step mode we have 5.625 degrees/step, corresponding to 64 steps per one revolution of the internal motor shaft. Because the motor is geared with a gear ratio of 64, the number of steps for one external complete revolution is 4096 steps/revolution (512 cycles with 8 steps per cycle).

Table 9.6 shows the motor winding pulse sequence for the half-step mode (this sequence is repeated. Reversing the sequence reverses the direction of rotation).

Step	4 (orange) IN1	3 (yellow) IN2	2 (pink) IN3	1 (blue) IN4
1	1	0	0	0
2	1	1	0	0
3	0	1	0	0
4	0	1	1	0
5	0	0	1	0
6	0	0	1	1
7	0	0	0	1
8	1	0	0	1

Table 9.6 Half-step mode

In this project, the stepper motor is controlled in Half-Step mode and Figure 9.33 shows the program listing (program: **stepper.py**).

The speed of the motor depends on the delay inserted between each step. In Half-step mode there are 4096 steps in a complete revolution. The motor speed in RPM is given by the following equation:

$$\text{RPM} = 60 \times 10^3 / (4096 \times T)$$

or,

$$\text{RPM} = 14.648 / T$$

Where, RPM is the motor speed in revolutions per minute, and **T** is the delay between each step in milliseconds. We usually want to know how much delay to insert between each step so that the required number of revolutions can be achieved. This is given in milliseconds by:

$$T = 14.648 / \text{RPM}$$

or,

$$T = 0.014648 / \text{RPM}$$

Where **T** is now in seconds.

At the beginning of the program, 2 queues named **command** and **rev**, and an event named **e** is created. Queue **command** stores the entered command, and **rev** stores the number of required revolutions. Event flag **e** is used to stop the motor.

The program consists of a process called **STEPPER**. Inside this process, the connections between the stepper motor driver pins **IN1**, **IN2**, **IN3**, and **IN4** and the Raspberry Pi are defined and these pins are configured as outputs. The **RPM** of the motor is set to 12, and the motor parameters **StepsPerRevolution**, **StepDelay**, and **HalfStep** sequence are defined. The remainder of process **STEPPER** implements the actual motor control. If the user command is **C** then the required number of revolutions is read from queue **rev** and function

CLOCKWISE is called to rotate the motor shaft in the clockwise direction. If on the other hand, the user command is **A**, the required number of revolutions is read and function **ANTICLOCKWISE** is called to rotate the motor shaft in the anticlockwise direction.

Inside functions **CLOCKWISE** and **ANTICLOCKWISE**, the program checks if the event flag **e** is set and stops the program if it is (this event flag is set if the user enters command **S** to stop the motor).

The main program prompts the user to enter one of the following valid commands on their keyboard. An error message is displayed if an invalid command is entered:

CWn	Rotate motor clockwise by n revolutions
Awn	Rotate the motor shaft anticlockwise by n revolutions
S	Stop the motor
X	Terminate the program

For example, entering **CW5** will rotate the motor shaft in the clockwise direction by 5 revolutions. Entering command **S** at any time on the keyboard will stop the motor immediately. The user command is put into queue **command**. The number of revolutions is extracted, converted into an integer, and then put into queue **rev**. Notice that user commands can be entered as either lower case or upper case since they are all converted to upper case before they are processed.

```
#-----
#                               STEPPER MOTOR CONTROL
#                               =====
#
# This is a stepper motor control project. A small stepper motor
# connected to the Raspberry Pi is controlled by entering the
# following commands from the keyboard:
#
# CW : rotate clockwise
# AW: rotate anticlockwise
# S  : stop
# X  : terminate program
#
# Author: Dogan Ibrahim
# File  : stepper.py
# Date  : June 2020
#-----
import RPi.GPIO as GPIO          # Import RPi
import multiprocessing          # Import multiprocessing
import time                    # Import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)         # GPIO mode BCM
```

```
command = multiprocessing.Queue()           # Create queue
rev = multiprocessing.Queue()               # Create queue
e = multiprocessing.Event()                 # Create event

#
# This function sends pulses to the motor in Half-Step mode
#
def SendPulse(k):
    global IN1, IN2, IN3, IN4mHalfStep
    GPIO.output(IN1, HalfStep[k][3])        # Send to IN1
    GPIO.output(IN2, HalfStep[k][2])        # Send to IN2
    GPIO.output(IN3, HalfStep[k][1])        # Send to IN3
    GPIO.output(IN4, HalfStep[k][0])        # Send to IN4

#
# This function rotates the motor clockwise. VARIABLe count is the
# number of times the motor shaft will rotate 360 degrees
#
def CLOCKWISE(count):
    global StepsPerRevolution, StepDelay
    for j in range(0, count):
        for m in range(0, StepsPerRevolution):
            for i in range(7, -1, -1):
                SendPulse(i)
                if e.is_set():                # Is event flag e set?
                    e.clear()                # Clear event flag e
                    return                    # Stop the motor
            time.sleep(StepDelay)

#
# This function rotates the motor anticlockwise. Variable count is the
# number of times the motor shaft will rotate 360 degrees
#
def ANTICLOCKWISE(count):
    global StepsPerRevolution, StepDelay
    for j in range(0, count):
        for m in range(0, StepsPerRevolution):
            for i in range(0, 8):
                SendPulse(i)
                if e.is_set():                # Is event flag e set?
                    e.clear()                # Clear event flag e
                    return                    # Stop the motor
            time.sleep(StepDelay)

#
```

```
# Process to control the stepper motor
#
def STEPPER():
    # Process STEPPER
    global IN1,IN2,IN3,IN4,RPM,StepsPerRevolution,StepDelay,HalfStep
    IN1 = 12 # IN1 is at GPIO 12
    IN2 = 16 # IN2 is at GPIO 16
    IN3 = 20 # IN3 is at GPIO 20
    IN4 = 21 # IN4 is at GPIO 21

    RPM = 12 # RPM
    StepsPerRevolution = 512 # Steps/Rev
    StepDelay = 0.014648 / RPM # Step delay

    GPIO.setup(IN1, GPIO.OUT) # IN1 is output
    GPIO.setup(IN2, GPIO.OUT) # IN2 is output
    GPIO.setup(IN3, GPIO.OUT) # IN3 is output
    GPIO.setup(IN4, GPIO.OUT) # IN4 is output
#
# Define Half-Step sequence
#
HalfStep = [[1,0,0,0],
             [1,1,0,0],
             [0,1,0,0],
             [0,1,1,0],
             [0,0,1,0],
             [0,0,1,1],
             [0,0,0,1],
             [1,0,0,1]]
#
# Motor control loop
#
while True:
    # Do forever
    if not command.empty():
        # If a command received
        c = command.get()
        # Get the command
        if c == "C":
            # Is it C?
            r = rev.get()
            # Get the revolutions
            CLOCKWISE(r)
            # Rotate clockwise
        elif c == "A":
            # Is it A?
            r = rev.get()
            # Get the revolutions
            ANTICLOCKWISE(r)
            # Rotate anticlockwise

#
# Create the process
#
p = multiprocessing.Process(target = STEPPER, args = ())
p.start()
```

```
#
# Read control parameters from the keyboard. Read a command and the
# required number of revolutions of the motor shaft
#
while True:                                # Do forever
    print("")
    cmd = input("Enter a command (CWn,AWn,S, X): ").upper()
    if cmd.find("CW") >= 0:
        revs = int(cmd[2:])
        rev.put(revs)
        command.put("C")                    # Put "C" in queue
    elif cmd.find("AW") >= 0:                # Is it "AW"?
        revs = int(cmd[2:])                 # Get rev count
        rev.put(revs)                       # Put rev count in queue
        command.put("A")                    # Put "A" in queue
    elif cmd.find("S") >= 0:                 # Is it "S"?
        e.set()                             # Set event flag e
    elif cmd.find("X") >= 0:                 # Is it "X"?
        p.terminate()                       # Terminate program
        print("End of program")              # Display message
        exit(0)                             # Exit
    else:
        print("Invalid command...")          # Display invalid
```

Figure 9.33 Program: stepper.py

An example display is shown in Figure 9.34.

```
pi@raspberrypi:~ $ python3 stepper.py

Enter a command (CWn,AWn,S, X): cw2

Enter a command (CWn,AWn,S, X): s

Enter a command (CWn,AWn,S, X): x
End of program
pi@raspberrypi:~ $ █
```

Figure 9.34 Example display

9.12.9 Project 9 – Setting the flashing rate of an LED with keypad

Description: In this project, an LED, keypad, and LCD are connected to the Raspberry Pi. The flashing rate of the LED is set using the keypad. This project aims to show how a keypad can be used in a multitasking environment. Like the 7-segment displays, keypads are ideal applications for multitasking.

Keypads are used in many microcontroller-based applications since they are small, portable, and do not require any external power supplies.

Block Diagram: Figure 9.35 shows the block diagram of the project. The **D** key is used as the **Enter** key.



Figure 9.35 Block diagram of the project

The Keypad: Several types of keypads can be used in microcontroller based projects. In this project, a 4x4 keypad (see Figure 9.36) is used. This keypad has keys for numbers 0 to 9 and letters A,B,C,D,*, and #. The keypad is interfaced to the processor with 8 wires with the names R1 to R4 and C1 to C4, representing the rows and columns respectively of the keypad (see Figure 9.37).



Figure 9.36 4x4 keypad

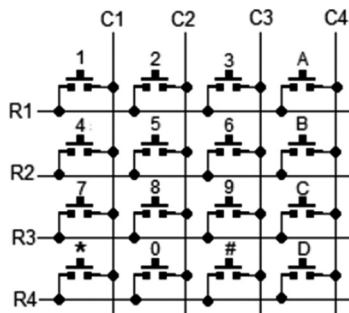


Figure 9.37 Circuit diagram of the 4x4 keypad

The operation of the keypad is very simple: the columns are configured as outputs and the rows as inputs. The key pressed is identified by using column scanning. Here, a column is forced low while the other columns are held high. Then the state of each row is scanned, and if a row is found to be low, the key at the intersection of the row (which is low) and this column is the key pressed. This process is repeated for all rows.

Circuit Diagram: Figure 9.38 shows the circuit diagram of the project. The I²C LCD is connected to the Raspberry Pi as in the previous projects using the LCD, where GPIO 2 and GPIO 3 are used as the SDA and SCL pins respectively. The LED is connected to GPIO 21 of the Raspberry Pi. The 4x4 keypad is connected to the following GPIO pins of the Raspberry Pi. The row pins are held high using 10K pull-up resistors to +3.3V (notice that Raspberry Pi has internal pull-up resistors when a pin is used as an input but the use of these pull-up resistors are not reliable):

Keypad pin	Raspberry Pi pin
R1	GPIO 14
R2	GPIO 15
R3	GPIO 12
R4	GPIO 23
C1	GPIO 24
C2	GPIO 25
C3	GPIO 8
C4	GPIO 7

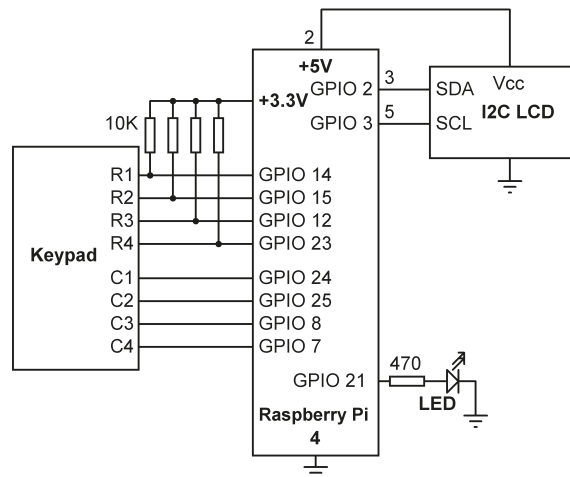


Figure 9.38 Circuit diagram of the project

Figure 9.39 shows the pin configuration of the 4x4 keypad used in the project.



Figure 9.39 Pin configuration of the 4x4 keypad

Test program

Before writing the project program we will, first of all, develop the code to read keys from the keypad. The basic steps to read a key are as follows:

```

Configure all columns as outputs
Configure all rows as inputs
Set all columns to 1
DO for all columns
  Set a column to 0
  DO for all rows
    IF a row is 0 THEN
      Return the key at this column and row position
    ENDIF
  ENDDO
ENDDO

```

Figure 9.40 shows the test program (program: **keypad.py**).

At the beginning of the program the keypad keys are defined after importing the required modules to the program. The keypad row and columns connections are defined using lists **ROWS** and **COLS** respectively. Columns are then configured as outputs and are set to 1. Similarly, the rows are configured as inputs. Function **Get_Key** reads the pressed key and returns it to the calling program. Two **for** loops are used in the function: the first loop selects the columns and sets them to 0 one after the other one. The second loop scans the rows and checks if a row is at 0. The main program calls the function and displays the pressed key on the screen.

```
#-----
#                               KEYPAD TEST PROGRAM
#                               =====
#
# This program shows how the a keypad can be used to display
# the pressed keys
#
# Author: Dogan Ibrahim
# File  : keypad.py
# Date  : June 2020
#-----

import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

KEYPAD = [                                # Keypad keys
    [1,2,3,"A"],
    [4,5,6,"B"],
    [7,8,9,"C"],
    ["*",0,"#", "D"]]

ROWS = [14,15,12,23]                     # Row pins
COLS = [24,25,8,7]                        # Column pins

for i in range(4):                         # Conf columns
    GPIO.setup(COLS[i], GPIO.OUT)
    GPIO.output(COLS[i], 1)

for j in range(4):                         # Conf rows
    GPIO.setup(ROWS[j], GPIO.IN)

#
# This function reads a key from the keypad
#
def Get_Key():
    while True:
        for j in range(4):
            GPIO.output(COLS[j], 0)        # Set col j to 0
            for i in range(4):             # For all rows
                if GPIO.input(ROWS[i]) == 0: # Row is 0?
                    return (KEYPAD[i][j])  # Return key
                    while GPIO.input(ROWS[i]) == 0:
                        pass
            GPIO.output(COLS[j], 1)        # Col back to 1
```

```
        time.sleep(0.05)                                # Wait 0.05s

try:
    while True:
        key = Get_Key()                                # Get a key
        print(key)                                     # Display the key
        time.sleep(0.5)

except KeyboardInterrupt:
    GPIO.cleanup()
```

Figure 9.40 Program: keypad.py

Program Listing: We are now ready to develop the program of this project. Figure 9.41 shows the program listing (program: **keypadled.py**). In this program, key **D** is assumed to be the **ENTER** key where all inputs to the keypad must be terminated by pressing the **ENTER** key. There is one process in the program called **FLASH**. This process flashes the LED at a rate set by variable **dly** (the default value of **dly** is set to one second). The flashing rate is extracted from the queue and is loaded into variable **dly**.

The main program controls the LCD and keypad to receive the required flashing rate. The top row of the LCD shows the text **Flash rate (ms):** The program runs in an endless loop where the keys entered by the user are read and the required total delay is calculated and stored in variable **Total** until the **ENTER** key is pressed. The LCD shows each number entered by the user in the second row and when the **ENTER** key is pressed, the required flashing rate is calculated in seconds by dividing the number read by 1000. This number (in variable **tim**) is then put into the queue so that it can be extracted by process **FLASH** to change the flashing rate. The **LED SET** text is then displayed for 2 seconds to confirm the entered number has been accepted and the flashing rate has been changed. After 2 seconds, the second row of the LCD is cleared, ready for the next entry.

```
#-----
#           SETTING LED FLASHING RATE WITH KEYPAD
#           =====
#
# In this program an LED is connected to the Raspberry Pi and
# the flashing rate of the LED is set using the keypad
#
# Author: Dogan Ibrahim
# File  : keypadled.py
# Date  : June 2020
#-----

import RPi.GPIO as GPIO
import time
import multiprocessing
import RPi_I2C_driver
```

```

LCD = RPi_I2C_driver.lcd()

q = multiprocessing.Queue()           # Create queue
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

KEYPAD = [                             # Keypad keys
    [1,2,3,"A"],
    [4,5,6,"B"],
    [7,8,9,"C"],
    ["*",0,"#", "D"]]

ROWS = [14,15,12,23]                  # Row pins
COLS = [24,25,8,7]                    # Column pins

for i in range(4):                     # Conf columns
    GPIO.setup(COLS[i], GPIO.OUT)
    GPIO.output(COLS[i], 1)

for j in range(4):                     # Conf rows
    GPIO.setup(ROWS[j], GPIO.IN)

#
# This function reads a key from the keypad
#
def Get_Key():
    while True:
        for j in range(4):
            GPIO.output(COLS[j], 0)      # Set col j to 0
            for i in range(4):           # For all rows
                if GPIO.input(ROWS[i]) == 0:  # Row is 0?
                    return (KEYPAD[i][j])    # Return key
                    while GPIO.input(ROWS[i]) == 0:
                        pass
            GPIO.output(COLS[j], 1)      # Col back to 1
            time.sleep(0.05)             # Wait 0.05s

#
# This process flashes the LED at the rate dly which is
# entered from the keyboard
#
def FLASH():                             # Process FLASH
    LED = 21                             # LED at GPIO 21
    GPIO.setup(LED, GPIO.OUT)            # LED is output
    dly = 1                              # 1 second (default)

```

```

while True:
    if not q.empty():
        dly = q.get()
        GPIO.output(LED, 1)
        time.sleep(dly)
        GPIO.output(LED, 0)
        time.sleep(dly)

p = multiprocessing.Process(target = FLASH, args = ())
p.start()

#
# Main program. Here the flashing rate is read from the keypad
# and displayed on the LCD and is then sent to process FLASH

LCD.lcd_clear()
LCD.lcd_display_string("Flash rate (ms):", 1)
Total = 0

while True:
    key = Get_Key()
    if key != "D":
        LCD.lcd_display_string(str(key), 2)
        N = int(key)
        Total = 10*Total + N
        LCD.lcd_display_string(str(Total), 2)
    else:
        tim = Total / 1000
        q.put(tim)
        LCD.lcd_display_string("LED SET", 2)
        time.sleep(2)
        LCD.lcd_display_string("      ", 2)
        Total = 0
        time.sleep(0.5)

```

Figure 9.41 Program: keypadled.py

An example display is shown in Figure 9.42. In this example, the flashing rate was changed to 100ms.



Figure 9.42 Example display

9.12.10 Project 10 – Secure door lock with keypad

Description: This is a multitasking secure door lock project. In this project, an LED, LCD, relay, and keypad are connected to the Raspberry Pi. The project activates the relay to open a door (or safe) if the correct code is entered on the keypad. The secret code in this project is preset to **1234** for simplicity. The LED flashes to indicate that the system is ready to accept the code. If the code is entered wrongly 3 times, the LED is turned OFF and the system is locked for 5 minutes, accepting no code entries. The LCD shows the code entered by the user.

Block Diagram: Figure 9.43 shows the block diagram of the project. The **D** key is used as the **Enter** key.

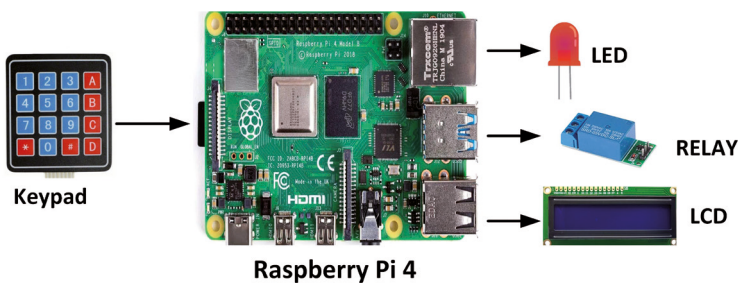


Figure 9.43 Block diagram of the project

Circuit Diagram: Figure 9.44 shows the circuit diagram of the project. The I²C LCD is connected to the Raspberry Pi as in the previous projects using the LCD, where GPIO 2 and GPIO 3 are used as the SDA and SCL pins respectively. The LED and the relay are connected to GPIO 21 and GPIO 20 of the Raspberry Pi. The 4x4 keypad is connected as in the previous project. It is assumed the relay is activated when logic 1 is applied to its terminals, and that the door mechanism is opened when the relay is activated.

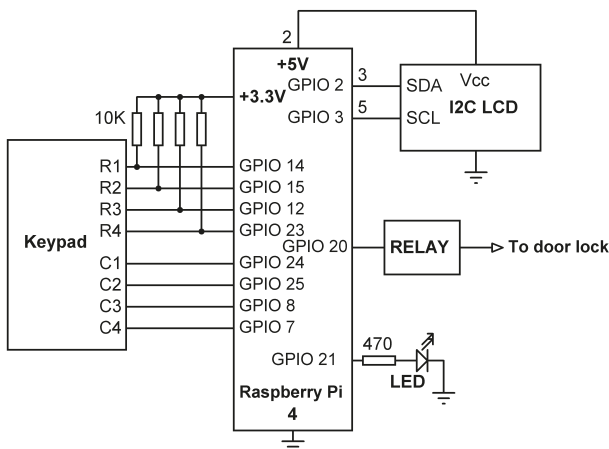


Figure 9.44 Circuit diagram of the project

Program Listing: Figure 9.45 shows the program listing (program: **door.py**). The operation of the program can be summarised as follows:

When the program is started, the relay is deactivated, the LED starts flashing, and the text **Enter code:** is displayed on the first row of the LCD. The user is then expected to enter the secret code to open the door. If the code **1234** is entered, the relay will be activated for 10 seconds, message **Opened** will be displayed on the LCD, and the relay will be deactivated at the end of 10 seconds. At this time, it is assumed that the door will be closed. If the user enters the wrong code, the message **Try Again** will be displayed on the second row of the LCD. The user is given 3 consecutive attempts to enter the correct code and if the code is wrong, the message **Disabled** will be displayed on the second row of the LCD and the LED will be turned OFF to indicate the system is disabled and does not accept a code. The system is disabled for 5 minutes. After this time, the LED is turned ON again so that it starts flashing, and the user is given the chance to enter the secret code again. The above process is repeated forever until the program is terminated by the user.

As shown in Figure 9.45, at the beginning of the program the modules used by the program are imported, an event is created and the keypad pins are configured. The program consists of a process called **FLASH**. This process flashes the LED every 0.25 seconds as long as the event flag **e** is cleared. If the event flag is set, then the LED is turned OFF. The event flag is set when the user enters the wrong code 3 consecutive times. The main program controls the relay and LCD. The relay is initially deactivated. Function **Get_Key** receives the code entered by the user. Notice that the code must be terminated by pressing the **ENTER** key which is key **D** on the keypad. When the **ENTER** key is detected, the program compares the code entered with the secret code stored in variable **SecretCode**. The relay will be activated if the entered code is correct as described earlier.

```
#-----
#                               SECRET DOOR LOCK WITH KEYPAD
#                               =====
#
# In this program an LED,a relay, an LCD and a keypad are connected
# to the Raspberry Pi. The relay is activated when the correct secret
# code is entered on the keypad. The LED flashes to indicate when the
# system is ready to accept the code. If the code is entered wrong 3
# times then the system is disabled for 5 minutes.
#
# Author: Dogan Ibrahim
# File  : door.py
# Date  : June 2020
#-----

import RPi.GPIO as GPIO
import time
import multiprocessing
import RPi_I2C_driver
LCD = RPi_I2C_driver.lcd()
```

```
e = multiprocessing.Event()                # Create event
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

KEYPAD = [                                  # Keypad keys
    [1,2,3,"A"],
    [4,5,6,"B"],
    [7,8,9,"C"],
    ["*",0,"#", "D"]]

ROWS = [14,15,12,23]                       # Row pins
COLS = [24,25,8,7]                         # Column pins

for i in range(4):                          # Conf columns
    GPIO.setup(COLS[i], GPIO.OUT)
    GPIO.output(COLS[i], 1)

for j in range(4):                          # Conf rows
    GPIO.setup(ROWS[j], GPIO.IN)

#
# This function reads a key from the keypad
#
def Get_Key():
    while True:
        for j in range(4):
            GPIO.output(COLS[j], 0)          # Set col j to 0
            for i in range(4):               # For all rows
                if GPIO.input(ROWS[i]) == 0: # Row is 0?
                    return (KEYPAD[i][j])    # Return key
                    while GPIO.input(ROWS[i]) == 0:
                        pass
            GPIO.output(COLS[j], 1)          # Col back to 1
            time.sleep(0.05)                # Wait 0.05s

#
# This process flashes the LED at the rate dly which is
# entered from the keyboard
#
def FLASH():                                # Process FLASH
    LED = 21                               # LED at GPIO 21
    GPIO.setup(LED, GPIO.OUT)              # LED is output

    while True:                             # Do forever
        while e.is_set():                  # If event is set:
```

```
        pass                                # Wait here
GPIO.output(LED, 1)                        # LED ON
time.sleep(0.25)                          # Wait 0.25 sec
GPIO.output(LED, 0)                        # LED OFF
time.sleep(0.25)                          # Wait 0.25 sec

p = multiprocessing.Process(target = FLASH, args = ())
p.start()

#
# Main program. Here the secret code is read and the relay is
# activated if the code is correct, otherwise the system is disabled
#
Relay = 20                                # Relay at GPIO 20
GPIO.setup(Relay, GPIO.OUT)               # Relay is output
GPIO.output(Relay, 0)                     # relay OFF

SecretCode = 1234                         # Secret code
LCD.lcd_clear()
LCD.lcd_display_string("Enter code:", 1) # Heading
Total = 0
ErrorCount = 0                           # Error count
e.clear()                                # Clear event flag

while True:                               # Do forever
    key = Get_Key()                       # Get a key
    if key != "D":                         # If not ENTER
        LCD.lcd_display_string(str(key), 2) # Display
        N = int(key)                     # In integer
        Total = 10*Total + N             # Total number
        LCD.lcd_display_string(str(Total), 2) # Display
    else:                                  # If ENTER
        if Total == SecretCode:           # If correct code
            GPIO.output(Relay, 1)          # Relay ON
            LCD.lcd_display_string("Opened ", 2)
            time.sleep(10)                 # Wait 10 secs
            GPIO.output(Relay, 0)          # Relay OFF
            LCD.lcd_display_string("      ", 2)
            Total = 0
            ErrorCount = 0
        else:                             # Wrong code
            ErrorCount = ErrorCount + 1    # Increment Error
            if ErrorCount == 3:             # 3 errors?
                e.set()                   # Set event flag
                LCD.lcd_display_string("Disabled ", 2)
                time.sleep(300)            # Wait 5 mins
```

```
ErrorCount = 0                                # Reset Errors
Total = 0
e.clear()                                     # Clear event flag
LCD lcd_display_string("          ", 2)
else:
    LCD lcd_display_string("Try again", 2)
    time.sleep(2)
    LCD lcd_display_string("          ", 2)
    Total = 0
time.sleep(0.5)                               # Wait 0.5s
```

Figure 9.45 Program: door.py

Figure 9.46 shows an example display from the program.



Figure 9.46 Example display from the program

9.12.11 Project 11 – Car park control

Description: This is a car park management system where the idea is to control the operation of a car park. It is assumed the car park has one level with a capacity of 100 cars. An LCD close to the car park shows the number of free spaces available in the car park. Only members of the car park are allowed to use the car park where each member is given an RFID (Radio Frequency IDentification) card for identification. A barrier is placed at the entrance to the car park which is operated (i.e. lifted) with a stepper motor. If there are spaces in the car park and when a valid member customer places an authorized RFID card close to the card reader near the entrance to the car park, the barrier is lifted automatically to let the driver into the car park. Unauthorized RFID cards are rejected and the barrier is not lifted. There is no barrier at the exit from the car park, but a passive pressure switch is placed under the exit route that detects the vehicles as they leave the car park. The number of spaces available inside the car park is calculated as the cars enter and leave the car park and this is displayed continuously on the LCD. The barrier is not lifted if there are no spaces inside the car park. A red light and a green light (e.g. LED in this project) are mounted on the entrance to the car park. When the red LED is ON, the driver is required to wait until the Green LED comes ON. The Green LED comes ON when the barrier is lifted, so that a car can enter the car park. When the barrier is down, the Red LED comes ON and the Green LED is turned OFF.

All operations of the car park are controlled using multitasking with the Python programming language on a Raspberry Pi 4.

Block Diagram: Figure 9.47 shows the block diagram of the car park. The physical interface to the Raspberry Pi is not shown in this Figure.

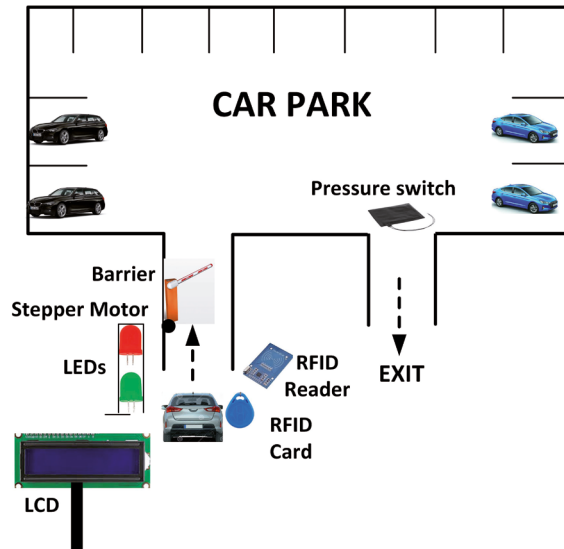


Figure 9.47 Block diagram of the car park

The stepper motor

As was described in an earlier project (Project 8), stepper motors can be driven in quite a few ways, such as by using bipolar transistors, using MOSFET transistors, or by using integrated circuits such as L293, ULN2003 and so on. In this project, the 28BYJ-48 unipolar stepper motor is used (see Figure 9.30) with the ULN2003 IC-based motor driver module (see figure 9.31) to control the barrier at the entry to the car park. The stepper motor rotates anticlockwise by 90 degrees to lift the barrier to let a car into the car park. Similarly, it is rotated by 90 degrees clockwise to close the barrier after the vehicle has entered the car park.

The pressure switch

A pressure switch (or button) is placed under the road in the path of the exit route that detects when a car is leaving the car park. The state of this switch is at logic 0 and goes to logic 1 when a car is over the switch. This is detected by the system and is used in calculating the available free spaces in the car park.

The RFID reader

RFID systems consist of RFID devices (or readers) and RFID tags (or cards). RFID devices use electromagnetic fields to automatically identify and track compatible RFID tags. The tags contain unique electronically stored information that is read by the RFID readers. RFID tags are used in many industries and commonly in security applications. For example, the reader-tag pairs can be used to unlock doors, they can be attached to possessions, or implanted in animals or even people so that they can be tracked. RFIDs are similar to the barcodes used in supermarkets to identify products, but unlike barcodes, the tags do not

need to be within the line of sight of the readers. Additionally, tags can contain much more information than simple barcodes.

RFID tags can be either passive or active (battery operated). Passive tags are cheaper and smaller and are more commonly used. Passive tags must be placed very close to the RFID readers (e.g. at 5cm) so that their contents can be read. These tags can be read-only or read-write type. Read-only tags are pre-programmed in factories with unique numbers and these numbers can be read using compatible RFID readers. Active RFID tags have the advantage that they can be read from longer distances, but they are much more expensive than passive tags.

In this project, a passive RFID tag system is used. The RFID reader used in this project is the RDM6300 type UART based reader (see Figure 9.48). This reader is EM4100 protocol compatible and operates with the 125kHz RFID tags (see Figure 9.49). The basic features of the RDM6300 readers are as follows:

- Operating frequency: 25kHz
- Working voltage: +5V
- Current consumption: < 50mA
- Receive distance: < 5cm
- Working temperature: -10°C~+70°C

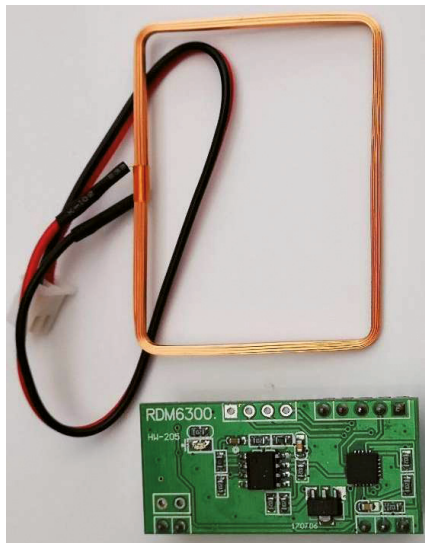


Figure 9.48 RDM6300 RFID reader



Figure 9.49 125kHz RFID tags

The RDM6300 is sold with a coil antenna that communicates with the RFID cards. This antenna must be connected to the reader board. Figure 9.50 shows the pin configuration of the RDM6300 reader.



Figure 9.50 RDM6300 pin configuration

There are 3 headers on the board with the following functions. Notice header 3 is for testing where an optional external LED can be connected. The LED goes from logic HIGH to LOW when an RFID tag is present:

Header P1

- | Pin number | Description |
|------------|--------------------|
| 1 | TX (UART transmit) |
| 2 | RX (UART receive) |
| 3 | Not used |
| 4 | GND |
| 5 | +5V power supply |

Header P2

1. Antenna
2. Antenna

Header 3

1. External LED
2. +5V power supply
3. GND

The RDM6300 reader operates at 9600 Baud, 8 data bits, 1 stop bit, and no parity bit. At 9600 baud, bit time is 104 μ s. The interface protocols available for this reader are the Weingang26 and TTL level RS232. In this project, the TTL level RS232 interface and protocol is used. Normally, the standard RS232 signal voltage levels are $\pm$ 12V. TTL based RS232 signal voltage levels are 0 or +5V (or 0 to +3.3V) where the line is normally at +5V and goes to 0V at the beginning of data transmission (i.e. the start bit is at logic 0V). Reader output consists of 14-bytes as follows:

- 1-byte start flag (0x02)
- 10 ASCII data characters
- 2-byte checksum
- 1-byte end flag (0x03)

The start and end flags are always 0x02 and 0x03 respectively. The checksum is calculated by exclusive Or'ing all the data bytes.

Notice the card number marked on the RFID cards is a 10 digit decimal number. For example, if the decimal number on the card is 007564912, it corresponds to hexadecimal number 00736E70.

Circuit Diagram: The circuit diagram of the project is shown in Figure 9.51. The I²C LCD is connected to GPIO 2 and 3 as in the previous I²C LCD projects. The stepper motor is attached to the motor driver board and the board pins IN1, IN2, IN3, IN4 are connected to port pins GPIO 23, GPIO 24, GPIO 25, and GPIO 8 respectively. The motor driver board is powered from an **external** +5V power supply. The pressure switch is connected to port pin GPIO 16. The output of this switch is at logic 1 and goes to logic 0 when a car is present on the switch. The TX pin of the RFID reader module is connected to UART pin GPIO 15 (RXD) of the Raspberry Pi. The RX pin of the RFID reader is not used in this project. Notice that although the RDM6300 operates with +5V, the voltage at its TX output does not get more than +3.3V and as a result of this, we can directly connect the TX pin to the RXD pin of the Raspberry Pi (GPIO 15). The red and the green LEDs are connected to GPIO 21 and GPIO 20 respectively through 470 Ohm current limiting resistors.

The interface between the Raspberry Pi and the various peripheral devices is summarized below:

External Device	Raspberry Pi GPIO
Pressure switch	GPIO 16
RFID reader TX	GPIO 15
Stepper motor	GPIO 23, GPIO 24, GPIO 25, GPIO 8
Red LED	GPIO 21
Green LED	GPIO 20
LCD	GPIO 2, GPIO 3
GND of devices	GND
+3.3V	Pressure switch
+5V	RDM6300 power

The antenna of the RFID reader module is connected to header pins P2. The LED interface of the RFID reader module is not used in this project.

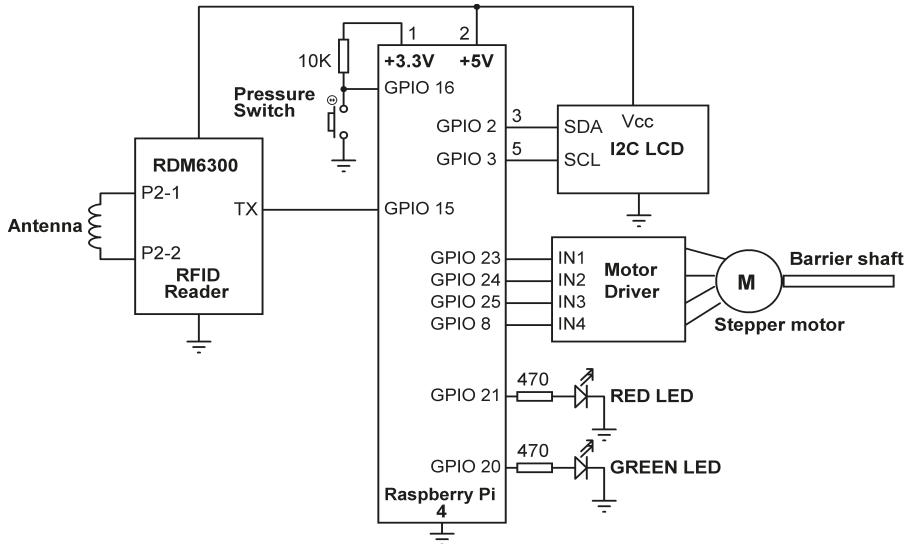


Figure 9.51 Circuit diagram of the project

Program Listing: The RDM6300 RFID reader uses the serial port of the host processor. We have to enable the serial port of the Raspberry Pi 4 before using the RFID device. This is described below in some detail.

The Raspberry Pis have two built-in UARTs: a PL011 and a mini UART. They are implemented using different hardware blocks and therefore have slightly different characteristics. On Raspberry Pis which are equipped with Wireless/Bluetooth modules (e.g. Raspberry Pi 3, Zero W, 4, etc), the PL011 UART is connected by default to the Bluetooth module, while the mini UART is the primary UART with the Linux console on it. In all other models, the PL011 is used as the primary UART. By default, `/dev/ttyS0` refers to the mini UART and `/dev/ttyAMA0` refers to the PL011. The Linux console uses the primary UART which depends on the Raspberry Pi model used. Also, if enabled, `/dev/serial0` refers to the primary UART (if enabled), and if enabled, `/dev/serial1` refers to the secondary UART.

By default, the primary UART (serial0) is assigned to the Linux console. Using the serial port for other purposes requires this default configuration to be changed. On startup, `systemd` checks the Linux kernel command line for any console entries and will use the console defined therein. To stop this behaviour, the serial console setting needs to be removed from the command line. This is easily done by using the **sudo raspi-config** utility by selecting option **5** (Interfacing Options) and then **P6** (Serial), and select **No** to the question **Would you like a login shell to be accessible over serial?** and then **Yes** to the question **Would you like the serial port hardware to be enabled?**. Click **OK** and Exit **raspi-config** and restart your Raspberry Pi (note: do not forget to re-enable the console serial port after you finish your project).

On Raspberry Pi 3 and 4, the serial port (**/dev/ttyS0**) is routed to two pins GPIO14 (TXD) and GPIO15 (RXD) on the header. This port is stable and of a good quality. This project is based on the Raspberry Pi 4.

To search for available serial ports on your Raspberry Pi, use the command:

```
pi@raspberrypi:~$ dmesg | grep tty
```

Figure 9.52 shows the display on the Raspberry Pi 4.

```
pi@raspberrypi:~$ dmesg | grep tty
[ 0.000000] Kernel command line: coherent_pool=1M 8250.nr_ua
rts=1 cma=64M
=256M smsc95xx.macaddr=DC:A6:32:00:E4:28 vc_mem.mem_base=0x3ec00000 vc_mem.
size=0x40000000 dwc_otg.lpm_enable=0 console=tty1 console=ttyS0,115200 root
TUID=7a119b46-02 rootfstype=ext4 elevator=deadline fsck.repair=yes rootwait
et splash plymouth.ignore-serial-consoles
[ 0.000259] console [tty1] enabled
[ 0.821856] fe201000.serial: ttyAMA0 at MMIO 0xfe201000 (irq = 33, base_b
= 0) is a PL011 rev2
[ 0.825058] console [ttyS0] disabled
[ 0.825090] fe215040.serial: ttyS0 at MMIO 0x0 (irq = 36, base_baud = 625
0) is a 16550
[ 0.825143] console [ttyS0] enabled
[ 2.264171] systemd[1]: Created slice system-serial\x2dgetty.slice.
pi@raspberrypi:~$
```

Figure 9.52 Available serial ports

The last line in the output is:

```
console [ttyS0] enabled
```

which indicates that the console is enabled on serial port **ttyS0**

After disabling the serial console, you should have the display shown in Figure 9.53 when the command **dmesg | grep tty** is entered. We can now use the serial port to read data from the RFID reader.

```
pi@raspberrypi:~$ dmesg |grep tty
[ 0.000000] Kernel command line: coherent_pool=1M 8250.nr_ua
rts=1 cma=64
=256M smsc95xx.macaddr=DC:A6:32:00:E4:28 vc_mem.mem_base=0x3ec00000 vc_mem.
size=0x40000000 dwc_otg.lpm_enable=0 console=tty1 root=PARTUUID=7a119b46-0
tfstype=ext4 elevator=deadline fsck.repair=yes rootwait quiet splash plymou
nore-serial-consoles
[ 0.000258] console [tty1] enabled
[ 0.821584] fe201000.serial: ttyAMA0 at MMIO 0xfe201000 (irq = 33, base_
= 0) is a PL011 rev2
[ 0.824836] fe215040.serial: ttyS0 at MMIO 0x0 (irq = 36, base_baud = 62
0) is a 16550
pi@raspberrypi:~$
```

Figure 9.53 Console serial port is disabled

Figure 9.54 shows the program listing (program: **carprk.py**). The program consists of 3 processes with the names: **PRESSURE**, **RFID**, and **STEPPER**. In this program, the processes exchange data with each other using the multiprocessing Value as described in an earlier Chapter.

At the beginning of the program, the modules used in the program are imported into the program. Note that `Value` is imported as **from multiprocessing import Value**. The **traceback** limit is set to zero so that error messages are not generated when the program is terminated using the **Ctrl+C** keys.

The operation of the processes are described below using PDL (Program Description Language):

Process STEPPER

- Define stepper motor connections to Raspberry Pi
- Define stepper motor parameters
- Configure stepper motor connections as outputs
- Define the Half-Step sequence

DO FOREVER

- Wait for event flag **e** to be set
- Clear event flag **e**
- Open the barrier (lift it)
- Wait 15 seconds
- Close the barrier (lower it)

ENDDO

Process PRESSURE

- Define connection between pressure sensor switch (button) and Raspberry Pi
- Configure the button as input

DO FOREVER

- Wait until the button is pressed
 - Get the number of space count
 - Increment the space count
 - Save the space count
- Wait until the button is released

ENDDO

Process RFID

- Define RED and GREEN LED interfaces
- Configure the LEDs as outputs
- Turn ON RED LED
- Turn OFF GREEN LED
- Open the serial port
- Define the valid RFID tags (only 3 are used)

DO FOREVER

Read RFID tag code

IF this is a valid code and there are spaces in the car park **THEN**

Decrement the space count by one

Save the space count

Set event flag **e** (activate stepper motors)

Turn OFF RED LED

Turn ON GREEN LED

ENDIF

Wait 5 seconds

ENDDO

The main program displays the space count on the LCD where the display is updated every second.

A typical operation cycle of the car park is as follows:

On entry:

- Car park space count is displayed on the LCD
- Red LED is ON, Green LED is OFF
- A car approaches the car park
- Member places the RFID tag close to the RFID reader
- If the card is valid then it is accepted
- Car park space count is decremented by one
- The barrier is lifted up
- Green LED is ON, Red Led is OFF
- A car enters the car park

On exit:

- A car is on the pressure switch at the exit
- Increment the space count by one
- Wait until the car leaves

LCD:

- Display the car park space count

In this program, 3 valid RFID tags are used for demonstration purposes. Notice the card reader returns 14 bytes, but only the bytes that represent the number on the cards are extracted and used in the program.

In this program, the data exchange between the processes is done using the multiprocessing **Value** objects, instead of using multiprocessing queues which was the case in earlier multiprocessing projects. The arguments of the processes that are required to exchange data are set to **val**. The processes themselves are created with an argument called **n**. The statement **n.value** is then used to exchange the space count (variable **spaces**) between processes.

```
#-----
#                               CAR PARK MANAGEMENT SYSTEM
#                               =====
#
# This is a car park control system. Vehicles whose owners have valid
# RFID cards can enter the car park after a barrier opens with the
# help of a stepper motor at the entrance. A green LED shows when it
# is safe to enter the car park. An LCD displays the number of free
# spaces inside the car park
#
# Author: Dogan Ibrahim
# File  : carprk.py
# Date  : June 2020
#-----

import RPi.GPIO as GPIO                # Import RPi
import RPi_I2C_driver                  # Import LCD driver
import multiprocessing                  # Import multiprocessing
from multiprocessing import Value       # Import Value
import time                            # Import time
import serial                           # Import serial
import sys

sys.tracebacklimit=0
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)                 # GPIO mode BCM
LCD = RPi_I2C_driver.lcd()             # LCD driver

e = multiprocessing.Event()            # Create event

#
# This function sends pulses to the motor in Half-Step mode
#
def SendPulse(k):
    global IN1,IN2,IN3,IN4              # HalfStep
    GPIO.output(IN1, HalfStep[k][3])    # Send to IN1
    GPIO.output(IN2, HalfStep[k][2])    # Send to IN2
    GPIO.output(IN3, HalfStep[k][1])    # Send to IN3
    GPIO.output(IN4, HalfStep[k][0])    # Send to IN4

#
# This function rotates the stepper motor clockwise by specified degrees
# (90 degrees to close the barrier)
#
def BarrierDown(degrees):
    global StepsPerCycle, StepDelay
    DegreeTurn = StepsPerCycle * degrees / 360.0
    d = int(DegreeTurn)
```

```
        for m in range(0, d):                # Do for req degrees
            for i in range(0, 8):              # Do for all steps
                k = 7 - i
                SendPulse(k)                   # Send pulse
                time.sleep(StepDelay)          # Delay

#
# This function rotates the motor anticlockwise by specified degrees
# (90 degrees to open the barrier)
#
def BarrierUp(degrees):
    global StepsPerCycle, StepDelay
    DegreeTurn = StepsPerCycle * degrees / 360.0
    d = int(DegreeTurn)

    for m in range(0, d):                    # Do for req degrees
        for i in range(0, 8):                # Do for all steps
            SendPulse(i)                     # Send pulse
            time.sleep(StepDelay)             # Delay

#
# Process to control the stepper motor. The motor is rotated clockwise
# or anticlockwise to close or open the barrier respectively
#
def STEPPER():                               # Process STEPPER
    global IN1,IN2,IN3,IN4,RPM,StepsPerCycle,StepDelay,HalfStep
    IN1 = 23                                 # IN1 is at GPIO 23
    IN2 = 24                                 # IN2 is at GPIO 24
    IN3 = 25                                 # IN3 is at GPIO 25
    IN4 = 8                                  # IN4 is at GPIO 8

    RPM = 10                                # RPM
    StepDelay = 0.014648 / RPM               # Step delay
    StepsPerCycle = 512                     # Steps per cycle

    GPIO.setup(IN1, GPIO.OUT)                # IN1 is output
    GPIO.setup(IN2, GPIO.OUT)                # IN2 is output
    GPIO.setup(IN3, GPIO.OUT)                # IN3 is output
    GPIO.setup(IN4, GPIO.OUT)                # IN4 is output

#
# Define Half-Step sequence
#
    HalfStep = [[1,0,0,0],
                 [1,1,0,0],
```

```
[0,1,0,0],
[0,1,1,0],
[0,0,1,0],
[0,0,1,1],
[0,0,0,1],
[1,0,0,1]]

#
# Motor control loop
#
    while True:                                # Do forever
        e.wait()                                # Wait for event
        e.clear()                              # Clear event
        BarrierUp(90)                          # Rotate clockwise
        time.sleep(15)                         # Wait 15 secs
        BarrierDown(90)                        # Barrier down

#
# Process PRESSURE. This process scans the pressure button (switch)
# and if a vehicle leaves the car park then the spaces count is
# incremented by one
#
def PRESSURE(n):                                # Process PRESSURE
    Button = 16                                # Button at 16
    GPIO.setup(Button, GPIO.IN)                # Button is input

    while True:                                # Do forever
        while GPIO.input(Button) == 1:        # Button not pressed
            pass
        spaces = n.value                      # Get spaces
        spaces = spaces + 1                   # Increment spaces
        n.value=spaces                        # Save spaces
        while GPIO.input(Button) == 0:        # Button not released
            pass

#
# Process RFID
#
def RFID(n):                                    # Process RFID
    RED = 21                                  # RED LED at 21
    GREEN = 20                                # GREEN LED at 20
    GPIO.setup(RED, GPIO.OUT)                 # LED is output
    GPIO.setup(GREEN, GPIO.OUT)               # LED is output
    GPIO.output(RED, 1)                       # Turn ON RED LED
    GPIO.output(GREEN, 0)                     # Turn OFF GREEN LED
    RFID = serial.Serial('/dev/ttyS0')        # Conf serial port
```

```
Tagitems = 3                                # 3 valid tags
Tags = ("007369087E",
        "2E00736A0A",
        "00E38F0998")

while True:                                # Do forever
    if not RFID.is_open:
        RFID.open()
    Data = ""
    header = RFID.read()                    # Read tag header
    if header == b'\x02':                    # Header read
        for i in range(10):                 # Read tag code
            rfid_data = RFID.read()
            d = rfid_data.decode("utf-8")
            Data = Data + d                  # Tag code in Data
        x=RFID.read(3)                       # Read remainder
        i = 0
        while (i < Tagitems) and (Data != Tags[i]):
            i = i + 1
        if i < Tagitems:                    # Valid tag
            spaces = n.value                 # Tag is valid
            if spaces > 0:                   # If there are spaces
                spaces = spaces - 1          # Decrement spaces
                n.value=spaces               # Save spaces
                e.set()                      # Open/close barrier
                GPIO.output(RED, 0)          # RED LED OFF
                GPIO.output(GREEN, 1)        # GREEN LED ON
                time.sleep(15)               # Wait 15 secs
                GPIO.output(RED, 1)          # RED LED ON
                GPIO.output(GREEN, 0)        # GREEN LED OFF
            RFID.close()                     # To flush the serial
            time.sleep(5)                    # Wait 5 secs

val=Value('i',0)                            # Shared integer

#
# Create the process
#
p = multiprocessing.Process(target = PRESSURE, args = (val,))
r = multiprocessing.Process(target = RFID, args = (val,))
s = multiprocessing.Process(target = STEPPER, args = ())
p.start()
r.start()
s.start()

#
```

```
# Display the number of free spaces on the LCD. First row of the LCD
# displays "Free Spaces", while the second row displays "Spaces:nn"
# where nn is the number of free spaces in the car park
#
TotalCapacity = 100                                # Total capacity
Spaces = TotalCapacity
LCD.lcd_clear()                                     # Clear LCD
LCD.lcd_display_string("Free Spaces:", 1)           # Display heading
val.value=Spaces                                    # Save spaces

try:
    while True:                                     # Do forever
        Spaces=val.value                           # Get spaces
        s = "Spaces:" + str(Spaces) + " "          # Add text
        LCD.lcd_display_string(s, 2)               # Display text
        time.sleep(1)                              # Wait 1 sec

except KeyboardInterrupt:
    GPIO.cleanup()
    print("End of program")
```

Figure 9.54 Program: carprk.py

Figure 9.55 shows an example display of the LCD.



Figure 9.55 Example display of the LCD

Appendix A • List of components used in the book

- 4 x Button
- 4 x 10K resistor
- 2 x Red LED
- 2 x Orange LED
- 2 x Green LED
- 7 x 470 Ohm resistor
- 1 x Light Dependent Resistor (KY-018)
- 1 x 4 digit 7-segment common cathode LED display (e.g. 2 x DC56-11EWA)
- 4 x NPN transistors (e.g. BC108 or any NPN type)
- 4 x 1K resistors
- 1 x DHT11 temperature and humidity sensor
- 1 x small relay
- 1 x DS18B20 sensor (e.g. KY-001)
- 1 x small buzzer
- 1 x I²C LCD
- 1 x Ultrasonic module (e.g. HC-SR04)
- 1 x small stepper motor (e.g. 28BYJ-48)
- 1 x ULN2003 stepper motor driver
- 1 x 4x4 keypad
- 1 x RDM6300 RFID reader
- 1 x RFID tag
- 1 x small breadboard
- Female-male jumper wires
- Female-female jumper wires

Additionally:

- 1 x Raspberry Pi 4 processor with power supply
- 1 x +5V external power supply (1A)

Appendix B • Raspberry Pi 4 pin configuration

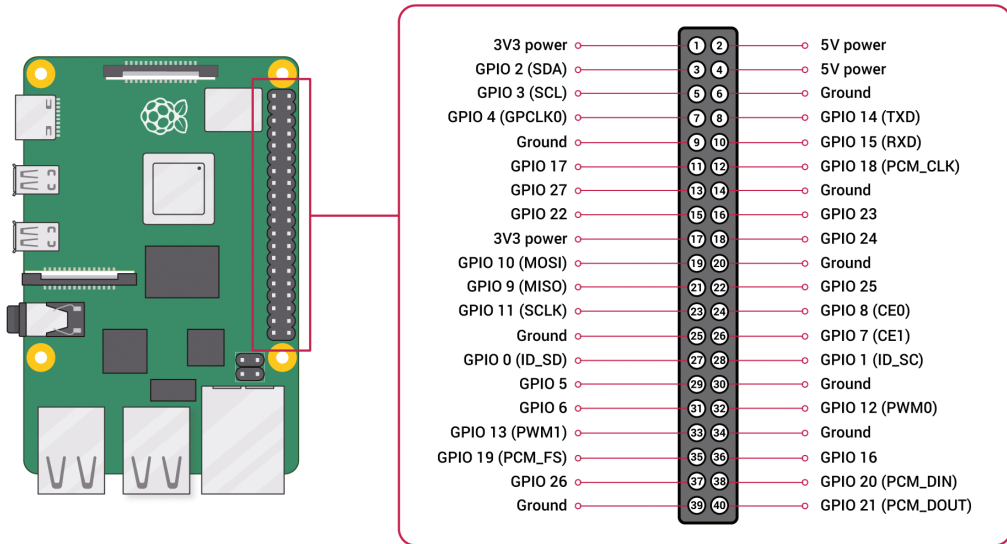


Figure A.1 Raspberry Pi 4 pin configuration

Index

A

Anonymous pipe 164

B

Background process 36

Bipolar stepper motor 194

C

cat 26

check_call 156

check_output 157

clear 152

common anode 103

common cathode 103

Conveyor belt 131

Co-operative scheduling 54

cp 29

CPU architecture 51

CPU utilization 46

crontab 39

43

D

date 31

DHT11 112

Disk usage 49

dpkg 32

Duty cycle 116

Dynamic priority scheduling 60

E

echo 31

Etcher 13

Events 151

exec 64

F

file 27

fg 36

First come, first served 59

foreground process 36

Full-step mode 197

G

get 164

grep 32

H

Half-step mode 197

halt 34

HC-SR04 182

help 28

history 33

I

I2C LCD 139

is_set 152

J

jobs 36

K

Keypad 203

kill 49

killall 68

KY-001 137

L

LDR 131

lock 149

ls 21

M

man 28

Memory usage 46

Metronome 171

mkdir 22

Multiple threads 148

Multilevel queue scheduling 60

Multiprocessing 161

Multitasking event counter 74

mv 30

N

Named pipe 164

P			
Pipes	164	TightVNC	19
popen	61	TightVNC viewer	19
Process fork	62	Timer	153
Process table	46	top	45
ps	47	Traffic lights	175
Pre-emptive scheduling	54	U	
Pressure switch	215	ULN2003	196
put	164	Ultrasonic	181
Putty	16	Unipolar stepper motor	192
pwd	21	Up/down counter	80, 93
PWM	117	V	
Q		VNC	18
Queues	164	VNC server	19
R		W	
Raspbian Buster	12	wait	152
raspi-config	15	Wildcard	30
RDM6300	216		
Relay	210		
RFID reader	215		
RFID tag	217		
Resource monitoring	45		
rmdir	30		
Round-robin scheduling	54		
S			
SCL	139		
SDA 139			
Sharing data	164		
Semaphore	150		
set	152		
Seven segment LED	101		
126			
Square waveform	116		
SSH	18		
Stepper motor	191		
Subprocess	156		
Synchronizing parent and child	78		
T			
Task scheduling	37		
Temperature controller	136, 167		
Threading	146		
Threads	84		

Multitasking with Raspberry Pi

Dogan Ibrahim



Prof. Dr. Dogan Ibrahim is a Fellow of the Institution of Electrical Engineers. He is the author of over 60 technical books, published by publishers including Wiley, Butterworth, and Newnes. He is the author of over 250 technical papers, published in journals, and presented in seminars and conferences.

Multitasking and multiprocessing have become a very important topic in microcontroller-based systems, namely in complex commercial, domestic, and industrial automation applications. As the complexity of projects grows, more functionalities are demanded from the projects. Such projects require the use of multiple inter-related tasks running on the same system and sharing the available resources, such as the CPU, memory, and input-output ports. As a result of this, the importance of multitasking operations in microcontroller-based applications has grown steadily over the last few years. Many complex automation projects now make use of some form of a multitasking kernel.

This book is project-based and its main aim is to teach the basic features of multitasking using the Python 3 programming language on Raspberry Pi. Many fully tested projects are provided in the book using the multitasking modules of Python. Each project is described fully and in detail. Complete program listings are given for each project. Readers should be able to use the projects as they are, or modify them to suit their own needs.

The following Python multitasking modules have been described and used in the projects: • Fork • Thread • Threading • Subprocess • Multiprocessing

The book includes simple multitasking projects such as independently controlling multiple LEDs, to more complex multitasking projects such as on/off temperature control, traffic lights control, 2-digit, and 4-digit 7-segment LED event counter, reaction timer, stepper motor control, keypad based projects, car park controller, and many more. The fundamental multitasking concepts such as process synchronization, process communication, and memory sharing techniques have been described in projects concerning event flags, queues, semaphores, values, and so on.

ISBN 978-1-907920-96-7



Elektor International Media BV
www.elektor.com